Kubernetes as a Virtual Machine Management System

A Comparative Study of KubeVirt and Traditional Approaches

EMIL KARLSSON

Master in Computer Science
Date: April 24, 2024
Supervisor: Han Fu
Examiner: Cyrille Artho
School of Electrical Engineering and Computer Science
Host company: Net Insight AB
Swedish title: Swedish title TBD

Abstract

Sammanfattning

Contents

1 Introduction **2**
1.1 Research Questions 6
1.2 Problem 6
1.3 Purpose 6
1.4 Goals 7
1.5 Research Method 7
1.6 Limitations 7
1.7 Structure of the thesis 8

2 Background **9**
2.1 Virtualization Fundamentals 9
2.2 Understanding Hypervisors 12
2.3 VM Management Essentials 15
2.3.1 Comparison of VM Management Systems 19
2.4 Containers and Kubernetes: A Paradigm Shift 22
2.4.1 Containers Explained 23
2.4.2 Orchestration Using Kubernetes 25
2.5 Containers vs. Virtual Machines 28
2.5.1 Comparative Overview 29
2.5.2 Decision Factors 32
2.5.3 Towards Stronger Container Isolation 33
2.6 Extending Kubernetes with KubeVirt 34
2.6.1 KubeVirt Technical Overview 36
2.6.2 KubeVirt and GitOps 37
2.6.3 Comparison with Traditional Approaches 37
2.7 Related Work 38
2.7.1 Current challenges 38
2.7.2 KubeVirt challenges 39
2.7.3 Usability of Kubernetes and KubeVirt 39

2.7.4 Performance Evaluations 41

3 Performance Analysis 43
3.1 Setup 43
3.2 Building a Framework 46
3.2.1 Design 46
3.2.2 Implementation 50

4 Usability Analysis 54
4.1 Setup 54
4.2 Task Design 55
4.3 Conducting the Usability Analysis 56

5 Results 57
5.1 Performance Analysis 58
5.2 Usability Analysis 69

6 Discussion 71
6.1 Limited Features and Result Intepretation 71
6.2 Kubernetes Overhead and Induced Load 73
6.3 Jagged Disk Utilization 75

7 Conclusions and Future Work 76

Bibliography 77

A Performance Analysis Components 89
A.1 VM Management System Interface 89
A.2 Metrics Collector 89
A.3 Metrics Collector Daemon 91

B Performance Analysis Results 92

C Usability Tasks 117

D Usability Analysis Answers 130
D.1 Participants 130
D.2 Task 1/3 130
D.3 Task 2/3 133
D.4 Task 3/3 134
D.5 Summary 136

;

Chapter 1

Introduction

Virtualization is a technique that either splits or combines computing resources into one or more operating environments by creating a software layer that surrounds or underlies existing hardware or software. This enables the creation of virtualized resources that are decoupled from the underlying hardware. Virtualization can occur in various forms including the virtualization of storage, networks and servers. Virtualizing networks and storage is achieved by creating an abstraction layer on top of the physical network and storage resources, respectively, to provide a more flexible and scalable environment for other resources to interact with. Server virtualization, on the other hand, is used to create virtual computing environments that are decoupled from the underlying computing resources. By combining virtualized resources into a single operating environment, virtualization allows heterogeneous hardware resources to be used homogeneously and thus it improves resource utilization and flexibility [1, 2, 3].

To enable server virtualization and manage virtual computing environments, a hypervisor is used. A hypervisor is a software layer that operates between the hardware and the operating system (OS) and is responsible for creating virtual computing environments called virtual machines (VMs) [3, 4, 5]. VMs are emulations of physical computers that are perceived as real computers by applications running inside them. They provide a complete and independent environment by virtualizing an OS, rendering them suitable for a wide range of applications, including legacy applications and applications that require a high level of isolation. For this reason, VMs have long been the core part of IT infrastructure, serving as foundational components in system architecture [1, 2].

In parallel with the evolution of VMs, containers have emerged as an alter-

native to VMs. Container virtualization is an OS-level virtualization technique that allows for running multiple isolated applications on a single OS instance. This means container virtualization does not require a hypervisor, and instead uses the features and capabilities of the OS kernel to create and manage containers. As such, containers are more lightweight, require less overhead, and have faster startup times than VMs [6]. This has made containers a popular choice for scenarios in which scaling and portability are key, such as in microservices, which are core components in modern cloud-native applications [7]. Figure 1.1 visualizes the differences between how VMs and containers are virtualized, and how the virtualization layer is placed on a lower level for VMs than for containers.

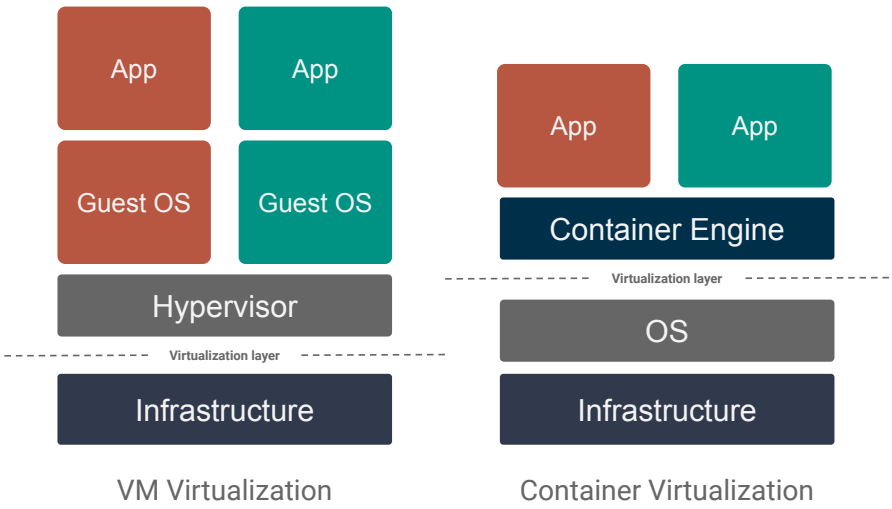

Figure 1.1: Overview of VMs and containers. The virtualization layer is placed on a lower level for VMs than for containers, illustrating the difference in their respective virtualization techniques.

However, while containers provide a lightweight and efficient alternative to VMs, they lack the deep isolation properties inherent to VMs. Sharing the OS kernel with the host and other containers opens up a larger attack surface, which implies that containers are more prone to security vulnerabilities. While there exist technologies that aim to unify the isolation benefits of VMs and the lightweight benefits of containers, such as Kata Containers, gVisor and Firecracker, it remains a fact that both containers and VMs have their use cases [8, 9].

Due to their different use cases, VMs and containers have traditionally been managed separately. VMs are often managed by a VM management system

that provides a set of tools and processes to aid in the life-cycle management of VMs. This makes a VM management system act as a higher-level abstraction layer that unifies hypervisors, typically through an application programming interface (API) such as a command line interface (CLI) or a web-based user interface. Figure 1.2 depicts a broad overview of the role of a VM management system where clients interact with an interface to manage VMs that are distributed across several hosts. Moreover, a VM management system is often part of a larger infrastructure, such as Infrastructure as a Service (IaaS), that provides a wider range of virtualized resources including virtual networks and storage. Examples of such systems include OpenStack [10], CloudStack [11], and OpenNebula [12]. Containers, on the other hand, are often managed by a container orchestration platform. These platforms provide a similar set of tools and processes as a VM management system but for containers. Apart from creating and deploying containers, a container orchestration platform provides tools specifically designed to manage the life cycle of containers, such as scheduling and scaling. Examples of such platforms include Kubernetes [13], Nomad [14], and Docker Swarm [15], with the former being the most widely adopted [16]. Furthermore, with the evolution and adaption of container orchestration platforms like Kubernetes, there has been an ongoing convergence in how VMs and containers are managed [17].

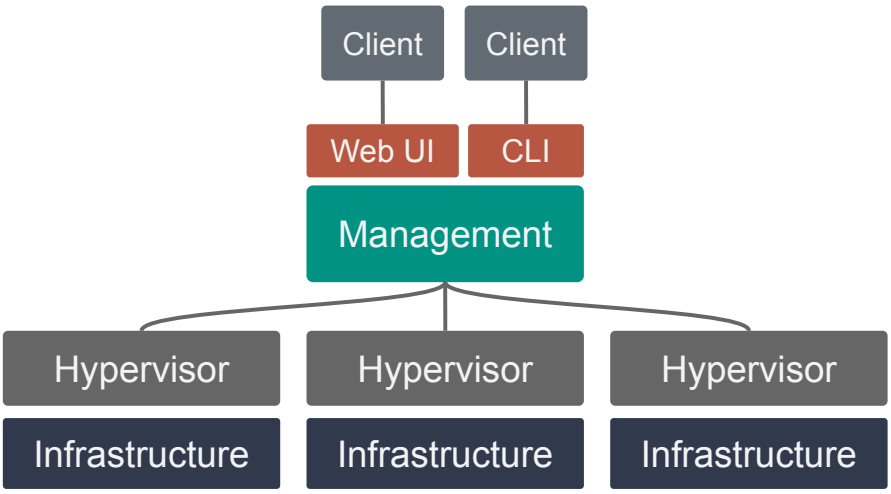

Figure 1.2: Brief overview of a monolithic VM management system where clients communicate with a management node through a set of interfaces to manage VMs distributed across several hosts. The indirect communication allows a client to manage VMs without having to interact with the hosts directly.

Kubernetes, originally designed to manage containerized applications at

scale, has recently been extended to accommodate VMs through the KubeVirt project. KubeVirt is an API extension for Kubernetes, that enables the running of VMs inside a Kubernetes environment alongside containers, creating a common environment for both virtualization technologies [18]. Figure 1.3 depicts a high-level overview of how VMs and containers coincide in a Kubernetes environment using KubeVirt, where traffic flows natively between the two types of resources. Further technical details of how KubeVirt achieves this will be discussed in 2.6.1.

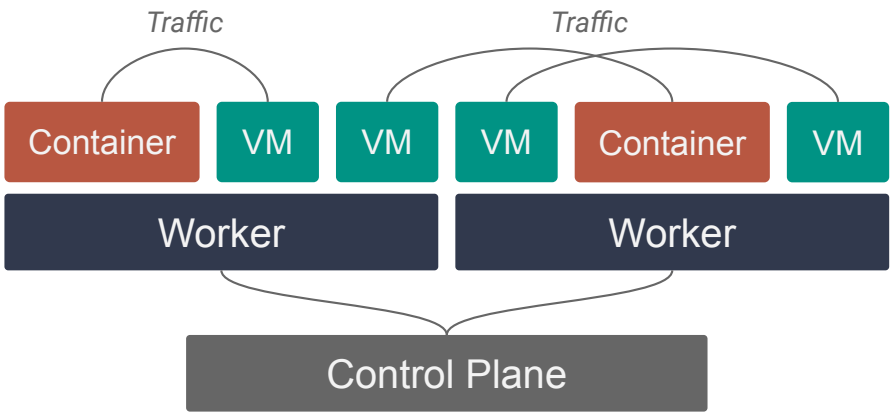

Figure 1.3: Overview of VMs and containers managed by using KubeVirt. The figure illustrates how VMs and containers communicate natively in a single unified environment.

The approach of incorporating VMs in Kubernetes not only benefits from the flexible, scalable and mature infrastructure that Kubernetes provides but also from tools and processes that are designed to integrate with or manage the life cycle of Kubernetes resources. This means by using KubeVirt, VMs can be managed similarly to containers, creating a more cloud-native approach to VM management [7]. Furthermore, consolidating the management of VMs and containers into a single platform could offer benefits in terms of reduced architectural complexity and simpler hardware management, but also practical aspects such as reduced required skill set in an organization [19, 20].

The integration of VM management into Kubernetes through KubeVirt represents a significant milestone toward a more agile, efficient, and unified IT infrastructure [20]. However, KubeVirt's practicality in a real-world scenario is yet to be fully assessed. As such, this project aims to evaluate Kubernetes with KubeVirt as a VM management system and compare it with traditional approaches to understand its practicality and limitations.

1.1 Research Questions

What performance, scalability, and operational efficiencies can be gained by integrating VMs in Kubernetes?

This could stem from using a single resource scheduler. Several metrics will be used to evaluate this, as described in the background and related work.

What are the current challenges and limitations in managing VMs and containerized applications in separate environments?

This is concerned with scenarios in which a traditional VM management system is operated alongside Kubernetes.

How can system administration be simplified by running VMs in Kubernetes using KubeVirt?

This is focused on the practical implications of using KubeVirt and Kubernetes instead of traditional VM management systems. For instance, the number of steps required to deploy a VM.

1.2 Problem

Due to the different use cases for VMs and containers, they have traditionally been managed separately. However, both VMs and containers are still part of the necessary infrastructure many organizations depend on. This has led to fragmented infrastructure with separate management systems and tools to manage VMs and containers with a direct implication of increased architectural complexity and a larger required skill set for operators. Furthermore, many cloud-native technologies are designed for Kubernetes, and as such, by running VMs outside of Kubernetes, these benefits might not be utilized.

1.3 Purpose

The purpose of this project is to evaluate if Kubernetes, a platform originally designed for container orchestration, is a suitable platform for managing VMs using KubeVirt. This evaluation is important in the field of cloud computing, as it could provide insights into how the management of VMs and containers

could be both simplified and unified, but also how the benefits of Kubernetes can be utilized for VMs.

1.4 Goals

This project aims to explore the integration of VMs into Kubernetes environments using KubeVirt. By comparing how VMs are managed using KubeVirt against traditional VM management systems, this project aims to address possible inefficiencies and complexities that arise when managing VMs outside of Kubernetes. The project will investigate the implications of moving into a KubeVirt-based system, such as the benefits and limitations of using KubeVirt, and how it compares to traditional VM management systems.

1.5 Research Method

This project will use a mixed-methods approach to investigate the research questions. Initially, a literature review will be carried out to understand the current state of the art in VM management systems and the current challenges faced and limitations in managing VMs and containers in separate environments. Following this, both a qualitative and quantitative comparison will be conducted to understand the practical implications of moving into a KubeVirt-based system. The qualitative comparison will be conducted to understand the practical implications of moving into a KubeVirt-based system. It will include collecting feedback by having system administrators and operators with experience in both traditional VM management systems and Kubernetes explore Kubevirt. The quantitative comparison will be conducted to evaluate measurable performance, scalability, and operational efficiencies that could arise when using Kubernetes with KubeVirt, and will include a set of benchmarks.

1.6 Limitations

1. This project will not investigate any solutions for shortcomings in KubeVirt itself, such as finding alternatives for features that exist in existing virtual machine management systems but not in KubeVirt. As such, this project acknowledges that KubeVirt might not be a complete replacement for traditional virtual machine management tools since it might not offer all the features of an IaaS system. Therefore, only features that

exist in KubeVirt and the compared system will be considered when testing.

2. This project will primarily focus on the practical implications of moving into a KubeVirt-based system. For this, there are mainly two scenarios that will be considered:

   - **Replacement**: A system uses only a traditional virtual management system. By using KubeVirt, the system will be replaced by a Kubernetes-based system.
   - **Unifying**: A system uses both traditional management and Kubernetes. By using KubeVirt, the system will be unified into a single platform for both VMs and containers.

1.7 Structure of the thesis

Chapter 2

Background

This chapter will discuss the relevant background information and concepts that are necessary to understand why KubeVirt has emerged and how it fits into the current landscape of VMs and containers. It is structured linearly, starting with a detailed discussion of virtualization, hypervisors and VMs, followed by a comparison of VM management systems. It then introduces containers and Kubernetes, and how they have emerged as an alternative to VMs, and finally discusses the integration of VMs into Kubernetes through KubeVirt.

2.1 Virtualization Fundamentals

Virtualization is a technology that enables partitioning or grouping computing resources into operating environments using methods such as partitioning, emulation, and time-sharing [2]. Virtualization can occur in various forms including the virtualization of storage, networks and servers. Storage virtualization is achieved by creating an abstraction layer on top of physical storage resources to provide a single interface to storage resources that appear as a single storage device. This allows for a more flexible environment where storage resources can be added or removed without affecting the application layer. Similarly, network virtualization enables the creation of networks with the properties of a physical network, but with its appliances such as routers and switches being implemented in software. This allows for flexibility when managing resources in a network since common practices such as networking partitioning and isolation can be achieved without acquiring additional hardware. Server virtualization, on the other hand, is concerned with creating virtual computing environments that are decoupled from the underlying computing resources. It is often referred to as hypervisor-based virtualization, as

it requires a software layer called a hypervisor that is responsible for creating and managing the virtual computing environments [1].

A virtual computing environment, more often referred to as a virtual machine (VM), is an emulation of a physical computer and is perceived as a *real* computer by the application running inside it even though the underlying resources are often shared with other VMs. To enable this, a hypervisor divides the hardware into virtualized resources, such as virtual CPUs and virtual memory, and provides an interface to manage these resources. For instance, an application running inside a VM is unaware that the resources it is using are virtualized, and instead interacts with the OS and hardware as if it were running on a physical machine. This is particularly useful for scenarios in which hardware is under-utilized, as it allows for the consolidation of workloads onto a smaller number of physical machines, which in turn reduces the need for acquiring additional hardware. [2, 3].

Depending on the required level of abstraction, virtualization can be achieved in different layers, where the three major being hardware-level, OS-level and application-level. For instance, storage and network virtualization both fall under the category of hardware virtualization, as they create an abstraction layer on top of the physical resources. However, in other scenarios, virtualization is not required to be as low-level, such as in application virtualization. Application virtualization is instead concerned with encapsulating an application in a lightweight virtualized environment, such that it is perceived as being run on a specific platform. A common example is the Java Virtual Machine (JVM) that enables programs written in the programming language Java to be run on multiple operating systems [21]. Figure 2.1 presents an overview of the differences between the two more common layers of virtualization; hardware and OS virtualization. While the figure does not include application-level virtualization, it is important to note that the virtualization layers are not mutually exclusive and that they can be used in combination. For instance, combining hardware-level and OS-level virtualization creates a flexible environment where a guest process can run inside a guest OS [2].

Furthermore, virtualization can be divided into two categories, full virtualization and paravirtualization. Full virtualization is the traditional approach where the entire set of hardware is emulated, and the VM is presented with an indistinguishable image of the underlying hardware. This allows for complete decoupling of the VM from the underlying resources and does not require any modification to the guest OS [2, 22, 23]. Paravirtualization, on the other hand, is a subset of full virtualization that aims to improve VM performance by reducing the number of resources that are emulated. The VM is presented with

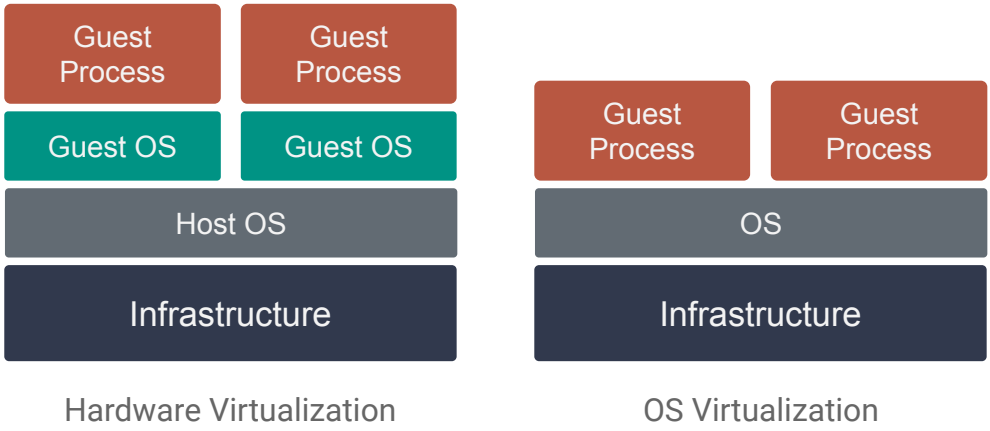

Figure 2.1: Overview of the different layers of virtualization for hardware-level and OS-level virtualization. The figure illustrates how hardware virtualization inhibits several internal OSes from running on the same hardware, while OS virtualization keeps a single OS for each process.

an image that is similar, but not identical, to the underlying hardware. As such, the guest OS is aware it is running inside a VM [22, 23].

Benefits and Challenges

Many of the benefits of virtualization stem from the fact that there exists an abstraction layer above the physical hardware. The abstraction layer decouples software from hardware, allowing for the use of vendors with different I/O in the same environment. This creates a more flexible and scalable computing environment as a whole where hardware is turned into a simple pool of computing resources [3]. As such, there are numerous benefits of virtualization, a few of which are listed below:

- Resource utilization: Virtualization allows for better utilization of hardware resources since multiple VMs can run on the same physical hardware. This is achieved by sharing the underlying resources between the VMs, such as CPU and memory [2].

- Isolation: VMs can isolate their entire state from one another, including the operating system (OS), filesystem and application stack. This is particularly useful for certain applications with stricter security requirements, such as applications managing sensitive or confidential data. Isolation is also beneficial in scenarios where reproducibility is key, such

as in testing and development environments [24].

- Flexibility: VMs are not tied to specific hardware, and can be moved between different physical resources. Flexibility is an especially useful factor when in scenarios where hardware is not fully utilized since VMs can be migrated to other physical machines to balance the load [2].
- Management: VMs can be created and destroyed on demand, and can be managed using software tools [2].
- Virtual hardware: A VM can be configured with hardware that does not exist in the physical environment, such as a virtual network card or a virtual hard drive. This allows for better support for legacy applications or certain applications that are dependent on a specific hardware configuration [2].

However, the portability made possible by virtualization comes at a cost of performance. Since a VM is a complete but emulated system, every instruction issued must be interpreted in software. This means that despite the level of abstraction layer used, there is an inescapable added overhead when using virtualization. A common hypervisor such as KVM (see 2.2) introduces an overhead of 5 to 10% [25], and the overhead can be even larger when using a full virtualization approach.

2.2 Understanding Hypervisors

A hypervisor is a software layer that enables managing multiple VMs on a single physical machine by allowing multiple operating systems (called *guests*) to share the same hardware [1, 5]. Hypervisors, often referred to as virtual machine monitors (VMMs) are responsible for creating and managing VMs and for providing the necessary abstractions to the VMs.

Hypervisors can be categorized into two types, *type 1* and *type 2*, based on their operational context and how they interact with the underlying hardware. While the goals and purpose of type 1 and type 2 hypervisors are the same, there are some core differences. Type 1 hypervisors, commonly referred to as *bare-metal* hypervisors, run directly on top of the physical hardware and do not require an OS underneath. A type 1 hypervisor is therefore scalable to terabytes of RAM and hundreds of CPU cores [26]. However, without any other additional software, it is also responsible to schedule and allocate required system resources to the VMs. Type 2 hypervisors, also known as *hosted hypervisors*, rely on an underlying OS to manage the calls to the CPU and memory

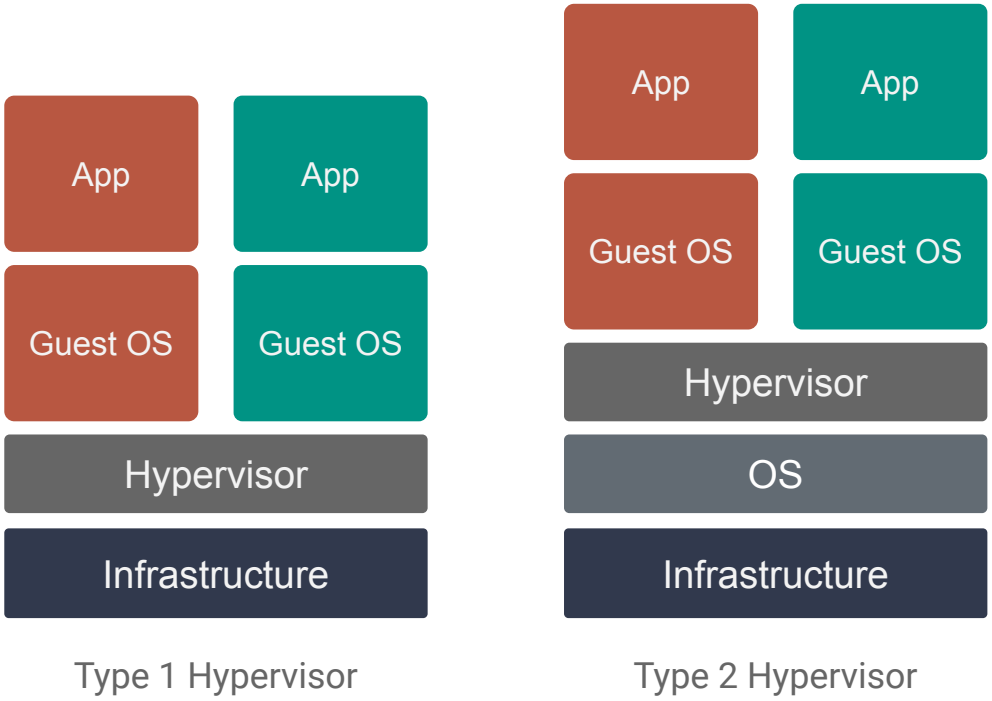

Figure 2.2: Overview of type 1 and type 2 hypervisors. The figure illustrates how a type 1 hypervisor does not use an underlying OS and instead runs directly on the hardware, while a type 2 hypervisor relies on an intermediary OS to manage the underlying hardware.

resources. The presence of an underlying OS inevitably introduces latency, which is why type 2 hypervisors are often used for development and testing purposes [5]. Figure 2.2 presents a high-level overview of the difference between a type 1 and type 2 hypervisor [5, 26].

Furthermore, hypervisors can be further categorized into *monolithic* and *microkernelized* hypervisors [27]. Monolithic hypervisors are built as a single large software stack, managing all the VMs and their resources, and hosting driving for all hardware access. Having all drivers reside within the hypervisor means that the hypervisor will only run on selected hardware. However, it also means no operating system is required at all, as the hypervisor acts as an operating system by supporting all the operations of its VMs. Microkernelized hypervisors, on the other hand, do not require device drivers to be present in the hypervisor. Instead, the drivers are installed in a form of managing VM, often referred to as parent VM. Each VM created by such a hypervisor is thus referred to as a child VM, as it uses the parent VM's drivers to access the

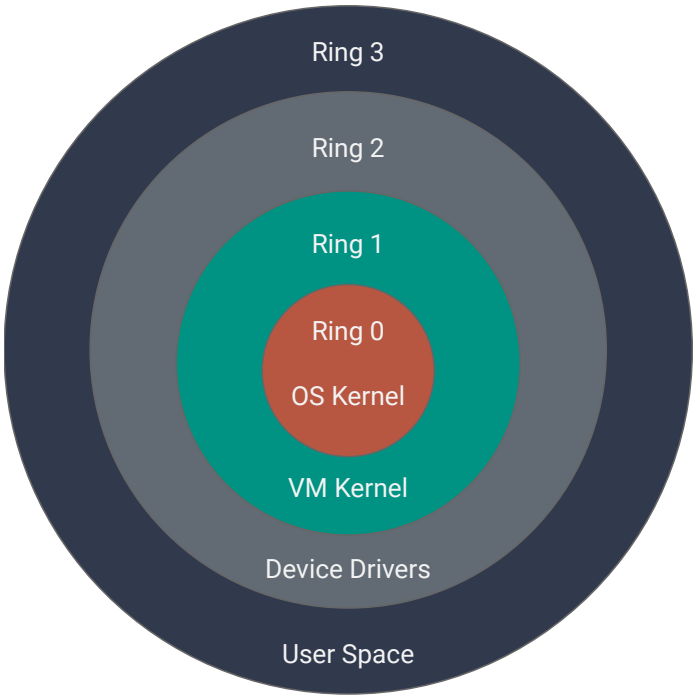

Figure 2.3: Overview of security rings in a hypervisor. The figure illustrates how the inner rings have higher privilege than the outer rings.

hardware. This approach is more flexible, as it can allow the hypervisor to run on any hardware that supports the parent VM's operating system.

In the realm of hypervisors, the concept of *security rings* is often used to describe the level of privilege different part of a hypervisor has. A security ring represents a hierarchical protection domain, where the innermost ring (ring 0), usually reserved for the OS kernel, has the highest privilege, and the outermost ring (ring 3), usually reserved for user applications, has the lowest privilege. The hypervisor itself is often placed in ring 0, its VM kernel space in ring 1, and the VM user space in ring 3. Security rings are thus used to define execution level to provide a system to limit the execution capabilities of VMs to regulate access to shared memory. This enabled a strong isolation layer between the VMs, and between the VMs and the hypervisor, thus a key part of a hypervisor's security model [27, 28]. Hypervisor security rings are visualized in figure 2.3.

Key Hypervisors

There are several hypervisors available, each with its own set of features and capabilities, some of which are listed below:

- VMware ESXi: A lightweight type 1 monolithic hypervisor often used in enterprise environments. ESXi is known for its ultra-lightweight footprint stretching merely a few hundred megabytes, and its ability to run directly on the hardware without an underlying OS [27].
- Microsoft Hyper-V: A type 1 microkernelized hypervisor that is part of the Windows Server operating system. As a microkernelized hypervisor, Hyper-V differentiates between parent and child VMs and uses the parent VM to manage hardware access for the child VMs [27, 29].
- Citrix Hypervisor: Formerly known as XenServer, is a type 1 microkernelized hypervisor based on the Xen Project. It runs using a modified Linux kernel in a VM, called Dom0, that manages hardware I/O and interacts with the other virtual machines, called DomU guests [23, 27].
- KVM: A type 1 hypervisor that is part of the Linux kernel. As such, KVM uses most of the functionalities from the Linux kernel including the scheduler and memory management. KVM has become a popular alternative to proprietary hypervisors, such as VMware ESXi and Microsoft Hyper-V since it is open-source and free to use [27, 30].

2.3 VM Management Essentials

A management system for VMs involves a set of tools and processes that are used to create, deploy, and manage VMs. The management system is responsible for managing the entire life cycle of a VM, including provisioning, deployment, monitoring, and scaling. Not be confused with a hypervisor, a VM management system operates on a higher level and aims to provide an abstraction layer in the form of an interface to allow interaction with the VMs, often enabling the support of a set of hypervisors [31]. As such, a VM management system unifies a set of hypervisors by allowing them to be managed as if they were homogeneous, typically through a higher-level API, such as a CLI or a web-based user interface. However, certain hypervisor features, such as live migration, could be implemented differently across hypervisors, and it is up to the VM management system to provide a unified interface to manage such features [32].

Concepts and Features

With a higher abstraction layer, a VM management system brings a set of concepts to simplify the management of VMs. Many such features and concepts exist and can be broadly grouped as clustering, availability, and scalability [31].

Clustering involves grouping a set of resources, such as VMs or storage nodes, to provide an image that is perceived as a single resource. By allowing multiple nodes to cooperate and share resources, clustering can allow for a more scalable and performant system. High-performance computing (HPC) is a common example of clustering where computing resources are aggregated to gain performance greater than that of a single computer. This enables HPC clusters to process large datasets and complex calculations that would take an infeasible amount of time to process on a single computer. Moreover, HPC clusters are designed such that they allow for the distribution of computational tasks where larger tasks are divided into smaller subtasks. However, the splitting of tasks is often not transparent to the end user, and the user perceives the cluster as a single resource [33, 34].

Availability refers to the ability to keep a resource active or running, which is, more often than not, clustered resources. A common practice for cloud computing is managing availability zones (AZs). An AZ groups resources such that each AZ is independent of the other AZs while providing the same service. This allows for outages to be isolated to a single AZ while the other AZs continue to provide the service. Thus, through redundancy and isolation, AZs enable high availability and fault tolerance [31, 35]. Figure 2.4 presents an overview of how 3 AZs provide redundant access to the same service.

Scalability involves technologies to ensure resources can manage an increase or decrease in computational loads. A VM management system often facilitates both scaling up and scaling out. Scaling up, also referred to as vertical scaling, involves increasing or decreasing the performance specification of a resource. For instance, a VM can be scaled up by increasing the number of CPUs or the amount of memory. Scaling out, also referred to as horizontal scaling, involves adding or removing entire resources, such as VMs. This means, that instead of upgrading a single VM, additional VMs are added to the system to handle the increased load. This approach is often regarded as more scalable than scaling up, as it allows for a more granular approach to resource management, and can be down without any downtime. Additionally, by avoiding downtime, the system appears as more available to the end user [36, 37]

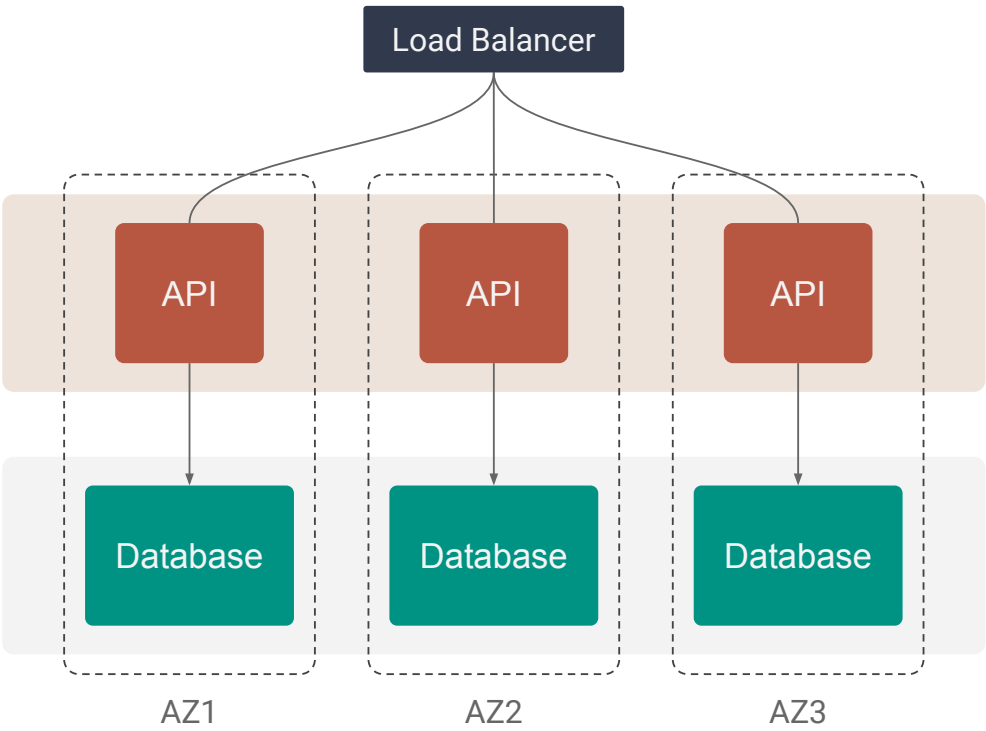

Figure 2.4: Overview of availability zones with several API gateways and databases operating in the same logical environment but in different availability zones to provide redundancy and fault tolerance.

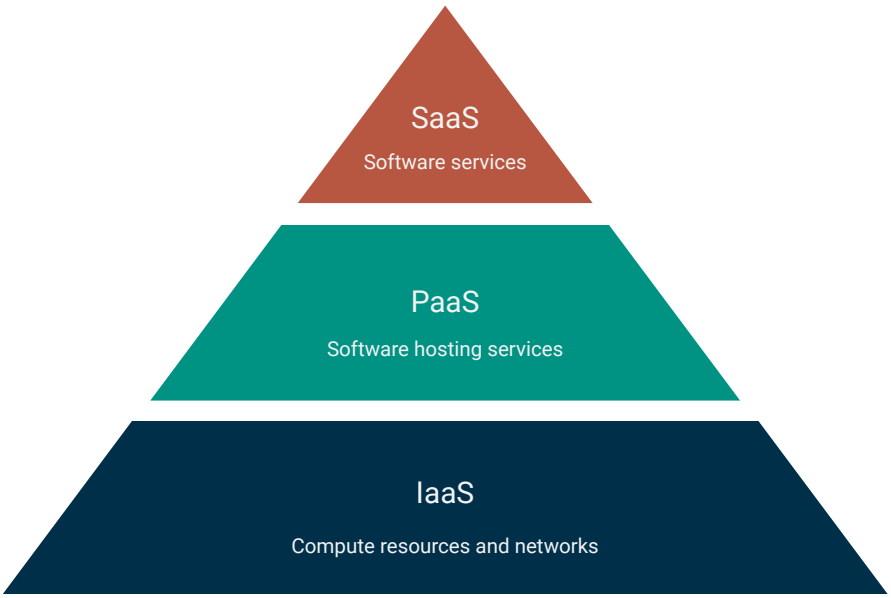

Figure 2.5: Overview of cloud service models. The figure is designed as a pyramid to illustrate the hierarchical relationship between the different cloud service models that allows a higher-level service to be built using a lower-level service.

Cloud Service Models

A VM management system is often part of a larger cloud computing infrastructure, such as Infrastructure as a Service (IaaS). IaaS is a lower-level cloud service model that aims to eliminate the need to install and manage physical hardware by providing virtualized computing resources, such as VMs, virtual networks and storage. As such, an IaaS platform provides a single interface to manage a wide range of virtualized resources that acts as a foundation for building and managing cloud environments [38].

Other cloud service models include Platform as a Service (PaaS) and Software as a Service (SaaS). They provide a higher level of abstraction than IaaS forming a hierarchical relationship with each other. This means a SaaS service can be built on top of a PaaS service, which in turn can be built on top of an IaaS service. While figure 2.5 presents an overview of these cloud service models, both PaaS and SaaS are out of the scope of this project [39].

2.3.1 Comparison of VM Management Systems

Managing VMs can be done in many ways, and there are numerous VM management systems available, each with its own set of features and capabilities. Many such systems are a part of a larger platform, such as an IaaS or even as large as a cloud platform. Some of the most prevalent open-source platforms that have been used in the industry are OpenStack, CloudStack, and OpenNebula, and some of the most popular public cloud platforms are Amazon EC2, Azure Virtual Machines, and Google Compute Engine.

OpenStack

An open-source IaaS platform that offers a large set of features to manage VMs, networks, and storage [32]. OpenStack's architecture is inherently modular, and it is designed to be deployed in a distributed manner [40]. The architecture includes three major parts:

- *Nova*: The compute service that manages VMs and provides an interface to manage VMs. Nova supports all operations regarding the life cycle of a VM, such as creating, deleting, and migrating VMs. Nova is itself divided into multiple components, each employing a *shared-nothing* and *messaging-based* policy, which means every component is independent and communicates with other components using a message queue.

- *Swift*: The object storage service that provides a scalable and redundant storage system for VMs. It is a long-term storage for static data, such as images and backups, and is comparable to Amazon S3.

- *Glance*: The image service that provides a catalog of VM images that can be used to create new VMs.

OpenStack supports features such as live migration, snapshot management, and scalability, and is often used in large-scale cloud environments [32]. However, these features are often hypervisor-specific and require both support from the hypervisor and specific configuration to work [41].

CloudStack

An open-source IaaS platform and cloud computing software that manages computing, networking and storage resources, and is used to build and manage public and private cloud environments [42]. On a high level, CloudStack is built in a monolithic fashion, encapsulating all the management features into

two software packages called the management server and agent. The management server is responsible for managing the life cycle of VMs, including provisioning, deployment, and scaling, and is comparable to a control node. The agent, however, is installed on each node designated to running VMs and is responsible for low-level management tasks of the VMs on that node, such as interacting with the underlying hypervisor. While it is a monolithic system, CloudStack is designed to be horizontally scalable and can be deployed in a distributed manner with multiple management servers [42].

CloudStack supports a large variety of hypervisors, such as VMware, KVM, and XenServer, and storage services, such as Amazon S3, allowing it to function as an abridgment layer for computing, network, and storage to unify a wide range of existing infrastructure [42]. Features such as live migration, snapshot management, and scalability are supported but are often hypervisor-specific and require specific configuration to work [43].

OpenNebula

An open-source IaaS platform and toolkit used to provide open, scalable, and comprehensive management of virtualized data centers [32]. OpenStack employs a modular architecture divided into three layers [44, 45]:

- *Tools layer*: Provides a set of interfaces such as a CLI and web-based user interface to interact with the system, but also high-level management tools such as scheduling.
- *Core layer*: Provides the core functionalities of the system, such as host management, VM management, and virtual network management.
- *Driver layer*: Provides an abstraction on top of the underlying infrastructure to enable the system to support a wide range of hardware, including networking, storage, and hypervisors. As such, the driver layer is responsible for allocation, deallocation and monitoring of resources, such as VMs and storage.

By dividing the system into these layers, OpenNebula provides modularity that enables the system to be extended and customized to fit specific needs, such as integrating with other systems or using OpenNebula to build a private cloud.

Amazon EC2

Amazon Elastic Compute Cloud (EC2) is a web service that provides on-demand compute resources over the cloud [46, 47]. EC2 is part of Amazon Web Services (AWS), is designed to be scalable and flexible, and is often used to host general-purpose applications, including web applications, databases and enterprise applications. While the EC2 platform uses Xen-based hypervisors (see 2.2) under the hood [48], it acts as a complete abstraction layer over the underlying hardware and hypervisors and instead provides a rich set of tools to manage the VMs.

EC2 provides a large set of pre-configured templates called Amazon Machine Images (AMIs) that can be used to create new VMs, as well as a configuration in terms of specifications. This enables EC2 to provide a wide range of VMs, such as general-purpose, compute-optimized, memory-optimized, and storage-optimized VMs [46]. Features such as live migration, snapshot management, and auto-scaling are supported,

Azure Virtual Machines

Azure Virtual Machines is a service provided by Azure to manage VMs in the cloud [49], and is designed to be a cost-effective alternative to on-premises hardware [50]. Azure Virtual Machine provides a set of preconfigured VMs that can be used to create VMs for specific purposes, such as general-purpose or high-performance computing (HPC).

Behind the scenes Azure Virtual Machines runs on Hyper-V (see 2.2) and provides a set of features such as live migration, snapshot management, and auto-scaling [51].

Google Compute Engine

Google Compute Engine is a service provided by Google Cloud to manage computing resources in the cloud [52]. Compute Engine consists of three major parts: VMs, storage, and networking, where VMs are the core component. VMs are offered in a wide range of configurations, such as standard, high-memory, and high-CPU, making them suitable for different computational workloads [53].

Compute Engine uses KVM as its hypervisor (see 2.2), and provides many features such as live migration, snapshot management, and auto-scaling [52, 48].

Comparison

OpenStack, OpenNebula, and CloudStack are all examples of software that can be used to build both private and public cloud platforms. A private cloud is an on-premise cloud that is hosted within an organization and is, more often than not, dedicated to internal use. A private cloud, on the other hand, is a cloud platform meant to facilitate external users. A common denominator for these systems is that they are open-source and that they are highly customizable and extendable. As such, much of the configuration and management of the system is done by the user, and the system can be integrated with other systems to fit specific needs. This means that many of the conveniences such as preconfigured VMs and features such as live migration often require specific configuration and setup [54, 55].

Amazon EC2, Azure Virtual Machines and Google Compute Engine are examples of VM management systems in public cloud platforms. They are designed to be more user-friendly and to provide a more complete abstraction layer over the underlying hardware and hypervisors. In comparison, these technologies can be seen as complete setups of the open-source platforms previously mentioned, with all the configuration and management done by a provider. As such, many of the conveniences and features are provided out of the box and are perceived as hypervisor- and hardware-agnostic. This means that the user does not need to worry about the underlying hardware and hypervisors, and can instead focus on managing the VMs. This creates a trade-off where the user has less control and customization but instead gains a more user-friendly and complete system [54].

Both private and public cloud solutions for VM management are designed to be scalable and flexible and support a wide range of use cases (although private cloud solutions scale worse in practice. Which solution to use depends on the specific needs of the organization, and the level of control, customization needs, and the required security level of the system [55]).

Briefly introduce traditional VM management systems like OpenStack, CloudStack, and OpenNebula, focusing on how they implement these core features.

2.4 Containers and Kubernetes: A Paradigm Shift

Besides the common cloud service models (IaaS, PaaS, and SaaS), another service model has emerged, called *Container as a Service (CaaS)* [56]. CaaS

is a container-based cloud service model that has gained significant traction due to the lightweight and portable nature of a container, allowing for simple application bundling that can run in a vast range of environments [57].

Containers are often used in conjunction with a container orchestration platform, such as Kubernetes, to manage the life cycle of the containers including deployment, scaling, and monitoring. As such, they are often used to build, test, and deploy applications consistently. A common use case for containers is to run microservices, which is a software architecture that employs a set of small, independent and loosely coupled applications [57].

2.4.1 Containers Explained

Containers are a form of virtualization that offers a lightweight alternative to VMs. Unlike VMs, containers do not require a hypervisor to run and instead run on top of an OS and share the OS kernel with other containers. By not requiring a hypervisor, containers can run with less overhead than VMs, making their life cycle relatively fast and efficient compared to VMs [6, 58]. The differences between container and hypervisor virtualization are visualized in figure 2.6.

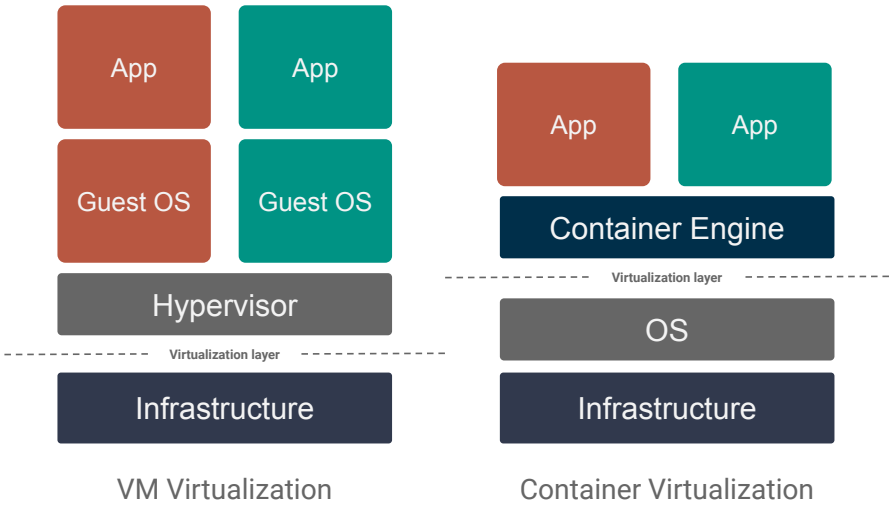

Figure 2.6: Overview of VMs and containers. The virtualization layer is placed on a lower level for VMs than for containers, illustrating the difference in their respective virtualization techniques.

A container is created from a container image, which is a lightweight executable software package that can run a specific application including depen-

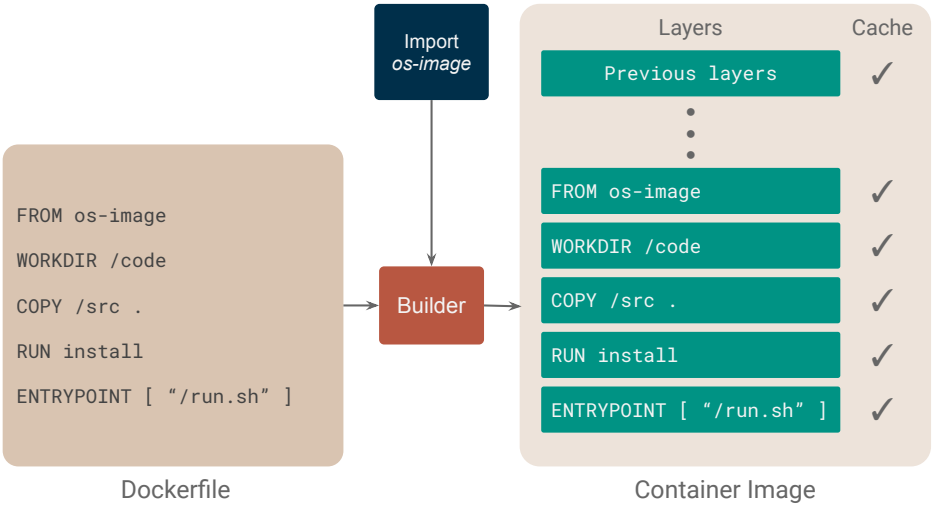

Figure 2.7: Overview of container image layers where each operation in a Dockerfile represents a layer that can be cached and reused. The layers are stacked on top of each other and appended to the parent image to form the final container image.

dencies and data files. By encapsulating an application and its dependencies in a single package, the container image becomes portable and independent allowing for the container to run consistently across different environments. A container image is created by describing its parent image, such as an OS image, and the steps to install and configure the application and its dependencies in a file, such as a Dockerfile. A parent image can itself have a parent image, creating a chain of dependencies. Furthermore, to enable caching and efficient storage, a container image can be divided into a series of steps referred to as layers. Each layer represents a change to the parent image, such as adding a file or installing a package. As such, layers act as checkpoints and allow for incremental builds where only the necessary layers are rebuilt when a change is made to the image [59, 60, 61]. Figure 2.7 visualizes the concept of container image layers, where each operation represents a change to the parent image.

Moreover, container images are often stored in container registries. A container registry provides central storage for container images and is used to distribute container images to environments that can run containers, such as a Kubernetes cluster. Common features for a container registry include versioning, access control, image scanning, and features to integrate with software development cycles, such as triggers to notify an external system when a new image is pushed to the registry [62, 63].

The Open Container Initiative (OCI) is a standardization effort that was created to ensure that containers are portable and interoperable across different platforms [64]. The OCI standard defines a runtime specification, image specification, and distribution specification, allowing for a container to be consistent in terms of its runtime environment, image format, and distribution method. Moreover, a container runtime, such as containerd, is used to create, deploy, and manage containers on a node [65]. Most runtimes, including containerd, implement the OCI runtime specification and can consistently run containers. This suggests that the OCI can be viewed as an abstraction layer, where the OCI runtime specification acts as an intermediary between the container runtime and the OS kernel, and the container image with the distribution specification acts as an intermediary between the container runtime and the container image registry.

2.4.2 Orchestration Using Kubernetes

Kubernetes is an open-source container orchestration platform that manages containers. It maintains a set of abstraction layers to manage the life cycle of containers and to provide a consistent environment for the containers to run in, facilitating tools for scheduling, scaling and networking [6].

Some of the abstractions used in Kubernetes to provide execution environments and communication include *Pods*, *Services* and *Deployments*. A Pod is the smallest deployable unit in Kubernetes, which groups one or more containers into a single unit that can be run on nodes. This means that Kubernetes ensures that all containers in a Pod are run on the same node. Encapsulating several containers allows for the logical grouping of coupled containers that share the same resources, such as networking and storage. This is commonly used when a Pod contains a main application container and a secondary container that provides logging and monitoring, called a sidecar container [66]. A Service represents a single-point-of-entry communication layer for a set of Pods and is used to provide a consistent way to access the Pods, such as load balancing. Services are especially useful in scenarios where Pods are fungible, meaning that the requesting client does not care which Pod it communicates with, as it creates an abstraction layer that hides the details of the Pods. This decouples frontends from backends and allows for a more scalable and resilient system. Furthermore, services are also useful for rolling updates, as they can be used to ensure that a certain part of the traffic is routed to the new version of the application [67]. Contrary to Pods and Services, a Deployment is a higher-level abstraction on top of Pods that is primarily used for replication control.

A Deployment does not represent an executable resource but instead describes a desired state in which a set of Pods should be. This includes the number of replicas and update strategy. As such, a Deployment proves useful when a set of Pods should be managed as a single unit, such as when scaling up or down, or when deploying a new version of an application [68, 69].

Furthermore, to support stateful applications Kubernetes mainly uses *PersistentVolumes*, *PersistentVolumeClaims* and *StatefulSets*. A PersistentVolume defines how and where persistent data is stored. Since there are many different ways to store data, such as local storage and network storage, a PersistentVolume is used to abstract the underlying storage. As such, each PersistentVolume is backed by a storage class, which defines the parameters and configuration to interact with the storage provider. A PersistentVolumeClaim, on the other hand, connects the PersistentVolume to a Pod. It acts as the bridge between the persistent storage and the execution environment effectively decoupling the Pod from the underlying storage. Contrary to PersistentVolumes and PersistentVolumeClaims, a StatefulSet is not used to manage the storage and is instead comparable to a stateful Deployment. A StatefulSet manages the life cycle of a set of Pods such that they are run predictably with their own unique identity. A common use case for a StatefulSet is scenarios that require each Pod to maintain a specific role, such as a primary or secondary database. A StatefulSet ensures that Pods maintain their role and that they are started in a predictable order, such as starting the primary database before the secondary database [70, 69].

Figure 2.8 presents a brief overview of the Kubernetes abstractions and how they interact with one another.

In certain scenarios, the built-in abstractions are not enough, and custom behavior is needed. For such scenarios, Kubernetes provides a way to extend its abstractions through Custom Resource Definitions (CRDs). A CRD is a form of extension for Kubernetes resources that allows for the creation of custom resources. It is used to extend the existing abstractions to provide additional features and capabilities that allow for a more feature-complete platform when custom behavior is needed. For instance, a CRD can be used to enforce certain fields to be specified to create the resource by applying custom validation rules [71, 69, 72].

There are many other abstractions in Kubernetes, each with its own set of features and capabilities that together create a feature-complete platform for container orchestration, but presenting them all in this report falls out of the scope of this project.

Kubernetes is inherently declarative, meaning that instead of issuing com-

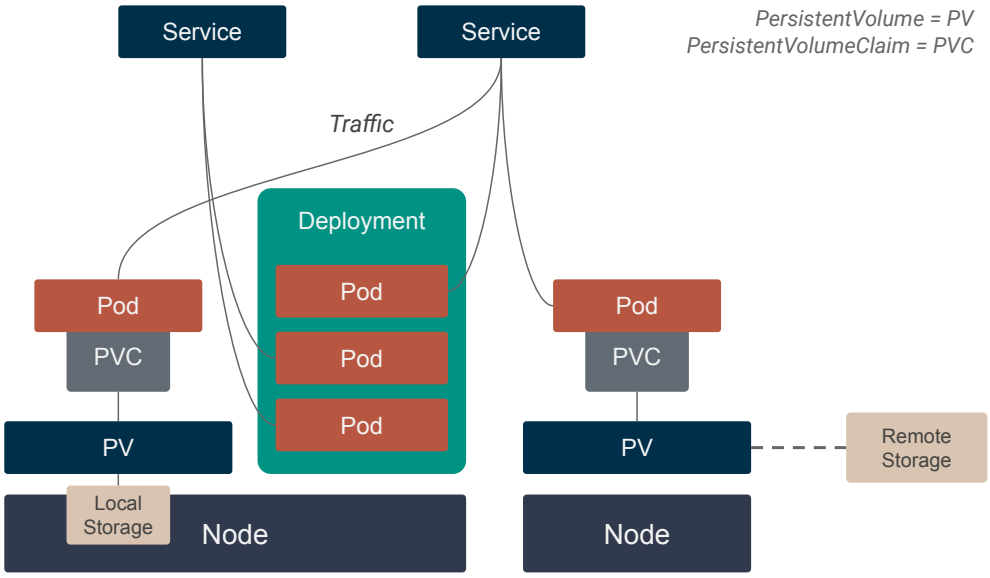

Figure 2.8: Brief overview of Kubernetes abstractions including many of the core components. The figure illustrates the communication between the different components and how storage is managed in Kubernetes.

mands to perform a specific task, such as starting a container, Kubernetes is given a desired state for a set of resources, and it is responsible for ensuring that the current state of the resources matches a desired state. For this reason, Kubernetes provides auto-restart, re-schedule, and replication control to ensure a resource stays in the desired state. By specifying the desired state, an entire application can be contained in a single configuration file, including its container, networking, storage and security requirements [6, 69]. The entity responsible for ensuring the desired state is referred to as a controller, and there are many different types of controllers depending on the resources they manage. For instance, a Deployment controller is the entity responsible for ensuring that the number of Pod replicas in a Deployment matches the desired amount. Moreover, managing resources using a configuration file and a controller is referred to as the *Controller pattern*, and is a common design pattern in Kubernetes as it provides a consistent and predictable way to manage resources [73].

Architecture-wise, Kubernetes uses a client-server approach. The server, often referred to as the control plane, is responsible for managing the state of the cluster. It is responsible for scheduling, scaling and networking by managing system-related services such as the API server, scheduler, and controller

manager. The API server can be viewed as the primary service for a Kubernetes cluster since it represents the gateway for all management tasks for the cluster by receiving and processing requests from the client. The client, often referred to as the worker node, is responsible for running the containers. The worker node consists of two primary components: the kubelet and the kube-proxy service. The kubelet is a service that wraps a container runtime and runs containers according to requests by the control plane [74, 69]. The kube-proxy service, on the other hand, manages the network communication for the containers. It can manage streams and forwarding rules for traffic using TCP, UDP or STCP [69, 75].

To interact with Kubernetes from the perspective of a user, a client is used to send requests to the API server. The client can be a CLI, such as kubectl, a web-based user interface, or a programmatic interface such as a REST API. The CLI tool kubectl is the most common way to interact with Kubernetes and is used to create, delete, and manage resources in the cluster [69].

Through its abstractions, life cycle management and declarative client-server architecture, Kubernetes provides a consistent and feature-rich platform for container orchestration. This has led to wide adoption of Kubernetes, and as such, it is often used in large-scale cloud environments [16].

2.5 Containers vs. Virtual Machines

Virtualization has enabled the commoditization of computing resources due to the benefits of being able to map virtual hardware to physical hardware (see 2.1) and has been key in enabling cloud computing. It has enabled the creation of flexible and scalable computing environments and has been used to build and manage large-scale cloud platforms. Virtualization has mainly brought two technologies: hypervisors and containers. Hypervisors are used to create VMs, including all associated aspects such as an OS. Containers, on the other hand, are used to create a lightweight and portable environment for an application to run in and are often run inside VMs. As such, a container comes with less overhead when compared to a hypervisor. However, by its nature, a VM can provide a more isolated environment than a container, leading to a more secure environment. Thus, the benefits of containers do not come without a cost and there is a trade-off between the two technologies; one is lightweight, portable, and quick to start while the other is more independent and secure [6, 76].

2.5.1 Comparative Overview

While containers and hypervisors are both technologies brought by virtualization, they operate on distinct levels of abstraction. Containers operate on a higher level than VMs using OS-level virtualization, which leads to the OS kernel with other containers. A hypervisor on the other hand can virtualize the entire hardware stack, including the entire OS, depending on the type of hypervisor used (see 2.2). As such, the two technologies have different use cases and are often used in conjunction with each other [76].

The main difference between containers and VMs is the level of isolation they provide. Containers share the OS kernel with other containers while VMs run on their OS. While this means a container can share resources more efficiently, it also means that a container is subject to a larger attack surface, as a vulnerability in the OS kernel can affect all containers running on the same host. VMs, on the other hand, reduce the attack surface by running more independently, meaning a compromised VM is less likely to affect other VMs running on the same host [77].

Another aspect of isolation is how co-located containers and VMs can share resources. When sharing resources such as CPU and memory, containers are vulnerable to the noisy neighbor problem. The noisy neighbor problem occurs when a resource-sharing mechanism, such as a scheduler, is unable to provide a fair share of resources. Some scenarios include when a container can consume all available resources due to a heavily loaded application or when a container intentionally consumes all available resources to starve other containers. Solutions exist to mitigate the noisy neighbor problem, including limiting the number of resources a container can consume through OS-level features such as *cgroups*. A cgroup is a feature in the Linux OS designed to limit, prioritize and isolate resource usage of a set of processes [78, 79, 80]. VMs, on the other hand, can provide a fair share of resources, as they operate in a more isolated environment, which leads to more resilient and predictable performance. However, the isolation does not mitigate the noisy neighbor problem for all resource types. For instance, a VM is still vulnerable to the noisy neighbor problem when sharing network-related resources such as network bandwidth and network-based storage [76].

Overcommitment is a technique used to improve packing efficiency, often referred to as *consolidation ratio*. Containers use the overcommitment mechanisms built into the OS itself, such as CPU overcommitment by multiplexing and memory overcommitment through virtual memory and memory swapping. Hypervisors, on the other hand, map virtual hardware devices to

physical hardware, creating virtual computing resources that are not directly tied to the physical resources. This works well for the CPU by creating virtual CPU (vCPU) cores. However, memory overcommitment is a more complex issue in hypervisors than in containers due to the way hypervisors allocate memory. Unlike containers, hypervisors assign a static amount of memory to a VM when it is created, and only reclaim the memory when the VM is deleted. Methods such as memory ballooning exist to address the limitation, which is a technique that treats memory as a shared resource across VMs and allows the hypervisor to reclaim memory from VMs when it is needed elsewhere. It is implemented as a driver that reserves unused memory (deflation) on a VM, rendering it unavailable to other processes running in the VM. The memory pages on the physical memory mapped to these pages in the VM are allocated to other VMs (inflation). This allows for VMs to share unused memory [81]. However, it is important to note that techniques like memory ballooning do not directly solve memory overcommitment for VMs, but instead reduce the need for it by managing the available memory more efficiently [76, 82]. Figure 2.9 visualizes the processes of deflating the memory of a VM and how it is used in another VM.

Live migration is a technique used to move a running resource from one host to another without interrupting the services provided by the resource. The technique is often used to balance the load or to perform maintenance on the hosting nodes. For VMs, it is a mature and widely technique for large data centers. Live migration works by copying memory pages from the source host to the destination host and then transferring the control of the VM from the source hypervisor to the destination hypervisor. Since VM disks are often stored on a shared storage, such as a Storage Attached Network (SAN), the VM disk is not transferred but rather reattached after the live migration is finished. For this reason, the VM can safely continue to run after its running state has been transferred to the destination host [83]. Live migration for containers, on the other hand, is not as mature as for VMs. Unlike VMs, containers require OS-level process migration, which is less reliable. However, several projects aim to provide live migration for containers, such as Checkpoint-restore in Userspace (CRUI). CRUI operates by checkpointing the state of a container, such as its memory and file descriptors, and then restoring the state on another host. While this is considered as container live migration, it is more comparable to snapshot and restore operations than a live migration operation, as CRUI freezes the container during the checkpointing, and converts the state of a container to a collection of files on disk, that can be used to restore the container on another host [84, 85, 86]. Another approach for live migrating

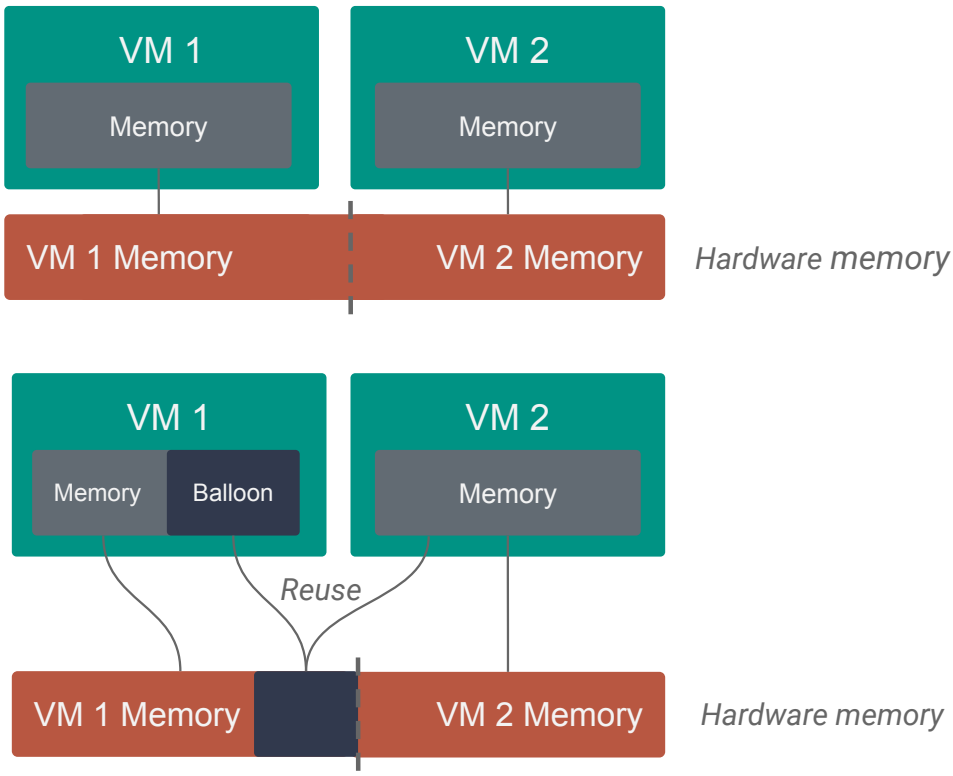

Figure 2.9: Overview of memory ballooning in two stages. The figure illustrates the memory of a VM is shrunk by the hypervisor (deflation) and then reused by another VM (inflation).

containers involves terminating them on one host and restarting them on another. This strategy assumes the container is stateless or utilizes shared storage for its state, as is often used in Kubernetes since it does not provide container live migration. Instead, Kubernetes facilitates persistent storage through Persistent Volume Claims (PVCs), essentially allowing a user to map portions of the container's state to shared storage, and assumes the rest can be discarded when a container is terminated [69].

2.5.2 Decision Factors

The choice between containers and VMs depends on the specific use case, and there are several factors to consider when deciding which technology to use. Some of the core factors are security, scalability, and legacy support [76].

Security is a key factor when deciding between containers and VMs. Using containers means sharing the OS kernel, which implies that a vulnerability in the OS kernel could affect all containers running on the same host. Containers are thus, from an attack-surface perspective, more prone to some vulnerabilities than VMs, making VMs more suitable in scenarios that require a higher level of isolation. For instance when an application manages sensitive or confidential data [87]. Moreover, sandboxing is a security mechanism that is used to restrict an execution environment such that it can safely run untrusted code without affecting the rest of the system. For instance, when analyzing the effect of malware a sandbox environment ensures that the malware is not able to escape the restricted environment. Sandboxing is done in multiple layers, including limiting the resources a process can access, such as memory and CPU, and limiting the system calls a process can make. As such, a sandbox environment requires a high level of isolation, which makes VMs a natural fit [88, 89].

Configuration drift is another aspect to consider regarding security. It is a phenomenon that arises when the configuration of a system changes over time due to several factors including manual intervention and software updates, and can lead to an undefined state in a system that could create security holes [90, 91]. Containers, as described in 2.5.1, are often run in a stateless manner using a single configuration file to describe the container, such as a Dockerfile. By being able to discard the state and recreate a container from a configuration file, the risk of configuration drift is reduced. As such, containers can provide a more consistent environment than VMs. Moreover, containers represent a more standardized and portable environment, using layers of dependencies, see figure 2.7. By standardizing the environment, external tools are more eas-

ily able to provide security features such as vulnerability scanning [92].

When deciding between containers and VMs scalability is an important factor. One of the main benefits of containers is their ability to share the underlying OS. However, as described in 2.5.1, this also means that containers are more prone to the noisy neighbor problem, where co-located applications can cause performance interference. While this issue is related to isolation and security, it is more prominent in larger-scale environments, especially in multi-tenant environments [80, 92].

Scalability is also related to cloud-native applications, which are applications that are designed to run in the cloud [7]. A cloud-native application is characterized by being horizontally scalable, stateless, and resilient, and is often built using microservices, see 2.4. Cloud-native applications thus require small and lightweight environments, making containers with an orchestration platform such as Kubernetes a natural fit.

Legacy support is another factor to consider when deciding between containers and VMs. Both VMs and containers can provide environments for legacy applications, such as applications that require a specific OS version or a specific set of dependencies. However, certain legacy applications are not able to run in a container due to the way they are designed, such as applications that require a specific OS kernel or a specific set of hardware. In such cases, a VM is often a better fit, as it enables virtualized hardware and a complete OS to be run [92, 93].

2.5.3 Towards Stronger Container Isolation

Containers are often used in conjunction with VMs due to the lack of strong isolation properties out of the box, see 2.5.1. However, several projects aim to provide the isolation properties of a VM and the lightweight properties of a container, such as Firecracker, Kata Containers, and gVisor.

Firecracker is a KVM-based hypervisor that creates lightweight VMs, called microVMs. Compared to a traditional VM, a microVM can start in a fraction of a second using only a fraction of the memory, comparable to that of containers. This is achieved through heavy optimization both in the hypervisor and in the guest functionality, such as reduced overhead through lightweight I/O and a stripped-down guest OS [8, 94].

Kata Containers is a project that, similar to Firecracker, aims to provide lightweight VMs that perform like containers while also providing strong isolation properties. Unlike Firecracker, Kata Containers uses both VMs and containers and replaces the container runtime, such as containerd (see 2.4.1)

with the Kata runtime. The Kata runtime implements the OCI runtime specification to be interoperable with container ecosystems. However, instead of running the container directly on the host, the Kata runtime creates a VM, in which the container runs. This provides the isolation properties of a VM and the lightweight properties of a container. Kata Containers can achieve this by heavily optimizing the VM and its creation process towards running containers, as this is the only purpose of the VM [8].

Another approach to strong container isolation is gVisor, a user-space kernel that provides a secure environment for the container. Unlike both Firecracker and Kata Containers, gVisor does not use VMs to enhance the isolation properties but instead operates as an extension around the container. gVisor consists of two core components, *Sentry* and *Gofer*. Sentry is a user-space kernel that intercepts and handles system calls made by the container and can provide a secure environment for the container. Gofer is a file access component that provides the container with file access. Separating the file access from the syscall access allows for a more secure environment. For instance, a compromised Sentry does not imply read or write access to any file [95].

In summary, Firecracker stands out among the three since it does not use container runtimes. As such, Kata Containers and gVisor are more comparable. Figure 2.10 visualizes the differences between how Kata Containers and gVisor wrap a container to provide a more secure execution environment. However, due to the varying added overhead, they are yet to be widely adopted [8].

2.6 Extending Kubernetes with KubeVirt

KubeVirt is a novel project that aims to provide a unified platform for managing both containers and VMs using Kubernetes. It is designed to naturally blend in with the Kubernetes ecosystem and provide a consistent interface to manage the life cycle of VMs, such as deployment, scaling, and monitoring. KubeVirt being built on top of Kubernetes means that it does not implement all the components found in a traditional VM management system, such as the scheduler, but instead uses the mature and feature-rich services the Kubernetes ecosystem provides [96]. By using CRDs (see 2.4), KubeVirt can extend the Kubernetes API to provide a specification for VMs to support features commonly found in traditional VM management systems, such as live migration, scaling and snapshot management. Moreover, CRDs allow KubeVirt to enforce validation on the resources to prevent invalid configurations, such as a VM that references a non-existing storage disk [97, 98].

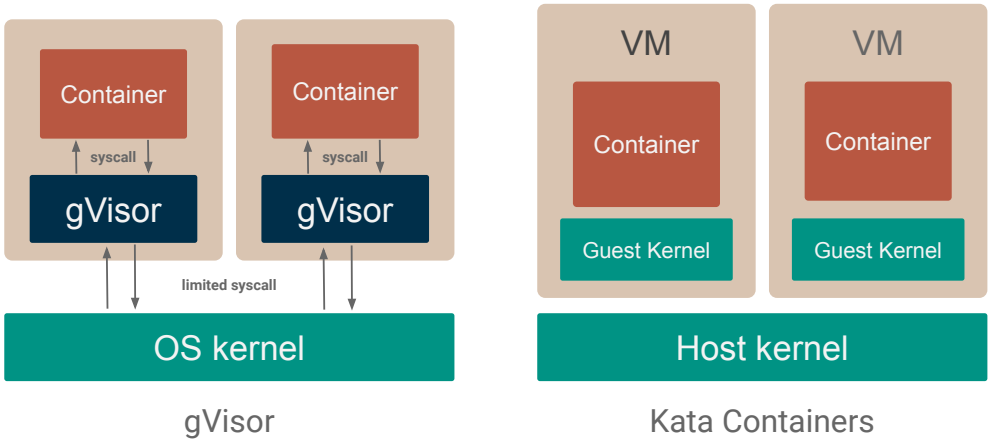

Figure 2.10: Overview of gVisor and Kata Containers and their approach to providing a more secure execution environment for containers. The figure illustrates the different approaches where gVisor operates as a proxy for system calls and file access, while Kata Containers wraps the container in a VM.

The design pattern chosen for KubeVirt's VM management is heavily inspired by other Kubernetes resources. A VM in KubeVirt is split into two resources called *VirtualMachine* and *VirtualMachineInstance*. The VirtualMachine resource is comparable to a StatefulSet and is used to describe the desired state of a VM, such as the VM's desired running state, CPU and RAM requirements, as well as cloud-init configuration. The VirtualMachineInstance, on the other hand, is comparable to a Pod and is created by the VirtualMachine resource to represent the running state of the VM. As such, a non-existing VirtualMachineInstance represents a non-running VM, and an existing VirtualMachineInstance represents a running VM. The VirtualMachineInstance contains runtime information about the VM such as the VM's IP address, running state and resource usage.

Similar to other Kubernetes resources, such as Deployments and StatefulSets, KubeVirt uses controllers to manage the life cycle of the VMs. These controllers are together responsible for creating the environment in which a VM can be run, but also that the desired state of VirtualMachine resource matches the current state of its VirtualMachineInstance resource. For instance, when VirtualMachine is updated with requirements for CPU cores, the KubeVirt controllers are responsible for updating the VirtualMachineInstance to match the new specification [99, 97].

Live migration and snapshots are implemented as CRDs, allowing for a declarative interface to manage these features. This is in stark contrast to tra-

ditional VM management systems, where live migration and snapshot management are commonly implemented as imperative operations. For instance, by creating a LiveMigration resource, KubeVirt initiates a live migration operation for the referenced VirtualMachine resource [100, 101]. This is comparable to the binding operation process for a PersistentVolume and its corresponding PersistentVolumeClaim, where the binding takes place when a PVC is created [102]

An additional benefit of KubeVirt being built on top of Kubernetes is that KubeVirt-related resources can be managed using the kubectl CLI tool. This allows for the declarative creation, modification and deletion of VMs using the same interface as for other Kubernetes resources. However, certain operations are not possible to perform using kubectl. For instance, accessing serial or graphical consoles of a VM is not possible using kubectl. To enable these operations, KubeVirt provides a complementary CLI tool called *virtctl*. While virtctl primarily enables console access, it also provides convenience features such as live migration and snapshot management [103].

2.6.1 KubeVirt Technical Overview

On a technical level, KubeVirt's VMs are hosted inside Kubernetes Pods. As described in 2.4.2, a Pod is the smallest deployable unit in Kubernetes. KubeVirt does not create a new abstraction for an execution environment and instead encapsulates the VM within a Pod. This brings many benefits as it allows for seamless communication between the VM and the Kubernetes ecosystem since it complies with Container Network Interface (CNI). For instance, a VM can be exposed to the network using a Service or use PersistentVolumeClaims to provide persistent storage.

KubeVirt consists of four main components to enable the running of VMs in Kubernetes: *virt-controller*, *virt-handler*, *virt-launcher* and the *libvirt daemon*. The virt-controller and virt-handler are responsible for managing the life cycle of the VMs, where the virt-controller is responsible for creating the Pod in which the VM will be hosted, and the the virt-handler is responsible for creating the VM itself. The virt-launcher, on the other hand, manages the runtime environment for a VM, including starting, stopping and monitoring the VM. Finally, the libvirt daemon (libvirtd) provides an interface to manage the VMs hosted by the underlying KVM hypervisor. The relationship between the three components is visualized in figure 2.11, where the virt-controller operates closest to Kubernetes API-server, virt-handler operates as a DaemonSet on every node to communicate with the underlying hypervisor, and virt-

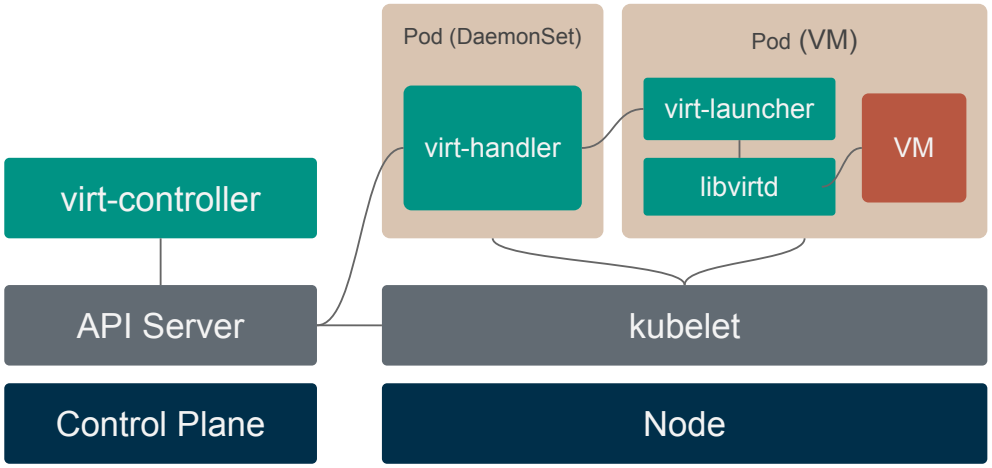

Figure 2.11: Overview of KubeVirt components, their logical place in the Kubernetes ecosystem and how they interact with each other. The green components represent the KubeVirt internal components.

launcher operates as a sidecar container in every Pod to manage the runtime environment of the VM [97, 99].

2.6.2 KubeVirt and GitOps

GitOps is a set of practices that enables using Git as a single source of truth for the desired state of the system, and it thus is heavily tied to the declarative nature of Kubernetes, as the desired state of the system is described in configuration files. By managing the infrastructure in Git, resources can be version controlled, and any changes to the system can be tracked and audited. This creates a developer-centric workflow, where developers can manage the infrastructure using the same tools and processes they use for managing application code [104]. Moreover, since GitOps is used for systems to manage their resources declaratively, it is a natural fit for KubeVirt. As such, KubeVirt opens up the possibility of managing VMs using GitOps, meaning that VM specification and life cycle management can be version-controlled and audited [105]. Being easily integrated with GitOps further indicates that KubeVirt can unify the management of both containers and VMs using Kubernetes.

2.6.3 Comparison with Traditional Approaches

KubeVirt does not represent a new virtualization or hypervisor technology but instead leverages QEMU and KVM to run the VMs, which means KubeVirt is

fundamentally similar to traditional VM management systems, such as OpenStack, CloudStack, and OpenNebula [106]. Therefore, KubeVirt can provide the same level of isolation and security as traditional VM management systems, as it uses the same underlying hypervisor technology. However, KubeVirt is not as feature-rich, as it is not designed to be a complete replacement for traditional VM management systems. For instance, OpenStack not only manages VMs and their life cycle but also provides additional services, including host provisioning and data warehousing [107]. KubeVirt, on the other hand, is primarily designed to manage VMs and their life cycle such that it can provide a unified platform for managing both containers and VMs. As such, KubeVirt integrates well with Kubernetes to provide a seamless experience for VM management. For instance, VMs created with KubeVirt support CNI plugins and integrate with PersistentVolumeClaims to provide storage [97, 108, 109].

2.7 Related Work

2.7.1 Current challenges

The Cloud Native Computing Foundation (CNCF) presents in a report some of the challenges of managing VMs using a traditional IaaS platform and uses OpenStack as an example [19]. While OpenStack has been a popular choice for managing VMs, it has been criticized for being complex and difficult to manage, as well as having a steep learning curve. They argue that the complexity of OpenStack is not always needed and that it is possible to manage VMs using a simpler and more efficient platform, such as Kubernetes with KubeVirt.

Additionally, the CNCF report highlights the importance of having a unified platform for managing both VMs and containers, as it could reduce the complexity of the underlying technology stack and management costs. These claims are further supported by a report by Kubacki [20]. He argues that managing VMs using traditional VM management systems alongside Kubernetes leads to unnecessary costs, such as maintaining personnel and skill sets for two technologies. Moreover, by allowing the coexistence of VMs and containers in a single platform, KubeVirt could potentially reduce any barrier of entry around managing VMs for organizations that are already using Kubernetes for container orchestration, or act as a catalyst for organizations considering adopting Kubernetes.

2.7.2 KubeVirt challenges

While KubeVirt is presented as a solution to the challenges of managing VMs using traditional VM management systems, it is important to consider the challenges of using KubeVirt itself. A study by Trakulmaiphol and Piromsopa presents some shortcomings when migrating from a traditional VM management system to KubeVirt [110]. One of the issues identified is the management of IP addresses, particularly when maintaining static IP addresses in a dynamic IP environment, such as Kubernetes. In the study, they developed a framework to solve some of the issues on the application level, such as DNS queries, and provided a set of guidelines related to KubeVirt's unique features, such as the Multus Container Network Interface.

A common practice when managing Kubernetes clusters is to deploy Kubernetes nodes on VMs created by a VM management system. This means that when using KubeVirt, VMs are created inside other VMs leading to nested virtualization. Ye et al. argue in a study that nested virtualization raises security concerns due to the introduction of a larger trusted computing base (TCB) [111]. By using a larger TCB, the size of the system's attack vector increases leading to an increased risk for vulnerabilities. For this reason, nested virtualization might not be suitable when strict security is required, such as applications managing sensitive or confidential data. This study is particularly relevant as it highlights a potential limitation if KubeVirt is simply seen as another Kubernetes resource, and not as a VM management system.

2.7.3 Usability of Kubernetes and KubeVirt

Using Kubernetes as a VM management system is a relatively new concept, and there is limited research on the usability of KubeVirt. However, several studies have evaluated the usability of Kubernetes, which is relevant as KubeVirt is an extension built on top of Kubernetes.

Kubernetes API is inherently declarative, meaning an approach that promotes the usage of documents that describe a desired state of a system. A study by Sonninen investigates many of the possible benefits of using a declarative approach in system architecture [112]. The study found that a declarative approach facilitates GitOps, a way to manage infrastructure using files in Git, which simplifies system and resource management and enables easier rollbacks. Using Git as a source of truth for the desired state of the system has the added benefits of version control, audit and collaboration. These benefits were exemplified when CERN migrated the internal CMS project from a VM cluster to Kubernetes, as described in the two studies they published. The first study

conducted by Imran et al. describes how the Kubernetes migration enhanced the overall system management compared to using a VM cluster [113]. While many of their experienced benefits stem from adopting containers instead of VMs, the study still suggests Kubernetes provided many advantages, such as reduced operational costs. The other study conducted by Badran presents the practical benefits they experienced by using Kubernetes and is more focused on the adoption of a declarative approach for system management [114]. It describes how using Kubernetes instead of a VM cluster allowed them to craft reusable and composable configurations that enabled their entire infrastructure to be managed as code.

The benefits of moving into Kubernetes are further exemplified in a study by Gelley [115]. It presents a case study where an organization moved its business application from a Platform as a Service (PaaS) provided by Cloud Foundry into Kubernetes. The study not only describes how Kubernetes provided tremendous flexibility in terms of configuration and deployment compared to the previous PaaS but also how the organization was able to leverage the Kubernetes ecosystem to provide additional features such as auto-scaling and monitoring. Moreover, by moving into Kubernetes, possible future migration to other platforms was made simpler due to how widely adopted Kubernetes is.

User experience in Kubernetes was investigated in a study by Packard et al. that gathered several scenarios in which projects were built and managed using Kubernetes [116]. Many aspects were considered in the study, including administration such as managing persistent storage and limiting user resources with quotas, but also developer experience such as building and deploying applications. The study found that while system administrators face a learning curve when using Kubernetes, such as understanding the Kubernetes API and the ecosystem, users without prior knowledge of Kubernetes were able to successfully deploy applications. This finding is important to consider when evaluating KubeVirt, as it suggests that KubeVirt is a viable option even for users without prior knowledge of Kubernetes. Another study that investigated the user experience of Kubernetes was conducted by Boronin [117]. The study presents a case study where an organization faced a migration from a traditionally monolithic system architecture to a more cloud-native design. The study is relevant as it not only describes the challenges or benefits of Kubernetes itself but also considers the organizational impact of migrating to Kubernetes. This involves transferring skill sets among personnel, and the impact on the organization's culture.

In summary, these studies suggest that the benefits of using Kubernetes

as a platform are not directly tied to what kind of resources are managed, but rather to the platform itself. Thus, when evaluating KubeVirt, it is important to not only consider benefits of KubeVirt itself but also the benefits that follow by adopting Kubernetes.

2.7.4 Performance Evaluations

Regarding performance, several studies have evaluated popular systems that can manage VMs. For instance, open-source alternatives such as OpenStack and CloudStack, or proprietary systems such as Amazon EC2.

A study by Kang et al. compared the performance of OpenStack and Amazon EC2 using a wide range of configurations and analyzed diverse aspects based on different hypervisor, storage and networking configurations [118]. Apart from the performance comparisons, the study also investigated performance impact when using more advanced features, such as Infiniband networking instead of Ethernet. The study showed mixed results, where OpenStack outperforms EC2 in some scenarios such as when using Infiniband networking, but EC2 tends to outperform OpenStack in parallel processing. As such, the study concluded that component-wise, each system has strengths. This study brings with it relevant perspectives as it highlights many of the aspects that are important to consider when evaluating the performance of VM management systems.

Another study by Mullerikkal and Sastri compared OpenStack and CloudStack regarding implementation complexity, overall stability and performance [119]. The performance tests were designed to measure disk- and network I/O, and not CPU or memory usage. The study found that while OpenStack had a more complex implementation, it was more stable. Furthermore, by using a set of benchmarks it found that OpenStack had a better overall performance compared to CloudStack. However, a few years later, a complementary comparison between OpenStack and CloudStack was conducted by Paradowski et al. [120]. The comparison focused on performance and used several benchmarks such as deployment time, deletion time, and CPU utilization during deployment and deletion, and revealed significant and quantifiable differences between the two platforms. These tests were designed to represent user interactions with the system. However, the scale of the tests used in the comparison was relatively small and did not explore how the systems manage VMs under higher load. Analyzing the performance of a VM management system under higher load is important, as it can reveal how well a system scales when the ratio between VMs and nodes increases.

In contrast, a later study by Amaral benchmarked KubeVirt uses a setup with a large number of VMs relative to the size of the nodes, aiming to explore the limits of KubeVirt's control plane (scheduler etc.) [121]. The study found that, while KubeVirt was able to create all VMs, the control plane suffered a heavy load, indicating a possible large overhead when KubeVirt provisions a large number of VMs. This finding is important to consider when evaluating KubeVirt, as it indicates that KubeVirt may not yet be suitable for managing a large number of VMs.

In terms of network performance, a study conducted by Kolahdouzan investigated the networking traffic pattern in KubeVirt and OpenStack by performing networking performance tests to evaluate potential VM overhead [122]. The study found that the network overhead was significantly larger for scenarios in which VMs reside on separate nodes. However, while the study found overhead for both KubeVirt and OpenStack, it found that OpenStack had a considerably larger overhead. While this project will not focus on networking performance, it is important to consider the findings of this study when evaluating KubeVirt.

Hypervisors are a key component of VM management systems, and several studies have evaluated the performance of hypervisors. A study by Hwang et al. evaluated the performance of a range of hypervisors including KVM, Hyper-V and Xen [123]. The study found that applications and assigned resources had a significant impact on the overhead of the hypervisors. However, the study concluded that no single hypervisor was the silver bullet and that the choice of hypervisor should be based on the specific use case. Reviewing the performance of hypervisors is important when evaluating KubeVirt, as KubeVirt uses KVM as its hypervisor.

Chapter 3

Performance Analysis

As described in the literature study, Paradowski et al. presented a set of benchmarks used to evaluate CloudStack and OpenStack [120]. This comparison was used as a foundational basis for the performance analysis in this project, employing many of its approaches for evaluating the performance of a VM management system. In this project, however, OpenNebula was compared with KubeVirt. Choosing OpenNebula as the second system was based on its popularity and the fact it is open-source, but also to ensure that the tests were relevant to Net Insight's current infrastructure.

3.1 Setup

The testing process was conducted on Azure using a group of VMs that acted as hosting nodes. This approach was chosen to ensure that the tests are not affected by the underlying hardware and to facilitate reproducibility.

A set of scripts was created to automate the entire testing process. The scripts automated node creation in Azure, as well as installation and configuration of the VM management system on the nodes. Once the tests were finished, the results were collected by the scripts and the nodes were deleted in Azure. The scripts were thus used to create a complete setup for each system in a consistent manner using a single command.

Payload-wise, this project used a similar approach to the study by Paradowski et al. with predefined types for VMs with CPU ranging from 1 to 4 CPU cores, and RAM ranging from 128 MB to 4 GB (see Table 3.4 for a detailed overview). Moreover, VMs used the lightweight Cirros Linux image, since a lightweight OS image is less likely to introduce noise in measurements [124]. This also meant a VM disk could be smaller, which allowed for more VMs to

be created per node and for tests to run quicker since a lightweight image is usually faster to boot than a larger image.

Each test case used a setup of three Azure nodes, one management node and two worker nodes. Having more than one worker node was crucial for certain tests to be able to run. For instance, a live migration test requires at least two worker nodes since it involves moving VMs between nodes. The Azure nodes were configured large enough to facilitate a broad range of tests, including tests with many small VMs and tests with a few large VMs. Azure offered many different VM types, many of which were suitable in terms of CPU and memory specification. However, only a few offered virtualization support, which was required when running nested VMs. This left the Azure Dv3-series as the most suitable VM type and, specifically, the D4v3 VM type that has 4 CPU cores and 16 GB of memory [125]. The price of the D4v3 VM type was 0.2 USD per hour, which was deemed acceptable for the project [126]. Table 3.1 shows the overview of the Azure node specification used in the tests.

Role	CPU	RAM	Disk
Control Plane (Azure D4v3)	4	16GB	100GB
Worker 1 (Azure D4v3)	4	16GB	100GB
Worker 2 (Azure D4v3)	4	16GB	100GB
Scale-up Workers 3-10 (Azure D4v3)	4	16GB	100GB

Table 3.1: Node specifications used for the nodes hosting the VM management systems.

To quantify the results of the tests, several metrics were collected on each Azure node for every test. This includes measuring the utilization of a node's CPU, memory and disk, as well as the duration of the specific task, see Table 3.2. Further technical details on how metrics were collected are described later in this chapter.

The tests performed were divided into three categories: basic, limit testing, and scaling. The basic tests were designed to evaluate the basic functionality of the VM management systems, such as creating and deleting VMs. The limit testing tests were designed to evaluate how a VM management system operates when the maximum capacity is reached in the worker nodes. The scaling tests were designed to evaluate how a VM management system handles the scale-out and scale-in of worker nodes, both with and without VMs on it (that would need to be migrated). Each test was run 10 times. See table 3.3 for an overview of the tests and their purpose.

Metric	Unit	Description
CPU utilization	%	The percentage of CPU used by the system
Memory utilization	%	The percentage of memory used by the system
Disk utilization	%	The percentage of disk used by the system
Task duration	s	The duration of a specific task

Table 3.2: Performance metrics that were used to quantify the results of the tests.

Test	Purpose
Create each type of VM and delete it	Evaluate the basic functionality of the VM management systems without addressing potential load issues.
Create 20 type-1 VMs and then delete them	Evaluate how the VM management systems manage a relatively large burst of VMs.
Live migrate a type-4 VM	Evaluate how well the VM management systems execute a migration operation without addressing potential load issues.
Live migrate 10 type-1 VMs	Evaluate how well the VM management systems manage a relatively large burst of migration operations.
Scale up and down 2 to 10 workers without VMs	Evaluate how well the VM management systems scale up and down without any load.
Scale up and down 2 to 10 workers with a type-1 VM on each worker	Evaluate how the VM management systems handle scaling operations that incur migration operations.

Table 3.3: Performance tests that were used when evaluating the VM management systems. During each test, metrics were collected for each node.

Type	CPU	RAM	Disk Size
Type 1	1	128MB	1GB
Type 2	1	1GB	5GB
Type 3	2	2GB	10GB
Type 4	4	4GB	20GB

Table 3.4: VM specifications for the different types of VMS that were used in the tests.

3.2 Building a Framework

Evaluating the performance of a VM management system is a complex task that requires a structured approach due to its many components and multi-node setup. For this reason, a framework to form standardized testing environments was created. The framework was designed to be VM management system agnostic, meaning that it can be used to evaluate any VM management system as long as the system supports the set of features defined in the framework. Consequently, this means that the framework can be extended to evaluate any other VM management system, such as the previously mentioned OpenStack and CloudStack, and not only OpenNebula and KubeVirt. However, it is important to note that the framework was designed to evaluate VM management systems that can be hosted on-premise since it quantifies the performance of a VM management system by measuring the utilization of nodes hosting the system. As such, the framework does not consider cloud-based VM management systems, such as Amazon EC2, Google Compute Engine or Azure Virtual Machines.

3.2.1 Design

The framework was designed to be modular and loosely coupled with external libraries. This design choice was made to ensure that the framework can be easily extended and modified and that it does not rely on external dependencies. To interact with a VM management system an interface was created that defines the methods that a VM management system must implement to be compatible. It defines methods for creating and deleting VMs, migrating VMs between nodes and scaling up and down the number of worker nodes, as well as methods for installing control and worker nodes. Each VM management system implements these methods in their respective packages. The full definition of the interface can be found in the appendix A.1. To host

each VM management system, the framework created a set of nodes in Azure, which were completely installed and configured before any performance analysis was conducted to form a fair starting point for every VM management system. However, any dependencies required by the VM management system were intentionally left to the installation, as unnecessary dependencies could introduce noise in the measurements.

Metrics were collected by running a daemon on each Azure node that collects utilization metrics, including CPU, memory and disk utilization. The daemon operated by fetching metrics every second and storing them in a JSON file locally. However, to only capture metrics when a test was running, the daemon only started collecting metrics when a test was initiated and stopped when the test was finished. This was controlled by placing an empty file called *run* where the daemon was running. The daemon checked for the file's existence periodically and started collecting metrics when it noticed the file was present. As such, each node timestamped its metrics locally, which meant that the timestamp of the data points could vary between the nodes. However, this distributed approach was chosen to reduce the number of network requests to a minimum and to ensure that no skew was introduced in the measurements due to network latency.

The utilization metrics were collected by a set of tools that were installed on each Azure node. Since the tools were required to be installed on every node, they were installed directly after creation. Table 3.5 shows an overview of the tools used by the daemon. Another important aspect considered was asynchronous execution. Each of the tools used by the daemon was executed in parallel to minimize the time it took to collect the metrics for a given timestamp, essentially allowing shorter intervals between each data point. The structure of the output varied between the tools and was parsed using built-in tools in Linux such as *awk* and *sed* before being stored in a JSON file. The final output file was fetched over SSH using *scp* and parsed by the framework to extract the metrics. The daemon was written in Bash to ensure that it could run on any Linux distribution and did not require any external dependencies. The code for the daemon can be found in the appendix A.2 and the framework design is visualized in Figure 3.1.

The workflow of the framework was split into four phases to form a structured approach that allows for modularity and caching. Each phase was responsible for a specific part of the workflow and, in most cases, independent of each other. For instance, the installation phase can be omitted if the nodes are already installed and configured from a previous run. However, if the teardown phase was executed in one run, the next run after it must start with the

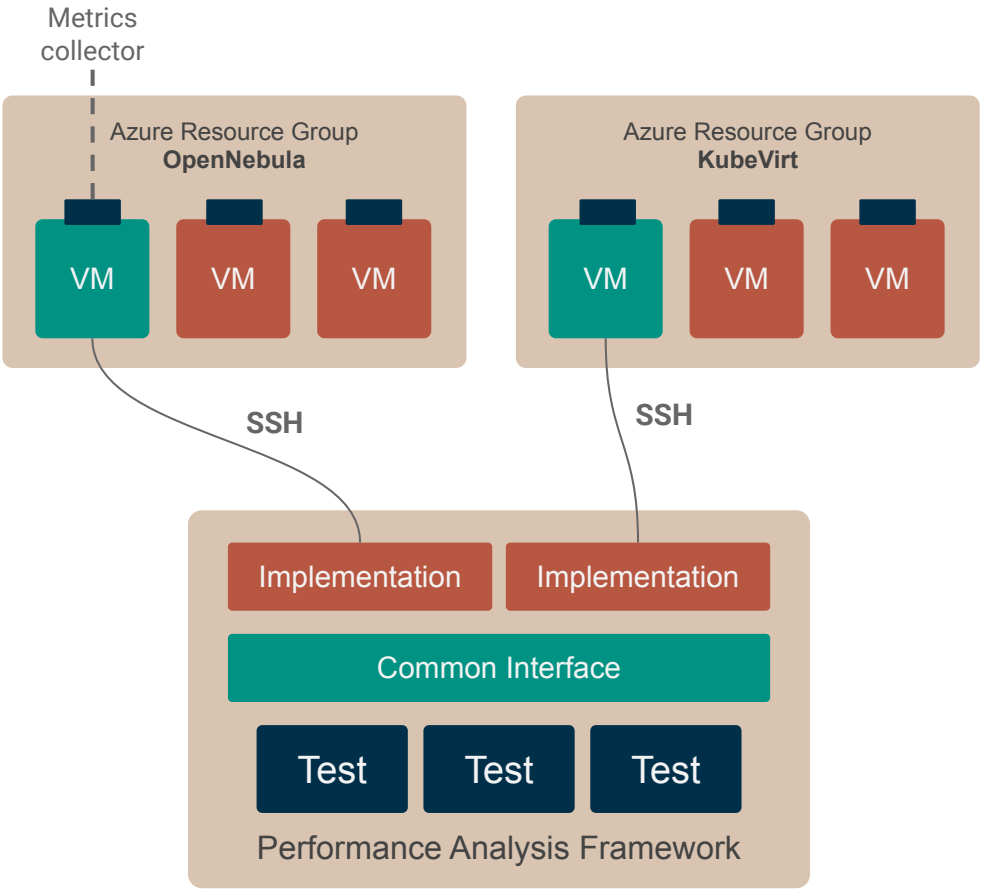

Figure 3.1: Overview of the framework designed with a framework communicating with the VM management systems through SSH. The metric collector was installed on each node to collect utilization metrics. The green VM figure represents the VM management system control node and the orange VM figures represent the worker nodes.

Tool	Utilization	Description
mpstat	CPU	Reports activities for each available processor
free	Memory	Displays the total amount of free and used memory in the system
iostat	Disk	Reports CPU utilization and disk I/O statistics

Table 3.5: Tools used by the metrics collector daemon. Each tool was installed on each Azure node during the installation phase.

setup phase. The design choice of dividing the workflow into phases was made to allow for more efficient testing, both when developing the scripts and when executing them. This was especially important considering that each test came with a cost in Azure. The entire workflow is illustrated in Figure 3.2 and is described below:

1. **Setup**: The setup phase is responsible for creating the nodes and networking in Azure. It is also responsible for installing base packages that every node requires, such as *sysstat* to collect utilization metrics and *jq* to parse JSON output.

2. **Installation**: The installation phase is responsible for installing and configuring the VM management system on the Azure nodes. Each VM management system implements an installation method that is responsible for installing and configuring the system with one control node and two worker nodes such that it can run a VM. The installation method also installs several standby workers that will be used when scaling up and down the number of worker nodes. By having standby workers, the framework can independently measure the cost of scaling up and down without the cost of downloading dependencies. While larger dependencies can be seen as a metric on their own, it is important to note that production environments usually generate OS images with prepared dependencies, rendering the cost of downloading dependencies irrelevant.

3. **Benchmark**: The benchmark phase is responsible for running the performance tests. The tests are described in 3.3 and are designed to evaluate the general performance of the VM management system. Each test is a combination of the methods the framework interface offers, such as creating VMs in different sizes, migrating VMs between nodes and

scaling up and down the number of worker nodes. This phase generates and saves the results of the tests in a JSON file.

4. **Teardown**: The teardown phase is responsible for deleting the Azure nodes and ensuring that no resources are left behind. Having this as a separate phase allows for tests to run sequentially without the time cost of creating and deleting nodes for each test.

3.2.2 Implementation

The framework was implemented in the programming language Go, a language popular for its performance and concurrency features. Concurrency was a particularly important factor as several tests involved running multiple operations in parallel, such as creating or migrating several VMs at the same time. As such, settling for Go reduced the risk of introducing noise in the measurements and could run each test more efficiently.

The most important part of the framework was arguably to provide fair and reproducible comparisons. As described previously, each VM management system implemented an interface that defined methods for creating and deleting VMs. While this provided uniform operations from the framework's point of view, it also made the method's implementation crucial: KubeVirt's implementation of creating a VM should be as close to OpenNebula's implementation as possible to ensure that the comparisons are fair and that no unnecessary overhead are introduced. One example of this was that any network-related configuration had to be omitted since network-related metrics were not considered in the project. Another example was that the scheduling configuration had to be edited to ensure that VMs were created in as similar a manner as possible. However, all features that a VM management system offered and that were beneficial for the specific operation were considered. For instance, OpenNebula did not allow a live migration action to be performed on a VM that is not running, while KubeVirt allowed the operation to be requested despite the state of a VM. This allowed KubeVirt to omit the waiting time for a VM to be in the correct state before requesting a migration action.

Below is an extraction of OpenNebula's and KubeVirt's implementations for creating a VM.

KubeVirt

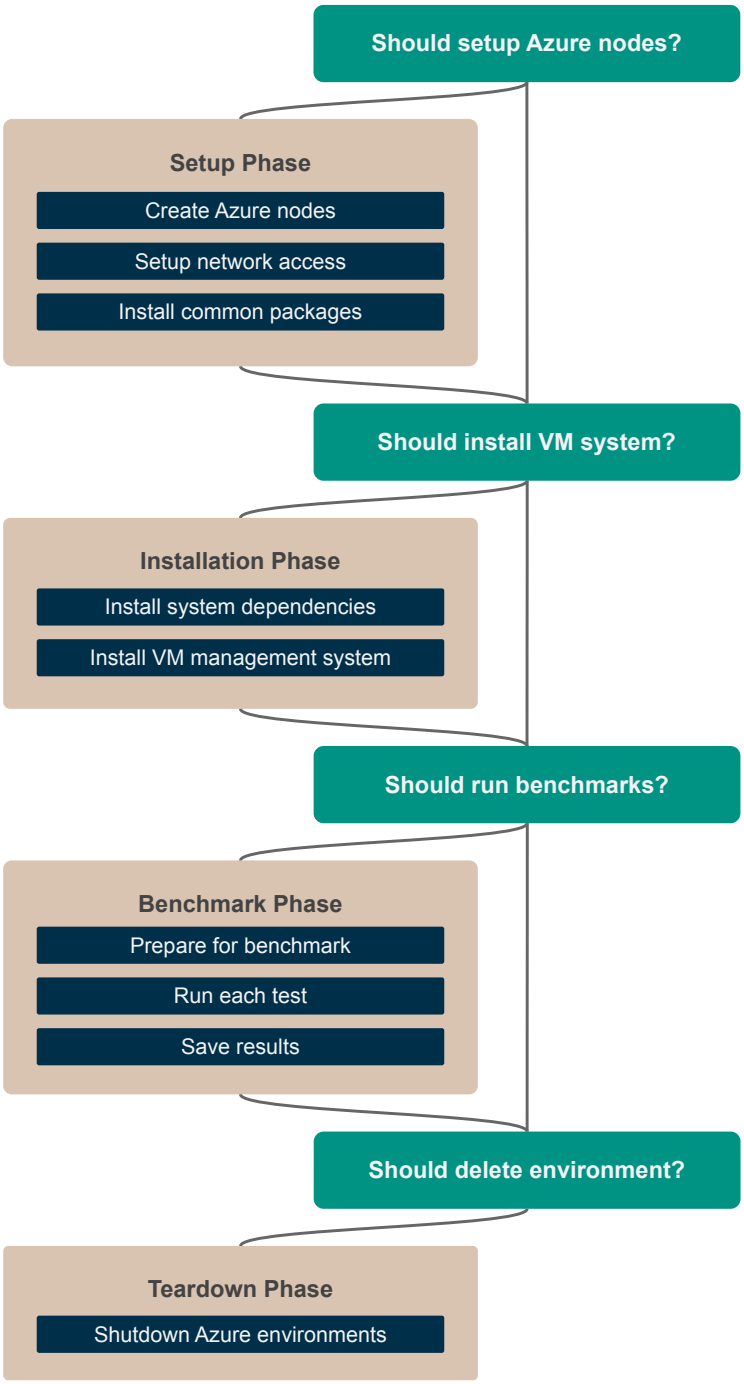

Figure 3.2: Overview of the workflow of the framework. The framework was divided into four phases that were responsible for different parts of the workflow. Each phase could be run independently of each other and disabled if not needed.

```
kubectl apply -f <<EOF
    apiVersion: kubevirt.io/v1
    kind: VirtualMachine
    metadata:
        name: $NAME
    spec:
        running: true
        template:
            spec:
            domain:
                devices:
                    rng: {}
                    disks:
                    - disk:
                        bus: virtio
                        name: containerdisk
                    - disk:
                        bus: virtio
                        name: emptydisk
                    resources:
                    requests:
                        cpu: 100m
                        memory: $MEMORYMi
                    limits:
                        cpu: $CPU_CORES
                        memory: $MEMORYMi
            volumes:
            - name: emptydisk
                emptyDisk:
                capacity: $DISK_SIZEGi
            - name: containerdisk
                containerDisk:
                image: $OS_IMAGE
EOF
```

OpenNebula

```
onetemplate instantiate $TEMPLATE_ID --name $NAME <<EOF
    CPU="0.1"
```

```
    VCPU="$CPU_CORES"
    MEMORY="$MEMORY"
    DISK=[
        SIZE="$DISK_SIZE"
        IMAGE="$OS_IMAGE"
    ]
EOF
```

The entire framework is open-source and available at `INSERTURLHERE`. The GitHub repository contains all the code required to run the framework, including documentation on how to set up and run the framework.

Chapter 4

Usability Analysis

To accurately measure the usability of KubeVirt, a set of tasks was crafted in cooperation with a system administrator at Net Insight. The tasks were designed to be representative of the tasks that a system administrator would perform when managing VMs. The tasks were designed to be performed using the Kubernetes command line interface (CLI) tool *kubectl* and KubeVirt's complementary CLI tool *virtctl*. This approach was chosen since KubeVirt did not directly offer a graphical user interface (GUI), and using any third-party solution could risk introducing biases in the results.

4.1 Setup

To host an environment that was easily accessible to the participants, an AKS cluster was set up. Alternatives included installing a Kubernetes cluster locally using tools such as *Minicube* or *k3d*, but this approach was deemed less suitable since it could be time-consuming and require more resources from the participants. Moreover, a cloud-based solution ensured that the participants could access the environment anywhere reducing possible barriers to participation. AKS specifically was chosen since it included preinstalled storage classes that enabled plug-and-play persistent storage functionality. KubeVirt's snapshot-restore API required a CSI storage class, which made AKS a suitable choice. Similar to the performance analysis, the cluster included one control-plane node and two worker nodes to ensure that the participants could perform the tasks without any issues. Having two worker nodes was crucial for the live migration task, as it required moving a VM between two nodes. Separating the control node from the worker nodes was not necessary for the tests, but it is a common practice since it allows workers to be recreated without affecting

the cluster's control components.

4.2 Task Design

The analysis was designed as a step-by-step guide that would take the participants through a set of tasks that were divided into three groups: basic usage, maintenance and debugging. The focus of the analysis was not to evaluate the participant's ability to perform the tasks or their Kubernetes knowledge but rather to expose them to KubeVirt and its workflows. As such, the tasks were designed to be as straightforward as possible, except for a debugging task that was designed to evaluate finding and resolving runtime issues. Moreover, the order of the tasks was designed to gradually increase in complexity to ensure a structured approach and allow the participants to familiarize themselves with KubeVirt. Below is a description of each group of tasks:

- **Basic usage**
  This task was designed to act as a starting point for the usability analysis. The participants were asked to create a single VM configured with a *containerDisk*, connect to it using SSH and VNC and then delete it. The purpose of this task was to evaluate how well KubeVirt supports the most basic operations and whether the participants deem the process intuitive.

- **Maintenance**
  The second task was designed to evaluate how well KubeVirt supports recurring tasks that a system administrator could face when managing VMs and the platform hosting them. The analysis divided this task into two parts: live migration and snapshotting. The participants were first asked to create a single VM and start it on a specific host, then migrate the VM to another host without any downtime. A lightweight VM configured with a *containerDisk* was chosen for this task to keep the focus on the live migration operation. The second part of the task was to create a snapshot of a VM and then restore it. A VM configured with a *DataVolume* disk was chosen for this task since a container disk cannot be snapshotted. Both live migration and snapshotting were included in the usability analysis since they are inherently different in KubeVirt compared to traditional VM management systems due to the declarative nature of Kubernetes, see 2.6.

- **Debugging**
  The third task was designed to evaluate how well KubeVirt supports de-

bugging and how intuitive the process of resolving runtime issues is. While KubeVirt does enforce correct specification upon creation with API validation, see 2.6, it is still possible that a VM does not start if the misconfiguration affects the runtime. For this reason, the participants were presented with a VM that was not accessible due to a cloud-init misconfiguration that prevents SSH access to the VM by having invalid password syntax and missing authorized SSH public keys. This means that the participants could solve the issue either by resolving the password formatting issue or supplying a valid SSH public key. The participants were presented with a basic problem statement and were encouraged to use the official documentation if needed. For this task, a lightweight VM configured with a containerDisk was used to keep the focus on the debugging operation.

After each group of tasks was finished, the participants were asked to reflect on the experience and provide feedback on the tasks. The participants were encouraged to use a free-form approach but could use a set of open-ended questions as a reference.

4.3 Conducting the Usability Analysis

The usability analysis was performed together with a group of system administrators at Net Insight. The participants were presented with a set of tasks as described in the previous section. Each participant was given a document containing all the tasks as well as an introduction to KubeVirt and each of the operations they would perform. For each task, the participant was encouraged to use the official KubeVirt documentation to aid them in completing the tasks. Together with the document containing the tasks, each participant was given a *kubeconfig* and the name for their Kubernetes namespace in which they would perform their tasks. They were given separate namespaces to ensure that none of the tasks would interfere with the results of the other participants. Once finished, the reflection and feedback were collected from each participant. They were given two weeks to complete the tasks, and while this was a relatively long time, it was chosen to ensure that the participants were given enough time to potentially explore features in KubeVirt that were not part of the given tasks. The entire usability analysis can be found in the appendix C.

Chapter 5

Results

This section contains the results of the performance analysis and usability analysis. Each test in the performance analysis was run 10 times, and the result visualizations were averaged using a rolling mean with a window size of 5% of the total number of data points. The usability analysis received feedback in different forms, either by answering questions or providing a free-form response.

5.1 Performance Analysis

This section contains the results of the performance analysis with KubeVirt and OpenNebula. Only the most significant results are presented in this section, and the complete list of results can be found in the appendix B.

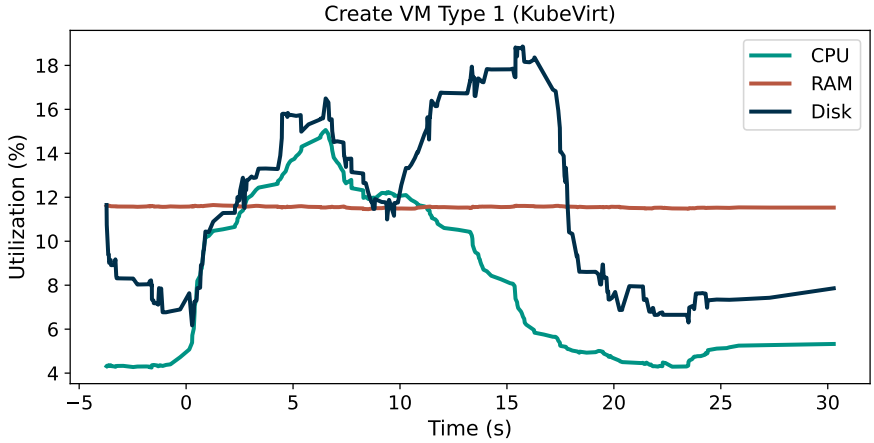

Figure 5.1: Resource utilization on the control node when creating a VM type 1 in KubeVirt. CPU and disk utilization follow a similar pattern nearly reaching 20% while memory utilization remains constant at 12% for the duration of the test.

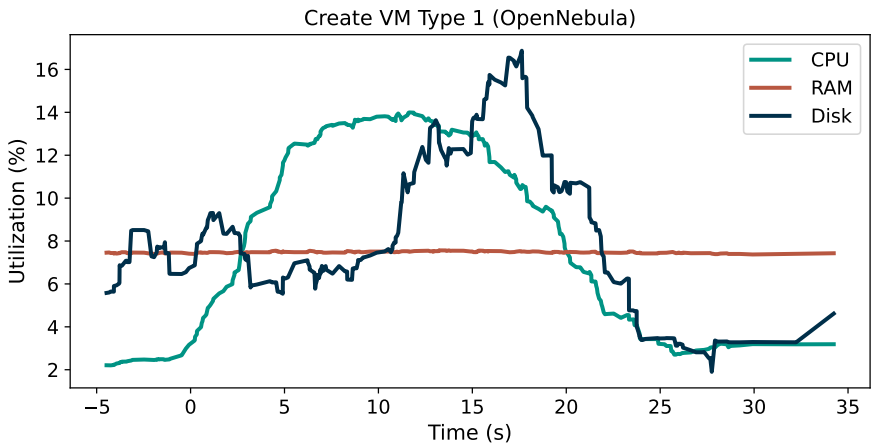

Figure 5.2: Resource utilization on the control node when creating a VM type 1 in OpenNebula. CPU and disk utilization both increase and peak after half the duration of the test. Memory utilization remains constant below 8% for the duration of the test.

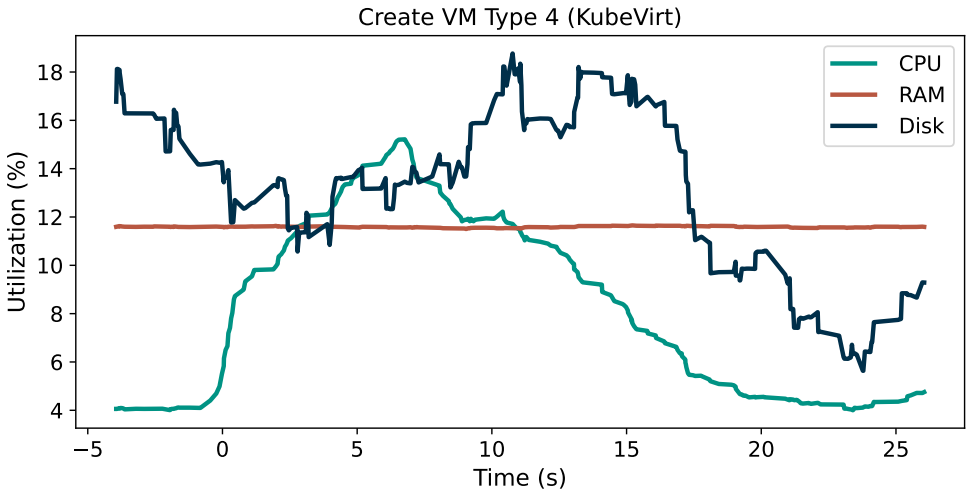

Figure 5.3: Resource utilization on the control node when creating a VM type 4 in KubeVirt. CPU utilization peaks quickly while disk utilization is sporadic. The sporadic behavior is not in line with the operations performed during the test. Memory utilization remains constant at 12% for the duration of the test.

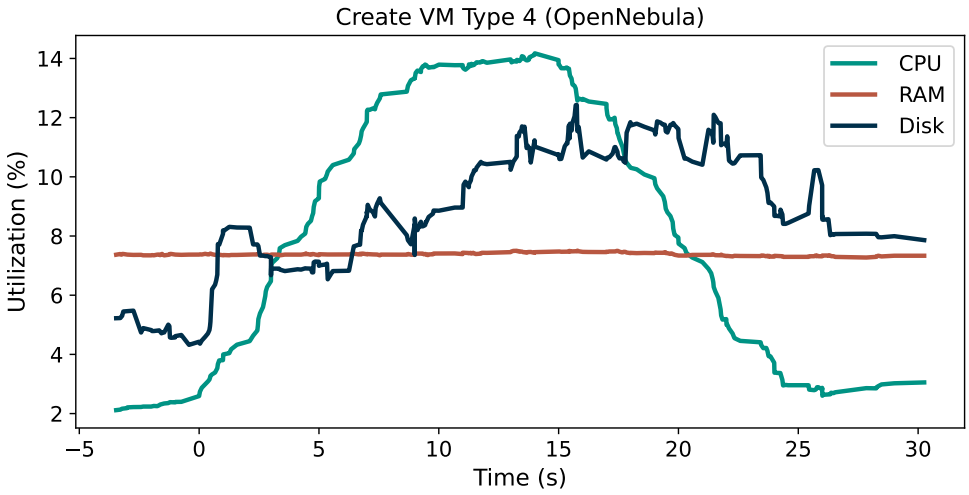

Figure 5.4: Resource utilization on the control node when creating a VM type 4 in OpenNebula. CPU utilization is the most prominent among the resources plateauing at nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

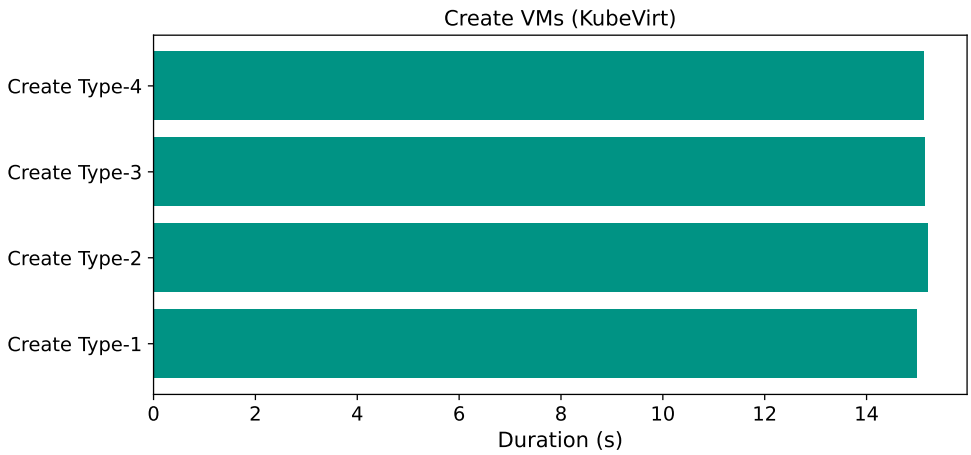

Figure 5.5: Duration of creating VMs in KubeVirt. The duration for all types remains constant for each VM type reaching slightly above 14 seconds

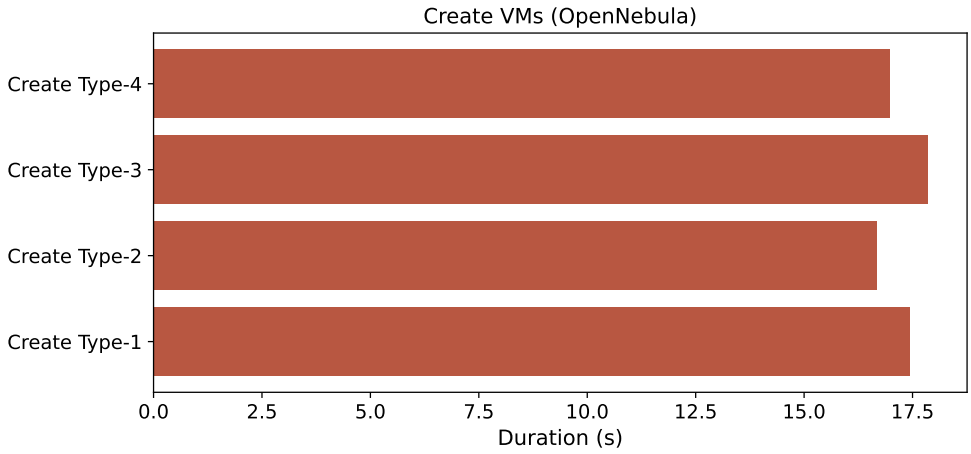

Figure 5.6: Duration of creating VMs in OpenNebula. The duration for all types remains constant for each VM type reaching nearly 18 seconds.

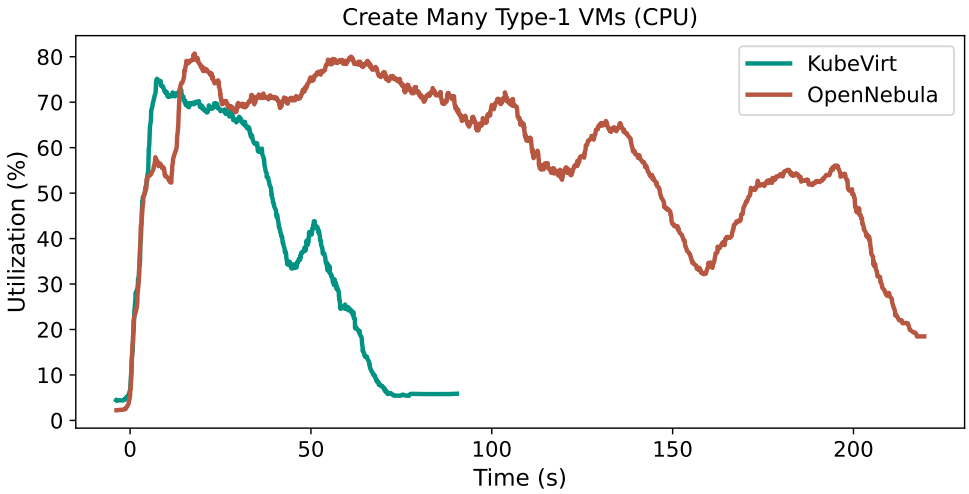

Figure 5.7: CPU utilization on the control node when creating 20 type-1 VMs. The CPU utilization peaks quickly for both KubeVirt and OpenNebula due to the burst of requests. KubeVirt reaches a slightly lower peak than OpenNebula and subsides significantly faster. OpenNebula does not manage to return to a utilization level of an idle state within the duration of the test, as evidenced by the utilization not reaching base utilization in the end.

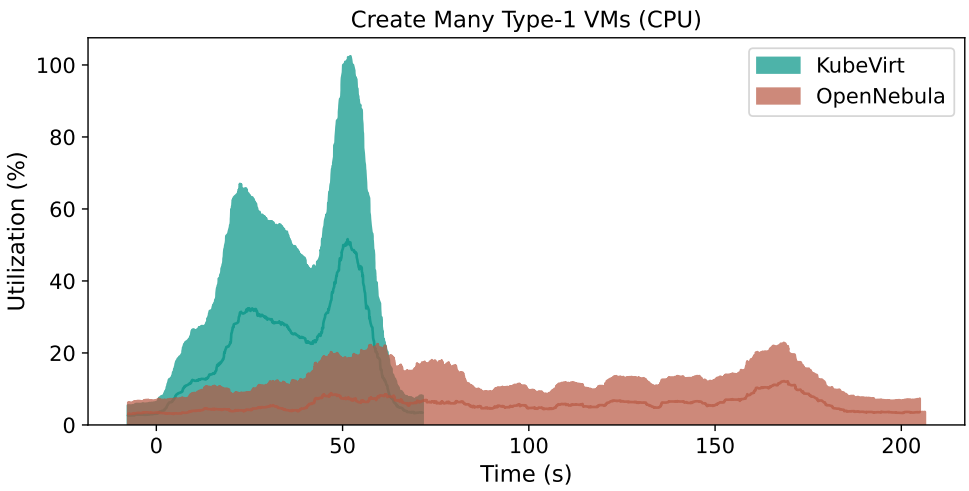

Figure 5.8: Cumulative CPU utilization among worker nodes when creating 20 type-1 VMs. The CPU utilization among the worker nodes is vastly different between KubeVirt and OpenNebula, where KubeVirt experiences a short-lived large burst of CPU utilization while OpenNebula barely reaches a cumulative 20% utilization.

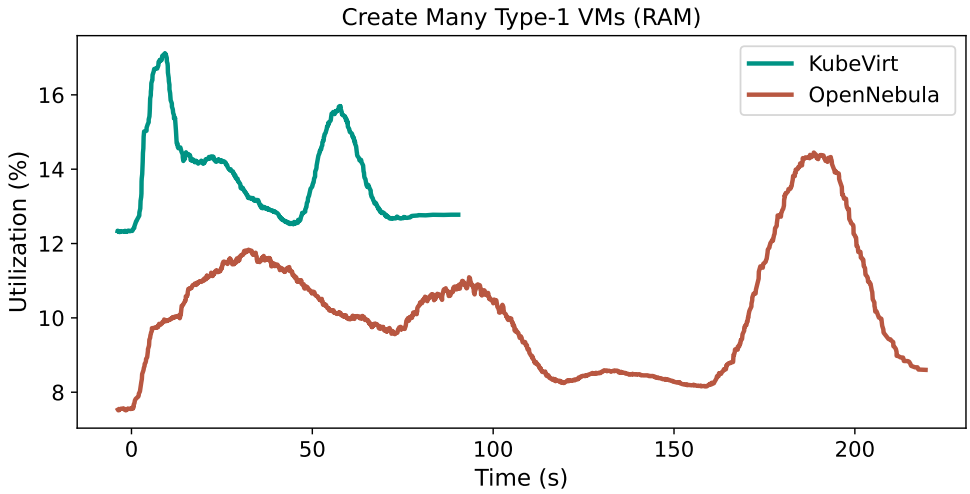

Figure 5.9: Memory utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a burst of memory utilization in the beginning, while OpenNeubla has a more gradual increase before spiking quickly toward the end of the test. While KubeVirt has a larger base memory utilization, OpenNebula experiences a larger induced load from the test, as evidenced by its longer slopes.

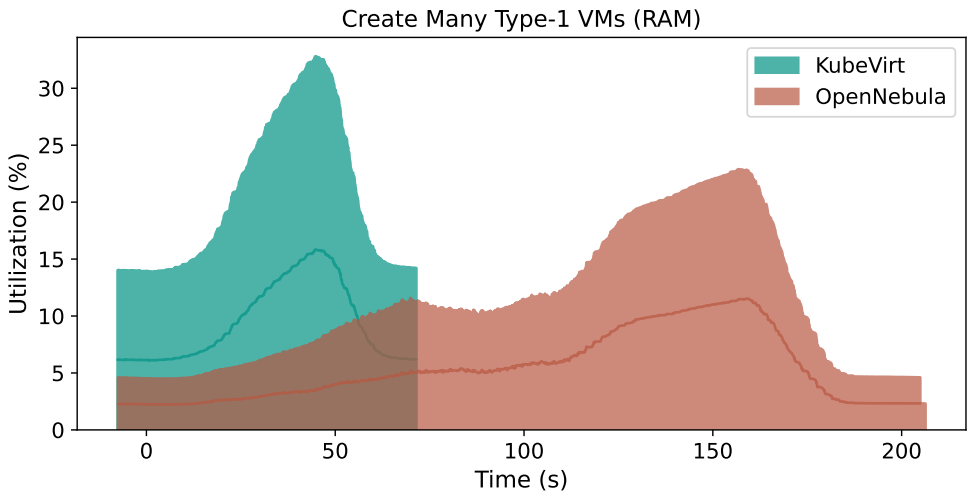

Figure 5.10: Cumulative memory utilization among worker nodes when creating 20 type-1 VMs. While KubeVirt experiences a larger memory utilization due to a larger base, both systems have a similar magnitude of the induced load, albeit KubeVirt's load is more short-lived.

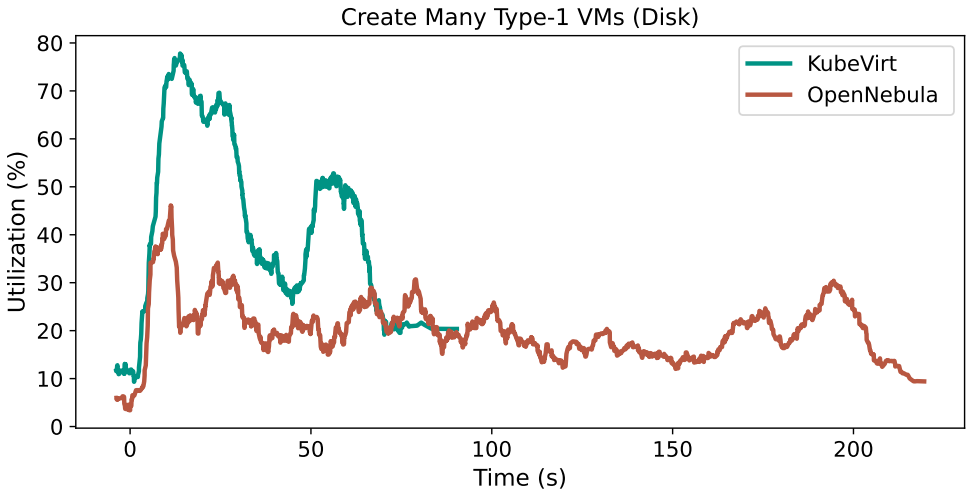

Figure 5.11: Disk utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a steady disk utilization throughout the test.

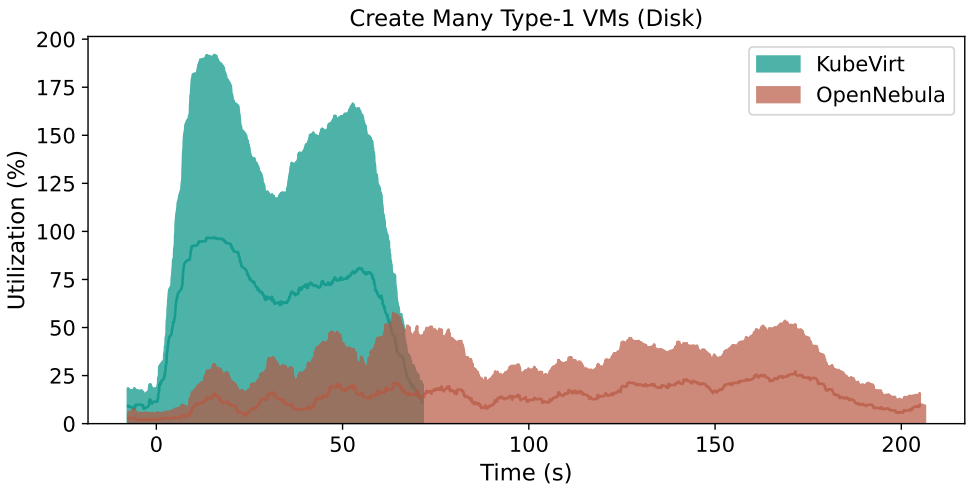

Figure 5.12: Cumulative disk utilization among worker nodes when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a low and steady disk utilization throughout the test.

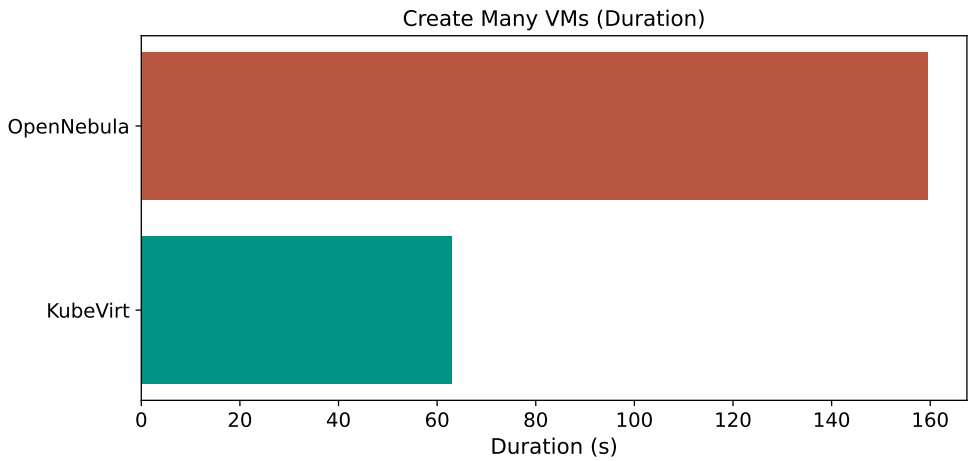

Figure 5.13: Duration of creating 20 type-1 VMs. KubeVirt is significantly faster to create 20 type-1 VMs than OpenNebula, taking 67 seconds compared to OpenNebula's 192 seconds.

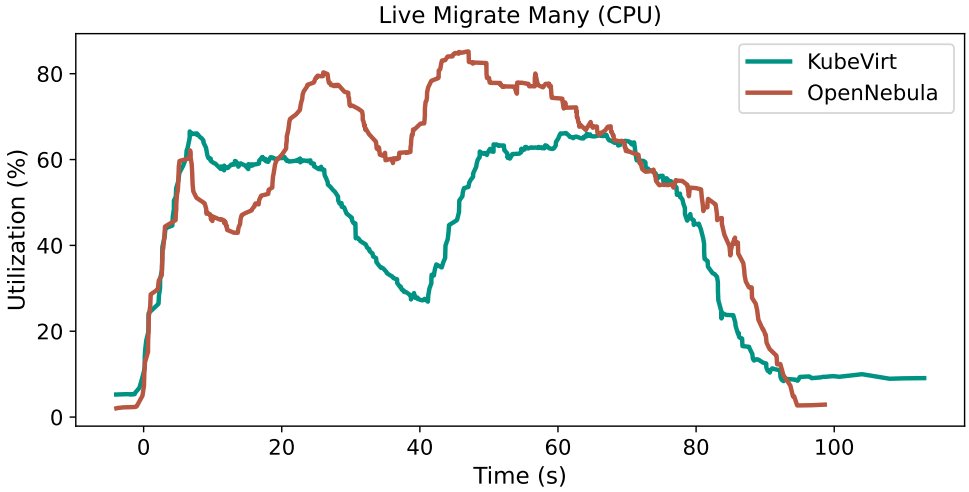

Figure 5.14: CPU utilization on the control node when live migrating 10 type-1 VMs. Both systems follow a similar pattern where KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is larger for OpenNebula.

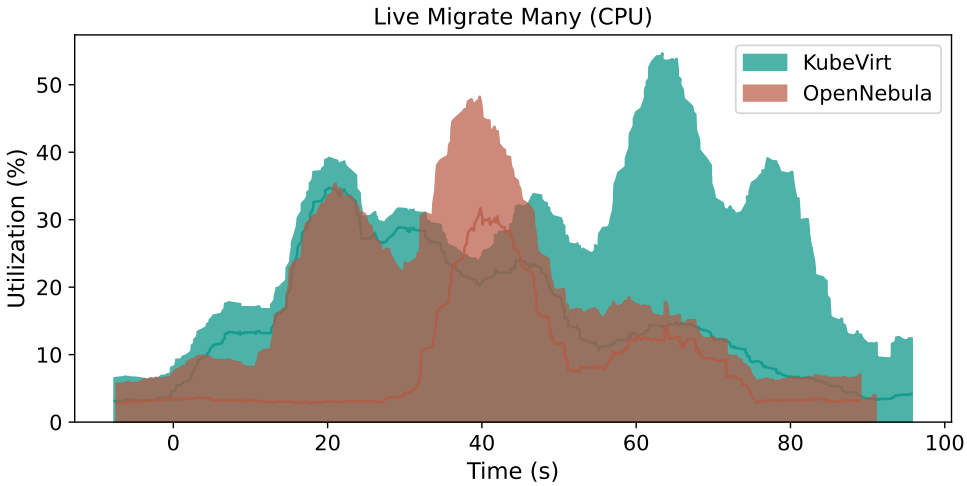

Figure 5.15: Cumulative CPU utilization among worker nodes when live migrating 10 type-1 VMs. Both systems experience a similar load for the first half of the test, but KubeVirt experiences a significantly larger load in the second half of the test.

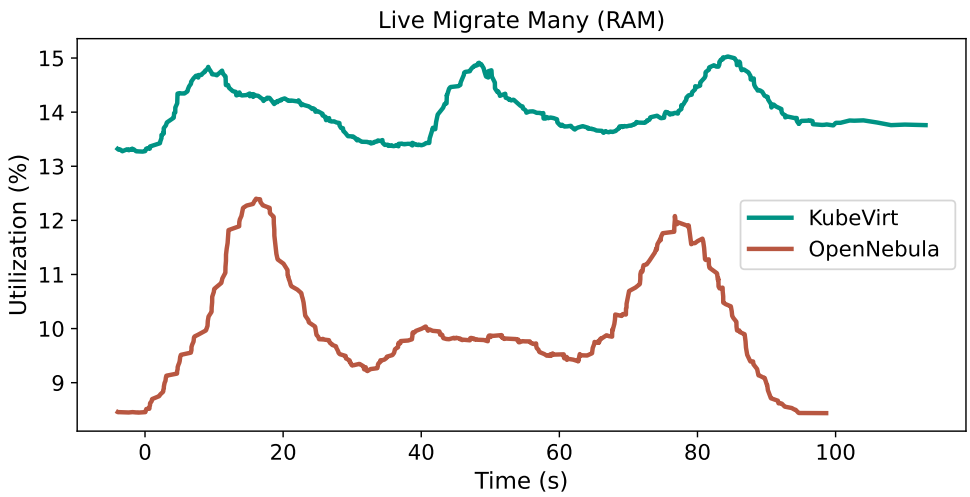

Figure 5.16: Memory utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but the induced load is more than double for OpenNebula.

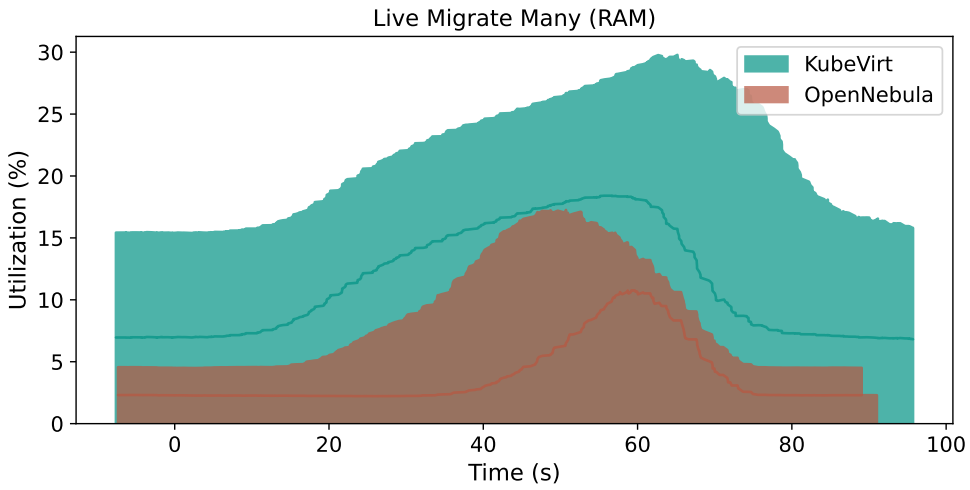

Figure 5.17: Cumulative memory utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but both systems experience a similar induced load. OpenNebula returns to an idle state faster than KubeVirt.

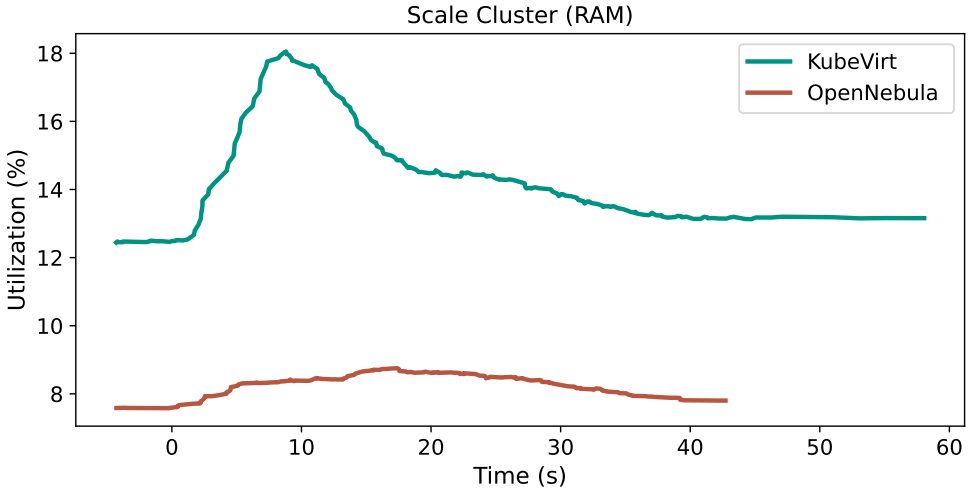

Figure 5.18: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger memory utilization than OpenNebula. OpenNebula's memory utilization is barely affected by the scaling operation.

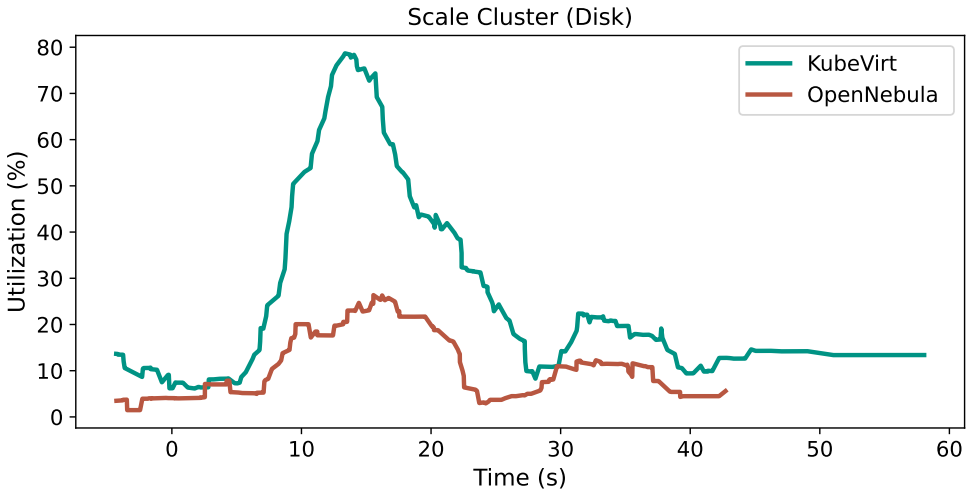

Figure 5.19: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a rapid increase in disk utilization, reaching a peak of over 70% utilization. OpenNebula has a less aggressive increase reaching barely 30% utilization.

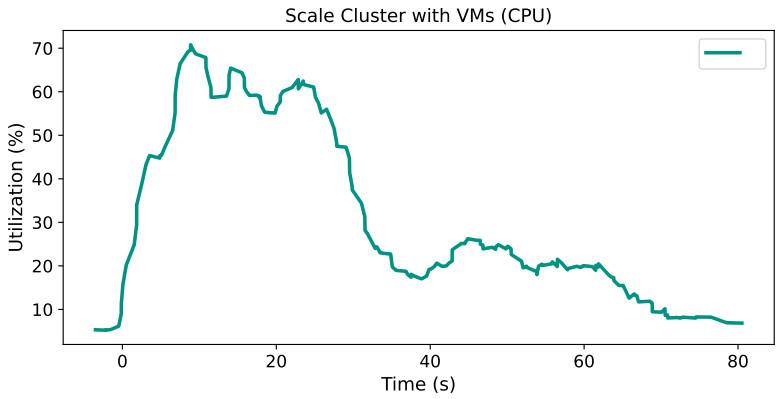

Figure 5.20: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences an initial burst of CPU utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

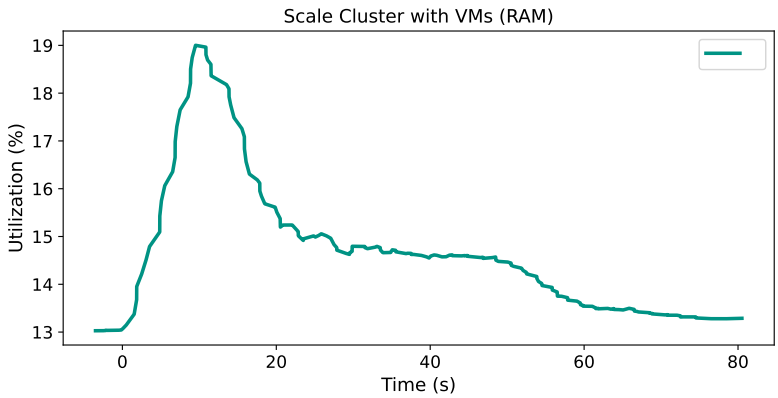

Figure 5.21: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large burst of memory utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation that gradually decreases towards the end. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

5.2 Usability Analysis

The collected answers from the participants are summarized below. Some participants chose a free-hand approach when answering the questions, while other participants chose to answer the questions directly in a structured manner. The full answers from every participant can be found in the appendix D.

Basic Usage

The participants found the process of creating a VM in KubeVirt to be mostly straightforward.

- Participant 1: *The act of doing kubectl apply -f and kubectl delete is quite neat.*
- Participant 2: *Very easy! I like that it is as simple as creating a YAML file and then applying it.*

However, participants expressed some confusion when connecting to the VM using SSH, as it was not clear which SSH public key was used to connect to the VM, and that conflicts with previously created VMs could occur. Moreover, participant 1, in particular, found the VM manifest definition complex, and that the process of creating it was not intuitive.

- Participant 1: *It would be quite difficult for me to figure out I need to use bus: virtio without having someone else tell me I need to.*

KubeVirt's complementary CLI tool *virtctl* was found to be useful when connecting to the VM, as it provided a single tool for both VNC and SSH.

- Participant 2: *Very simple! Simply doing virtual ssh cirros@my-vm and then typing in the password and then I was in!*

Maintenance

The maintenance tasks received a broad range of feedback. Some of the participants found the process of live migration to be simple, while other participants deemed the process to be overly complex and non-intuitive. Participant 1 in particular regarded many of the operations as flawed, such as the live migration object not being deleted after the migration was completed, but also that

KubeVirt lacked proper documentation around the practical aspects of how to perform a live migration. Similar to the basic usage tasks, the participants experienced issues with connecting to VMs using SSH, as it was not clear which SSH public key was used to connect to the VM.

- Participant 2: *I had to delete the entry from /.ssh/kubevirt_known_hosts each time I restarted the VM because virtual ssh complained about a key mismatch.*

Debugging

Compared to both the basic tasks and maintenance, the debugging tasks received the most negative feedback. The participants found the process of debugging to be complex and unclear. Finding the correct documentation was another issue some participants faced, as both the official documentation and the *virtctl –help* command did not provide the necessary information to solve the issue.

- Participant 1: *I did not manage to solve the key mismatch issue as there is very little in the way of debugging virtctl ssh.*

However, while debugging the issue presented in the task was found to be complex, participant 3 regarded the YAML files in general as a positive aspect of debugging.

- Participant 3: *Debugging the issue was easy because the manifest uses standard constructs that are easily googleable.*

Overall

The feedback to KubeVirt was in general mixed. While the workflow was found to be intuitive due to its similarities with ordinary Kubernetes resources, it was also found to be complex with a steep learning curve. The documentation was found to be lacking practical examples of how to get started outside the scope of the given tasks and instead focused more on the operational aspects of KubeVirt.

Chapter 6

Discussion

The research questions involved investigating potential efficiencies that could be gained by integrating VMs in Kubernetes using KubeVirt. This project has evaluated KubeVirt using two different approaches: a performance analysis and a usability analysis. The discussion aims to explore deeper into the results of the project to provide insight into how feasible KubeVirt is as a VM management system.

6.1 Limited Features and Result Intepretation

The goal of the performance analysis was to find factual and provable claims regarding the performance of a VM management system. This included creating a framework that could be used to craft environments that were both reusable and fair among the systems being evaluated. The framework also provided a common interface for which an implementation was created per VM management system. By definition, such a system is only as good as the implementations are, which is why the implementations were designed to be as similar as possible to ensure a fair comparison. However, certain features and settings were not able to be tuned in the same way, such as the scheduling configuration where OpenNebula provides only a single option for scheduling: the number of VMs that can be dispatched concurrently. Not being able to accurately configure the systems identically in every aspect leaves the interpretation of the results open for discussion. This project has chosen the approach where if the system does not provide a specific feature to be tuned, the system is considered to be as optimized as possible. This approach was chosen as it favors the VM management system that provides the most features and flexibility, and as such, the results should be interpreted with this in

mind. Having this aspect in mind is particularly important when interpreting the *Create Many Type-1 VMs* tests, see figures 5.7, 5.9, 5.11 and 5.13. KubeVirt, using the scheduler built into Kubernetes, was able to create 20 type-1 VMs in 67 seconds, while it took OpenNebula 192 seconds on average. However, the results also suggest that KubeVirt is more resource-intensive, as the utilization of CPU, memory and disk was significantly higher than OpenNebula, which can viewed as a trade-off between the duration to complete a task and the resources required to complete it. This project has chosen the duration to be the more important metric, as it indicates that the system is capable of handling a burst of requests efficiently and using its allocated resources to its fullest potential.

It is not always the case that KubeVirt's larger resource utilization is short-lived compared to OpenNebula, where some results suggest that KubeVirt has a significantly larger disk utilization than OpenNeubla for a similar period. This is particularly evident in figure 5.19 where KubeVirt and OpenNebula follow a similar characteristic when scaling the cluster but KubeVirt disk utilization is significantly more intense. For this specific test, the result suggests that KubeVirt does not provide a trade-off between the duration to complete the task and the resources required to complete it. This could be due to the way KubeVirt implements adding additional nodes to the cluster, which could be a resource-intensive operation. Another explanation could be that the specific Kubernetes distribution used in the tests is not optimized for scaling the cluster. However, exploring solutions for this is outside the scope of this project.

The final test used in this project included scaling the cluster with VMs running on each node. It involved scaling the cluster from 2 worker nodes up to 10 worker nodes, then creating a type-1 VM on each worker node before scaling it down to 2 worker nodes. This caused the VM management system to migrate the VMs to available nodes. This test posed a challenge for the scheduler of both systems since a VM might be scheduled to the wrong host if the scale-down operation was not handled immediately. The test was also an extension to the *Scale Cluster* test that involved the same steps but without the VMs. The result shows that OpenNebula was not able to complete the test due to some VMs not being migrated to the correct host, and instead remained on the host that was requested to be removed. Since the test was designed to disconnect the worker and then wait for each host to be empty, the OpenNebula implementation hung indefinitely. A possible explanation for this could be that OpenNebula attempted to move a VM from a host that disconnected to another host that was just about to disconnect, but hadn't yet, and could cause the receiving host to end up in a bad state. Another explanation could be that

OpenNebula requires additional control plane nodes to handle such bursts for migration operation since the test environment only provided a single control plane node to reduce the cost of the test since the nodes were hosted in Azure. However, this is purely speculative and exploring the cause of the issue is outside the scope of this project. The issue of VMs stuck on hosts could be resolved by manual intervention, but since this was deemed unfair to the other system, OpenNebula was disqualified from the scale-with-VMs test. As such, the result for scaling with VMs was omitted for OpenNebula and only KubeVirt was evaluated for this test.

6.2 Kubernetes Overhead and Induced Load

The results of the performance analysis indicate that both KubeVirt and OpenNebula have a consistent overhead and that it differs between the systems. Having a larger overhead implies the system has more ongoing computational work in the background while not performing a requested operation. This is particularly evident in the tests creating different VM-types, see figures 5.1, 5.2, 5.3 and 5.4. KubeVirt CPU utilization hovers around 4% with a memory utilization of just below 12%. OpenNebula, on the other hand, has a CPU utilization just above 2% and a RAM utilization of 8%. This suggests that OpenNebula has a lower required overhead to maintain the system compared to KubeVirt. The overhead is also evident in the tests including live migration, see 5.16. OpenNebula starts the test utilizing just above 8% of the available memory while KubeVirt uses over 13%. However, while this would suggest that KubeVirt is less resource-efficient than OpenNebula, it is important to first consider the surrounding factors. Firstly, since Kubernetes was originally designed to manage containers, it has a different set of requirements compared to a traditional VM management system such as OpenNebula. Certain operations enabling container orchestration might not be necessary for a VM management system, which could manifest as an overhead. Secondly, the overhead shown in the results is not necessarily connected to KubeVirts implementation but rather Kubernetes itself, which could in turn be tied to the specific Kubernetes distribution used in the tests. However, exploring possible optimizations to reduce such overhead or the source of the overhead is outside the scope of this project. Lastly, as described in 2.6.1, KubeVirt achieves VM hosting in Kubernetes by using custom resource and controllers that wraps a VM instance inside a Pod. While this approach allows for seamless integration of VMs in Kubernetes, it also introduces an unavoidable overhead. In conclusion, the overhead is a trade-off for the flexibility and features that KubeVirt provides, and the results

suggest that KubeVirt is more resource-intensive than OpenNebula when not performing operations.

The flip side of this is that many of the results suggest the opposite regarding induced load. Induced load is the added load on the system when performing a specific operation and is calculated as the difference between the total utilization and the base utilization. This can be difficult to interpret since a large induced load over a short period is not necessarily worse than a small induced load over a longer period. However, this project has chosen a short-lived burst of utilization to be favorable over a longer period of low to medium utilization since it suggests that the system is capable of using its allocated resources to its fullest extent. The results suggest that KubeVirt has a lower induced load in general than OpenNebula, where in many cases the induced load is more than double for OpenNebula compared to KubeVirt. This is particularly evident in figure 5.7 where OpenNebula both take twice as long to complete the test and have twice as large induced load from the operation, around 6% compared to KubeVirt's 3%. The difference in induced load is further illustrated in figure 5.14. Not only does it show that OpenNebula reaches an induced load of 76% while KubeVirt reaches just above 50%, but also that induced load is more prominent in the total utilization for both systems. This suggests that induced load is a more significant factor when evaluating the performance of VM management systems and, as such, should be considered when interpreting the results. Figure 6.1 further illustrates this point where for OpenNebula, the induced load is a significant part of the total utilization. Having a smaller induced load suggests that the system is better at scale while having a smaller base overhead suggests that the system is more efficient when the system is idle or performing few operations. However, whether this is true for all operations a VM management system offers remains to be investigated.

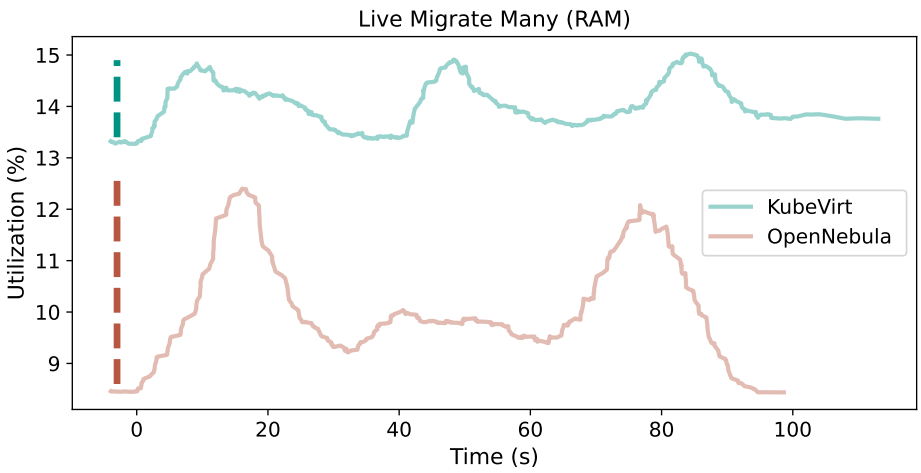

Figure 6.1: Induced memory utilization on the control node when live migrating 10 type-1 VMs. The induced load is a more significant part of the total utilization for OpenNebula than for KubeVirt. The figure is based on the results from figure 5.16.

6.3 Jagged Disk Utilization

Throughout the results, there is a recurring and notable pattern specifically tied to disk utilization. The disk utilization graphs are jagged and, in many cases, do not follow the expected pattern of the specific operation performed in the test. This is particularly evident in the *Create VM Type 4* tests, see figures 5.3 and 5.4. The disk utilization for both Kubevirt and OpenNebula is initially larger than expected and does not return to the same state after the test is finished. Since the tests were run multiple times in no particular order, noise was likely introduced in the measurement that caused each test to generate data points in different ranges. When the induced disk utilization was low for a particular test, background disk utilization such as OS-level operation was more prominent in the results. By then applying a rolling mean smoothing function to the dataset, the noise was reduced, but due to the variability, some jagged features remained. This is further supported by the fact that the disk utilization is less jagged for tests with a larger expected disk utilization, such as the *Create Many Type-1 VMs* tests since these tests generate a larger induced load. While the jagged disk utilization is not a problem in itself, it is important to consider when interpreting the results, especially those where disk utilization is small, as it could introduce bias unrelated to the operation performed.

Chapter 7

Conclusions and Future Work

Bibliography

[1] Awodele Oludele et al. "On the evolution of virtualization and cloud computing: A review". In: *Journal of Computer Sciences and Applications* 2.3 (2014), pp. 40–43.

[2] Susanta Nanda Tzi-cker Chiueh and Stony Brook. "A survey on virtualization technologies". In: *Rpe Report* 142 (2005).

[3] M. Rosenblum and T. Garfinkel. "Virtual machine monitors: current technology and future trends". In: *Computer* 38.5 (2005), pp. 39–47. DOI: 10.1109/MC.2005.176.

[4] IBM. *Virtualization*. Feb. 2024. URL: https://www.ibm.com/topics/virtualization.

[5] Ankita Desai et al. "Hypervisor: A survey on concepts and taxonomy". In: *International Journal of Innovative Technology and Exploring Engineering* 2.3 (2013), pp. 222–225.

[6] Vitor Goncalves da Silva, Marite Kirikova, and Gundars Alksnis. "Containers for Virtualization: An Overview." In: *Appl. Comput. Syst.* 23.1 (2018), pp. 21–27.

[7] Dennis Gannon, Roger Barga, and Neel Sundaresan. "Cloud-native applications". In: *IEEE Cloud Computing* 4.5 (2017), pp. 16–21.

[8] Ramzi Debab and Walid Khaled Hidouci. "Containers runtimes war: a comparative study". In: *Proceedings of the Future Technologies Conference (FTC) 2020, Volume 2*. Springer. 2021, pp. 135–161.

[9] Xingyu Wang, Junzhao Du, and Hui Liu. "Performance and isolation analysis of RunC, gVisor and Kata Containers runtimes". In: *Cluster Computing* 25.2 (2022), pp. 1497–1513.

[10] OpenStack Foundation. *OpenStack*. Accessed 2024-03-06. Mar. 2024. URL: https://www.openstack.org/.

[11] Apache CloudStack. *Apache CloudStack*. Accessed 2024-03-06. Mar. 2024. URL: https://cloudstack.apache.org/.

[12] OpenNebula. *OpenNebula*. Accessed 2024-03-06. Mar. 2024. URL: https://opennebula.io/.

[13] Kubernetes. *Kubernetes*. Accessed 2024-03-06. Mar. 2024. URL: https://kubernetes.io/.

[14] HashiCorp. *Nomad*. Accessed 2024-03-06. Mar. 2024. URL: https://www.nomadproject.io/.

[15] Docker. *Docker Swarm*. Accessed 2024-03-06. Mar. 2024. URL: https://docs.docker.com/engine/swarm/.

[16] Subrota Kumar Mondal et al. "Kubernetes in IT administration and serverless computing: An empirical study and research challenges". In: *The Journal of Supercomputing* (2022), pp. 1–51.

[17] Guoqing Li et al. "The Convergence of Container and Traditional Virtualization: Strengths and Limitations". In: *SN Computer Science* 4.4 (May 2023). https://doi.org/10.1007/s42979-023-01827-9, p. 387. ISSN: 2661-8907. DOI: 10.1007/s42979-023-01827-9.

[18] The KubeVirt.io website CNCF. *Kubevirt.io*. Jan. 2024. URL: https://kubevirt.io/.

[19] Zou Nengren September 22, Ambassador post by Zou Nengren, and September 22. *Best practices for transitioning VM clusters to KubeVirt 1.0*. Sept. 2023. URL: https://www.cncf.io/blog/2023/09/22/best-practices-for-transitioning-vm-clusters-to-kubevirt-1-0/.

[20] Marcin Kubacki. *what is Kubevirt*. Feb. 2024. URL: https://storware.eu/blog/what-is-kubevirt/.

[21] Fred Douglis and Orran Krieger. "Virtualization". In: *IEEE Internet Computing* 17.2 (2013), pp. 6–9. DOI: 10.1109/MIC.2013.42.

[22] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Full and para-virtualization with Xen: a performance comparison". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.9 (2013), pp. 719–727.

[23] Tim Abels, Puneet Dhawan, and Balasubramanian Chandrasekaran. "An overview of xen virtualization". In: *Dell Power Solutions* 8 (2005), pp. 109–111.

[24] Carl Boettiger. "An introduction to Docker for reproducible research". In: *SIGOPS Oper. Syst. Rev.* 49.1 (Jan. 2015). https://doi.org/10.1145/2723872.2723882, pp. 71–79. ISSN: 0163-5980. DOI: 10.1145/2723872.2723882.

[25] Lars Rasmusson and Diarmuid Corcoran. "Performance overhead of KVM on Linux 3.9 on ARM cortex-a15". In: *SIGBED Rev.* 11.2 (Sept. 2014). https://doi.org/10.1145/2668138.2668143, pp. 32–38. DOI: 10.1145/2668138.2668143.

[26] Stephen J. Bigelow. *What's the difference between type 1 vs. type 2 hypervisor?: TechTarget*. Jan. 2021. URL: https://www.techtarget.com/searchitoperations/tip/Whats-the-difference-between-Type-1-vs-Type-2-hypervisor.

[27] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[28] Jordan Shropshire. "Analysis of monolithic and microkernel architectures: Towards secure hypervisor design". In: *2014 47th Hawaii International Conference on System Sciences*. IEEE. 2014, pp. 5008–5017.

[29] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[30] Pankaj Kadam and Nilesh Alone. "Review on KVM Hypervisor". In: *International Journal of Recent Technology and Engineering (IJRTE) ISSN* (2014), pp. 2277–3878.

[31] David Freet et al. "Open source cloud management platforms and hypervisor technologies: A review and comparison". In: *SoutheastCon 2016* (2016), pp. 1–8.

[32] Xiaolong Wen et al. "Comparison of open-source cloud management platforms: OpenStack and OpenNebula". In: *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery*. IEEE. 2012, pp. 2457–2461.

[33] Google Cloud. *High Performance Computing*. Accessed 2024-03-19. Mar. 2024. URL: https://cloud.google.com/discover/what-is-high-performance-computing.

[34] Oracle. *High Performance Computing*. Accessed 2024-03-19. Mar. 2024. URL: https://www.oracle.com/se/cloud/hpc/what-is-hpc/.

[35] Microsoft. *Availability Zones*. Accessed 2024-03-19. Mar. 2024. URL: https://learn.microsoft.com/en-us/azure/reliability/availability-zones-overview?tabs=azure-cli.

[36] Maged Michael et al. "Scale-up x scale-out: A case study using nutch/lucene". In: *2007 IEEE International Parallel and Distributed Processing Symposium*. IEEE. 2007, pp. 1–8.

[37] IBM. *Cloud scalability: Scale-up vs. scale-out*. Accessed 2024-03-19. Mar. 2024. URL: https://www.ibm.com/blog/cloud-scalability-scale-up-vs-scale-out/.

[38] Sunilkumar S Manvi and Gopal Krishna Shyam. "Resource management for Infrastructure as a Service (IaaS) in cloud computing: A survey". In: *Journal of network and computer applications* 41 (2014), pp. 424–440.

[39] Chnar Mustafa Mohammed and Subhi RM Zeebaree. "Sufficient comparison among cloud computing services: IaaS, PaaS, and SaaS: A review". In: *International Journal of Science and Business* 5.2 (2021), pp. 17–30.

[40] Tiago Rosado and Jorge Bernardino. "An overview of openstack architecture". In: *Proceedings of the 18th International Database Engineering & Applications Symposium*. 2014, pp. 366–367.

[41] OpenStack Foundation. *OpenStack Live Migration*. Feb. 2024. URL: https://docs.openstack.org/nova/pike/admin/live-migration-usage.html.

[42] Rakesh Kumar et al. "Apache cloudstack: Open source infrastructure as a service cloud computing platform". In: *Proceedings of the International Journal of advancement in Engineering technology, Management and Applied Science* 111 (2014), p. 116.

[43] Apache CloudStack. *CloudStack Live Migration*. Feb. 2024. URL: https://svn.apache.org/repos/asf/cloudstack/docsite/html/docs/en-US/Apache_CloudStack/4.2.0/html/Admin_Guide/manual-live-migration.html.

[44] Daniel Moritz et al. "Enhancing a virtual security lab with a private cloud framework". In: *Proceedings of 2013 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*. IEEE. 2013, pp. 314–320.

[45] Younes Khair, Abdeslam Dennai, and Youssef Elmir. "An experimental performance evaluation of OpenNebula and eucalyptus cloud platform solutions". In: *Artificial Intelligence and Heuristics for Smart Energy Efficiency in Smart Cities: Case Study: Tipasa, Algeria*. Springer. 2022, pp. 450–457.

[46] Anurag Choudhary, Pradeep Kumar Verma, and Piyush Rai. "A walkthrough of Amazon Elastic Compute Cloud (Amazon EC2): A Review". In: *International Journal for Research in Applied Science and Engineering Technology* 9.11 (2021), pp. 93–97.

[47] Amazon Web Services. *Amazon EC2*. Feb. 2024. URL: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html.

[48] Pallavi Wankhede, Minaiy Talati, and Rutuja Chinchamalatpure. "Comparative study of cloud platforms-microsoft azure, google cloud platform and amazon EC2". In: *J. Res. Eng. Appl. Sci* 5.02 (2020), pp. 60–64.

[49] Marshall Copeland et al. "Getting started with Azure virtual machines". In: *Microsoft Azure: Planning, Deploying, and Managing Your Data center in the Cloud* (2015), pp. 147–176.

[50] Rawan Aljamal, Fahed Jubair, and Ali El-Mousa. "Analyzing the Cost-Effectiveness of Running HPC Applications on Microsoft Azure Virtual Machines". In: *2021 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)*. IEEE. 2021, pp. 78–82.

[51] Microsoft. *Azure Virtual Machines*. Feb. 2024. URL: https://learn.microsoft.com/en-us/azure/virtual-machines/overview.

[52] TechTarget. *Google Compute Engine*. Feb. 2024. URL: https://www.techtarget.com/searchaws/definition/Google-Compute-Engine#:~:text=Google%20Compute%20Engine%20(GCE)%20is,compute%20clusters%20for%20that%20purpose..

[53] SPT Krishnan et al. "Google compute engine". In: *Building Your Next Big Thing with Google Cloud Platform: A Guide for Developers and Enterprise Architects* (2015), pp. 53–81.

[54] Solanke Vikas et al. "Private vs public cloud". In: *International Journal of Computer Science & Communication Networks* 3.2 (2013), p. 79.

[55] Guruh Aryotejo, Daniel Y Kristiyanto, et al. "Hybrid cloud: bridging of private and public cloud computing". In: *Journal of Physics: Conference Series*. Vol. 1025. 1. IOP Publishing. 2018, p. 012091.

[56] Khouloud Boukadi et al. "From VM to container: A linear program for outsourcing a business process to cloud containers". In: *On the Move to Meaningful Internet Systems. OTM 2017 Conferences: Confederated International Conferences: CoopIS, C&TC, and ODBASE 2017, Rhodes, Greece, October 23-27, 2017, Proceedings, Part I*. Springer. 2017, pp. 488–504.

[57] Nane Kratzke. "About microservices, containers and their underestimated impact on network performance". In: *arXiv preprint arXiv:1710.04049* (2017).

[58] Arnaldo Pereira Ferreira and Richard Sinnott. "A performance evaluation of containers running on managed kubernetes services". In: *2019 IEEE international conference on cloud computing technology and science (CloudCom)*. IEEE. 2019, pp. 199–208.

[59] Docker. *Layers*. Feb. 2024. URL: `https://docs.docker.com/build/guide/layers/`.

[60] Docker. *Dockerfile reference*. Feb. 2024. URL: `https://docs.docker.com/reference/dockerfile/`.

[61] Carl Boettiger. "An introduction to Docker for reproducible research". In: *ACM SIGOPS Operating Systems Review* 49.1 (2015), pp. 71–79.

[62] Steve Buchanan et al. "Container registries". In: *Introducing Azure Kubernetes Service: A Practical Guide to Container Orchestration* (2020), pp. 17–34.

[63] Red Hat. *What is a container registry?* Feb. 2024. URL: `https://www.redhat.com/en/topics/cloud-native-apps/what-is-a-container-registry`.

[64] Open Container Initiative. *Open Container Initiative*. Feb. 2024. URL: `https://opencontainers.org/`.

[65] Lennart Espe et al. "Performance Evaluation of Container Runtimes." In: *CLOSER*. 2020, pp. 273–281.

[66] Kubernetes. *Pods*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/pods/.

[67] Kubernetes. *Services*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/services-networking/service/.

[68] Kubernetes. *Deployments*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/.

[69] Arun Gurta. "Kubernetes". In: *Kubernetes. Retrieved May* 24 (2019), p. 2019.

[70] Kubernetes. *StatefulSets*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/.

[71] Kakoli Banerjee et al. "A Survey on Kubernetes Policy Report Custom Resource Definition Kube-Bench Adapter". In: *Advances in Data and Information Sciences: Proceedings of ICDIS 2022*. Springer, 2022, pp. 315–322.

[72] Kubernetes. *Custom Resource Definitions*. Feb. 2024. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/.

[73] Kubernetes. *Controllers*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/concepts/architecture/controller/.

[74] Kubernetes. *kubelet*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/.

[75] Kubernetes. *kube-proxy*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/.

[76] Prateek Sharma et al. "Containers and virtual machines at scale: A comparative study". In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[77] Alessandro Randazzo and Ilenia Tinnirello. "Kata containers: An emerging architecture for enabling mec services in fast and secure way". In: *2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*. IEEE. 2019, pp. 209–214.

[78] Shashank Mohan Jain and Shashank Mohan Jain. "Cgroups". In: *Linux Containers and Virtualization: A Kernel Perspective* (2020), pp. 45–80.

[79] Amazon Web Services. *Noisy Neighbor*. Feb. 2024. URL: https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/noisy-neighbor.html.

[80] Prateek Sharma et al. "Containers and virtual machines at scale: A comparative study". In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[81] Jinchun Kim et al. "Dynamic memory pressure aware ballooning". In: *Proceedings of the 2015 International Symposium on Memory Systems*. 2015, pp. 103–112.

[82] Ishan Banerjee et al. "Memory overcommitment in the ESX server". In: *VMware Technical Journal* 2.1 (2013), pp. 2–12.

[83] Christopher Clark et al. "Live migration of virtual machines". In: *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*. 2005, pp. 273–286.

[84] Andrey Mirkin, Alexey Kuznetsov, and Kir Kolyshkin. "Containers checkpointing and live migration". In: *Proceedings of the Linux Symposium*. Vol. 2. 2008, pp. 85–90.

[85] Radostin Stoyanov and Martin J Kollingbaum. "Efficient live migration of linux containers". In: *High Performance Computing: ISC High Performance 2018 International Workshops, Frankfurt/Main, Germany, June 28, 2018, Revised Selected Papers 33*. Springer. 2018, pp. 184–193.

[86] CRIU. *CRIU*. Feb. 2024. URL: https://criu.org/.

[87] Red Hat. *Containers vs VMs*. Feb. 2024. URL: https://www.redhat.com/en/topics/containers/containers-vs-vms.

[88] Kong. *What Are Virtual Machines (VMs)?* Feb. 2024. URL: https://konghq.com/learning-center/service-mesh/virtual-machines.

[89] Chris Greamo and Anup Ghosh. "Sandboxing and virtualization: Modern tools for combating malware". In: *IEEE Security & Privacy* 9.2 (2011), pp. 79–82.

[90] Tripwire. *What is Configuration Drift?* Feb. 2024. URL: https://www.tripwire.com/state-of-security/what-is-configuration-drift.

[91] Shared Rotlevi. *Configuration Drift Explained*. Feb. 2024. URL: https://www.wiz.io/academy/configuration-drift.

[92] Sysdig. *Containers vs. Virtual Machines – Making an Informed Decision*. Feb. 2024. URL: https://sysdig.com/learn-cloud-native/kubernetes-security/containers-vs-virtual-machines-making-an-informed-decision/.

[93] Microsoft. *Virtual machines: virtual computers within computers*. Feb. 2024. URL: https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-a-virtual-machine.

[94] Anjali, Tyler Caraza-Harter, and Michael M Swift. "Blending containers and virtual machines: a study of firecracker and gVisor". In: *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 2020, pp. 101–113.

[95] Anjali, Tyler Caraza-Harter, and Michael M Swift. "Blending containers and virtual machines: a study of firecracker and gVisor". In: *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 2020, pp. 101–113.

[96] Kubernetes. *kube-scheduler*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kube-scheduler/.

[97] KubeVirt. *KubeVirt Architecture*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/architecture/.

[98] KubeVirt. *API Validation*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/api_validation/.

[99] Kubernetes. *Getting to know KubeVirt*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/blog/2018/05/22/getting-to-know-kubevirt/.

[100] KubeVirt. *Snapshot Restore API*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/snapshot_restore_api/.

[101] KubeVirt. *Live Migration*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/live_migration/.

[102] Kubernetes. *Persistent Volumes*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/concepts/storage/persistent-volumes/.

[103] KubeVirt. *virtctl*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/virtctl_client_tool/.

[104] Florian Beetz and Simon Harrer. "Gitops: The evolution of devops?" In: *IEEE Software* 39.4 (2021), pp. 70–75.

[105] KubeVirt. *GitOps*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/gitops/.

[106] Platform9. *KubeVirt: How to Run VMs on Kubernetes*. Tech. rep. Apr. 2020. URL: https://platform9.com/wp-content/uploads/2020/05/KubeVirt-How-to-Run-VMs-on-Kubernetes.pdf.

[107] KubeVirt. *Comparing KubeVirt to other technologies*. Accessed 2024-02-29. Feb. 2017. URL: https://kubevirt.io/2017/technology-comparison.html.

[108] KubeVirt. *Network Binding Plugins*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/network_binding_plugins/.

[109] KubeVirt. *Disks and Volumes*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/disks_and_volumes/.

[110] Supakorn Trakulmaiphol and Krerk Piromsopa. "An Implementation of DNS Operator and Network Guideline for Migrating Virtual Machine to KubeVirt". In: *2022 26th International Computer Science and Engineering Conference (ICSEC)*. 2022, pp. 84–89. DOI: 10.1109/ICSEC56337.2022.10049369.

[111] Mengmei Ye et al. "Free the Turtles: Removing Nested Virtualization for Performance and Confidentiality in the Cloud". In: *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*. 2023, pp. 275–281. DOI: 10.1109/CLOUD60044.2023.00039.

[112] Oskar Sonninen. "Perceived benefits of declarative software deployment: an exploratory case study". In: (2020).

[113] Muhammad Imran et al. "Migration of CMSWEB cluster at CERN to Kubernetes: a comprehensive study". In: *Cluster Computing* 24.4 (2021), pp. 3085–3099.

[114] Rukaiah NJH Badran. *Automation and improvement in CMSWEB Kubernetes clusters*. Tech. rep.

[115] Sravanthi Gelley. "Migrate Cloud Foundry Application to Kubernetes". In: (2022).

[116] Michael Packard et al. "Real-World, Self-Hosted Kubernetes Experience". In: *Practice and Experience in Advanced Research Computing*. 2021, pp. 1–5.

[117] Mikhail Boronin. *Hybrid Cloud Migration Challenges. A case study at King*. 2020.

[118] Mikyung Kang et al. "A comparison of system performance on a private openstack cloud and amazon ec2". In: *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*. IEEE. 2017, pp. 310–317.

[119] Jaison Paul Mullerikkal and Yedhu Sastri. "A comparative study of OpenStack and CloudStack". In: *2015 Fifth International Conference on Advances in Computing and Communications (ICACC)*. IEEE. 2015, pp. 81–84.

[120] Aaron Paradowski, Lu Liu, and Bo Yuan. "Benchmarking the Performance of OpenStack and CloudStack". In: *2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*. 2014, pp. 405–412. DOI: 10.1109/ISORC.2014.12.

[121] Marcelo Amaral. *Kubevirt scale test by creating 400 vmis on a single node for FOSDEM 2022: IBM Research*. Feb. 2022. URL: https://research.ibm.com/publications/kubevirt-scale-test-by-creating-400-vmis-on-a-single-node.

[122] RISSO FULVIO GIOVANNI OTTAVIO and Alireza Kolahdouzan. "Network Automation, Orchestration, Cloud: Tools and Technologies". In: (2019).

[123] Jinho Hwang et al. "A component-based performance comparison of four hypervisors". In: *2013 IFIP/IEEE International Symposium on Integrated Network Management (IM 2013)*. IEEE. 2013, pp. 269–276.

[124] Robert-Steve Schmoll et al. "Comparing and evaluating application-specific boot times of virtualized instances". In: *2019 IEEE 2nd 5G World Forum (5GWF)*. IEEE. 2019, pp. 602–606.

[125] Microsoft. *Dv3 and Dsv3 series*. Accessed 2024-03-20. Mar. 2024. URL: https://learn.microsoft.com/en-us/azure/virtual-machines/dv3-dsv3-series.

[126] Microsoft. *Azure Pricing Calculator*. Feb. 2024. URL: https://azure.microsoft.com/en-us/pricing/calculator/.

Appendix A

Performance Analysis Components

A.1 VM Management System Interface

```
type VmManagementSystem interface {
    Setup() error
    GetVM(name string) *models.VM
    ListVMs() []models.VM
    CreateVM(vm *models.VM, hostIdx ...int) error
    DeleteVM(name string) error
    MigrateVM(name string, hostIdx int) error
    ConnectWorker(workerIdx int) error
    DisconnectWorker(workerIdx int) error
    WaitForRunningVM(name string) error
    DeleteAllVMs() error
}
```

A.2 Metrics Collector

```
#!/bin/bash

interval=1
OUTPUT_FILE="./output.json"

cpu_tmp_file=$(mktemp)
```

```
ram_tmp_file=$(mktemp)
disk_tmp_file=$(mktemp)

while true; do
    if [ ! -f "./run" ]; then
        sleep $interval
        continue
    fi

    (mpstat -P ALL "$interval" 1 | awk '/^[0-9]/␣{if␣(NR
       >1)␣print␣$3/100}' | jq -R -s -c 'split("\n")
       [1:-1]' | jq '[.[]␣|␣gsub(",";␣".")␣|␣tonumber]'
       > "$cpu_tmp_file") &

    (free | awk '/Mem:/␣{print␣$3/$2}' | sed 's/,/./' >
       "$ram_tmp_file") &

    (iostat -dx "$disk" 1 2 | awk '/Device/{flag=1;␣next
       }␣flag␣&&␣/^[^␣]/␣{util=$NF}␣END{print␣util/100}'
        | sed 's/,/./' > "$disk_tmp_file") &
    timestamp=$(date -u +"%Y-%m-%dT%H:%M:%SZ")

    wait

    name=$(hostname)
    cpu_core_utilization=$(cat "$cpu_tmp_file")
    cpu_utilization_avg=$(echo "$cpu_core_utilization" |
        jq 'add/length')
    ram_utilization=$(cat "$ram_tmp_file")
    disk_utilization=$(cat "$disk_tmp_file")

    # Create the JSON object for the current datapoint
    datapoint=$(jq -n \
                  --argjson cpuCores "
                     $cpu_core_utilization" \
                  --argjson cpuAverage "
                     $cpu_utilization_avg" \
                  --arg name "$name" \
                  --arg ramUtilization "$ram_utilization
                     " \
                  --arg diskUtilization "
                     $disk_utilization" \
```

```
                    --arg timestamp "$timestamp" \
                    '{name:␣$name,␣cpu:␣{cores:␣$cpuCores,
                       ␣average:␣$cpuAverage},␣ram:␣(
                       $ramUtilization␣|␣tonumber),␣disk:␣
                       ($diskUtilization␣|␣tonumber),␣
                       timestamp:␣$timestamp}')

    if [ ! -f "$OUTPUT_FILE" ]; then
        echo "[]" > "$OUTPUT_FILE"
    fi

    jq --argjson newData "$datapoint" '.␣+=␣[$newData]'
       "$OUTPUT_FILE" > tmp.$$.json && mv tmp.$$.json "
       $OUTPUT_FILE"

    sleep $interval
done
```

A.3 Metrics Collector Daemon

```
[Unit]
Description=Metrics scraping
[Service]
WorkingDirectory=/home/user
ExecStart=/bin/bash /home/user/metrics.sh
Restart=always
[Install]
WantedBy=multi-user.target
```

Appendix B

Performance Analysis Results

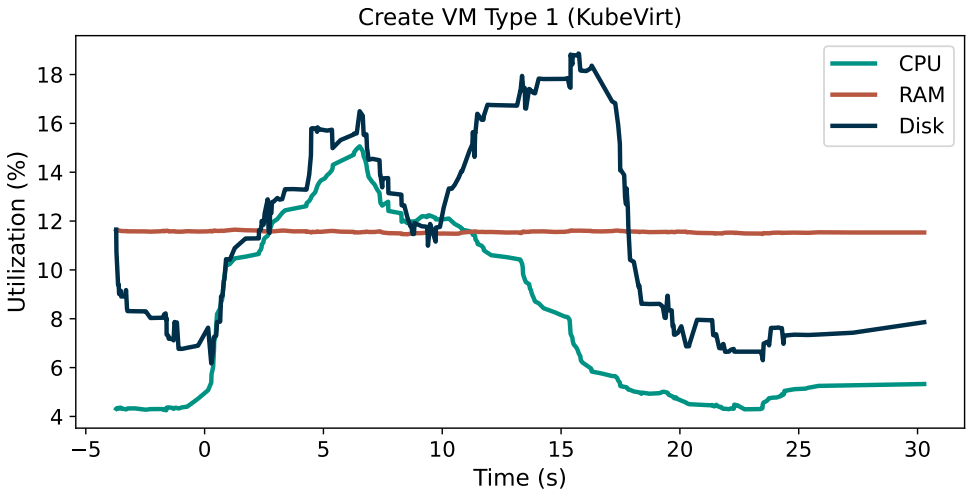

Figure B.1: Resource utilization on the control node when creating a VM type 1 in KubeVirt. CPU and disk utilization follow a similar pattern nearly reaching 20% while memory utilization remains constant at 12% for the duration of the test.

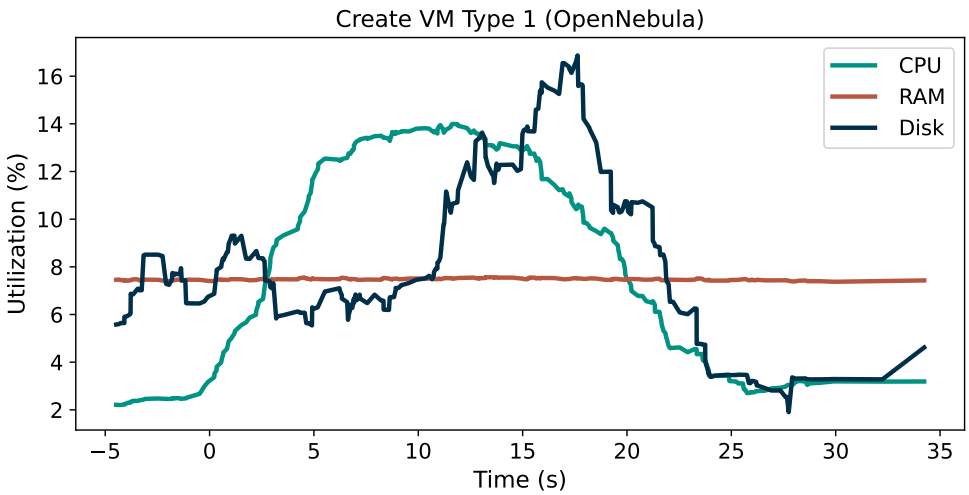

Figure B.2: Resource utilization on the control node when creating a VM type 1 in OpenNebula. CPU and disk utilization both increase and peak after half the duration of the test. Memory utilization remains constant below 8% for the duration of the test.

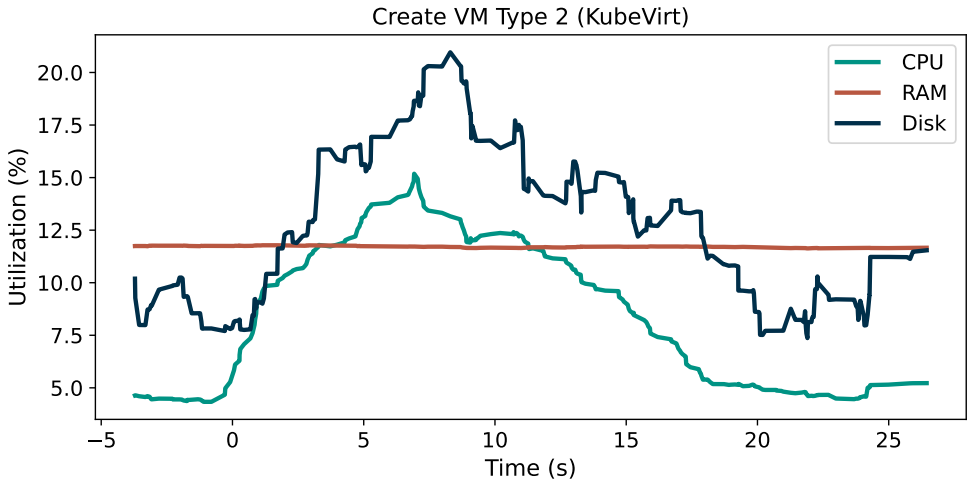

Figure B.3: Resource utilization on the control node when creating a VM type 2 in KubeVirt. The disk utilization is the most prominent among the resources, reaching over 20% utilization. Memory utilization remains constant at 12% for the duration of the test.

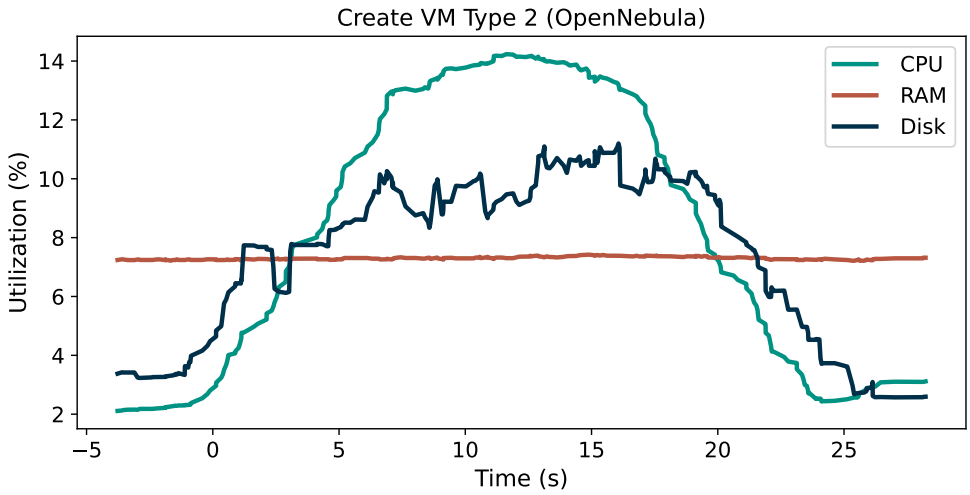

Figure B.4: Resource utilization on the control node when creating a VM type 2 in OpenNebula. Disk utilization plateaus quickly reaching just above 10% utilization while CPU utilization peaks just below 15%. Memory utilization remains constant at 8% for the duration of the test.

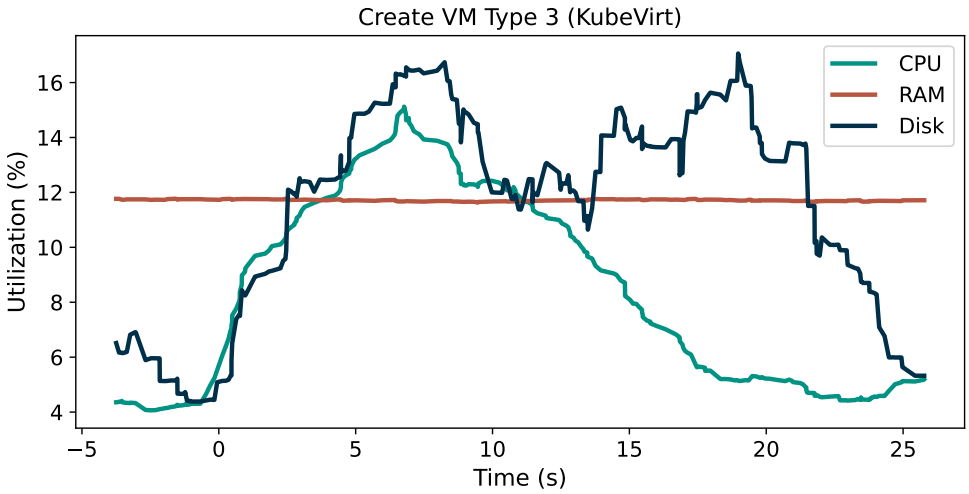

Figure B.5: Resource utilization on the control node when creating a VM type 3 in KubeVirt. CPU and disk utilization initially follow a similar pattern, but disk utilization remains high for the duration of the test. Memory utilization remains constant at 12% for the duration of the test.

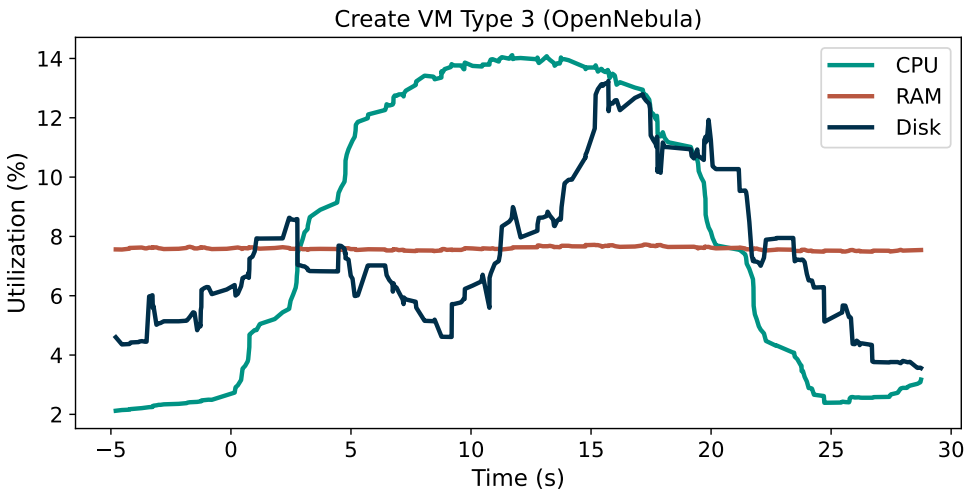

Figure B.6: Resource utilization on the control node when creating a VM type 3 in OpenNebula. CPU utilization is the most prominent among the resources reaching nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

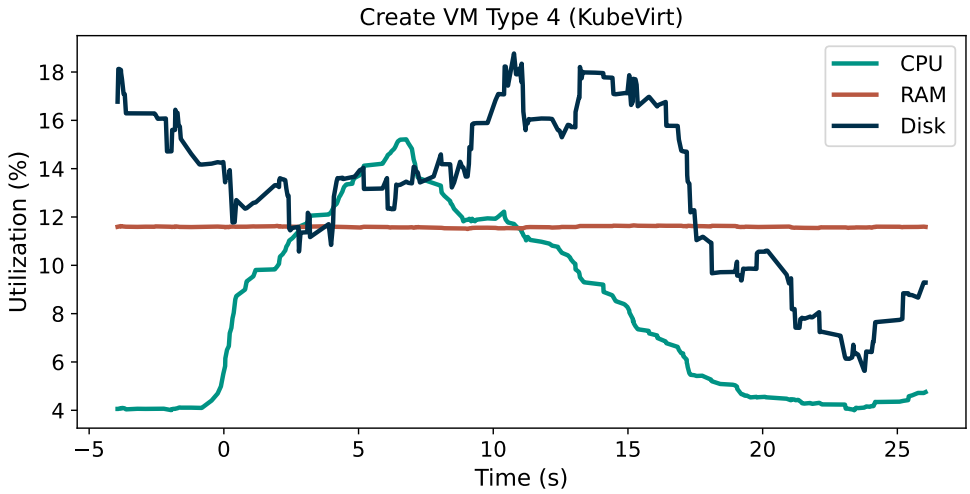

Figure B.7: Resource utilization on the control node when creating a VM type 4 in KubeVirt. CPU utilization peaks quickly while disk utilization is sporadic. The sporadic behavior is not in line with the operations performed during the test. Memory utilization remains constant at 12% for the duration of the test.

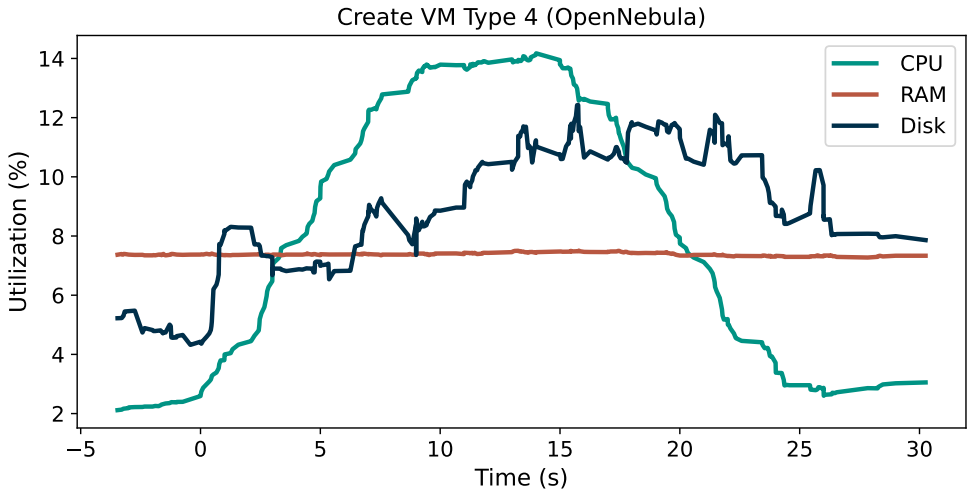

Figure B.8: Resource utilization on the control node when creating a VM type 4 in OpenNebula. CPU utilization is the most prominent among the resources plateauing at nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

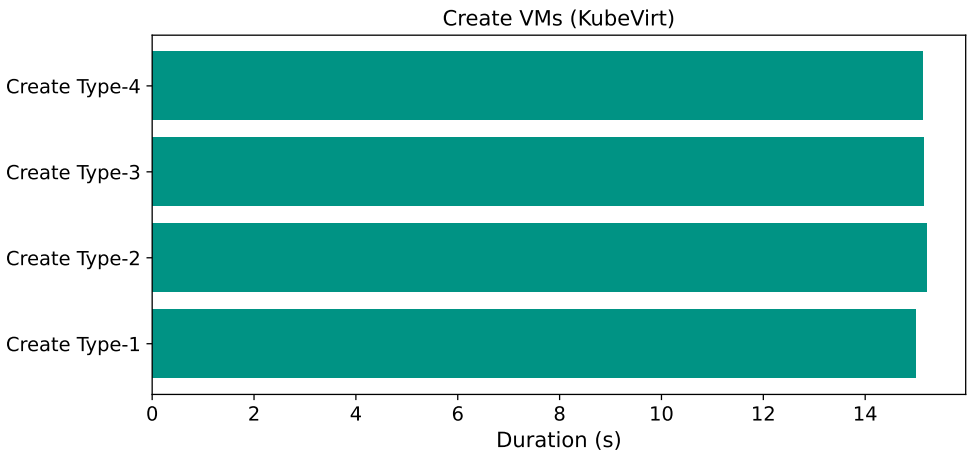

Figure B.9: Duration of creating VMs in KubeVirt. The duration for all types remains constant for each VM type reaching slightly above 14 seconds

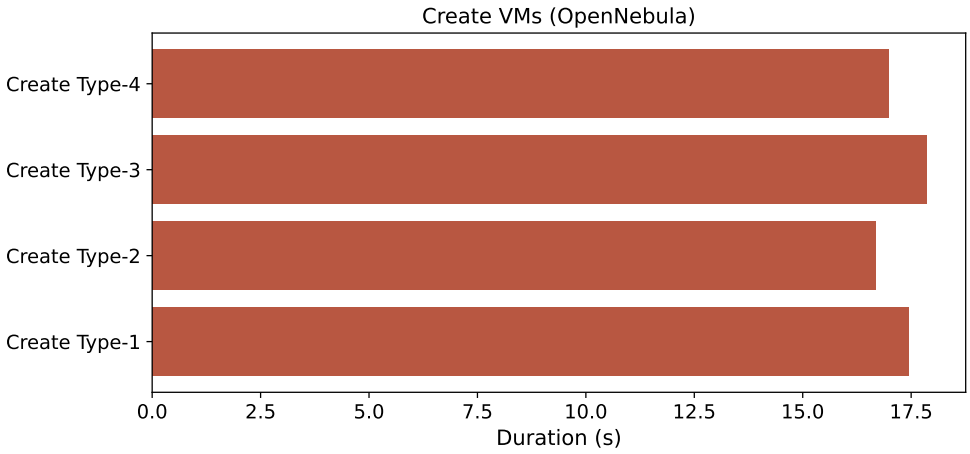

Figure B.10: Duration of creating VMs in OpenNebula. The duration for all types remains constant for each VM type reaching nearly 18 seconds.

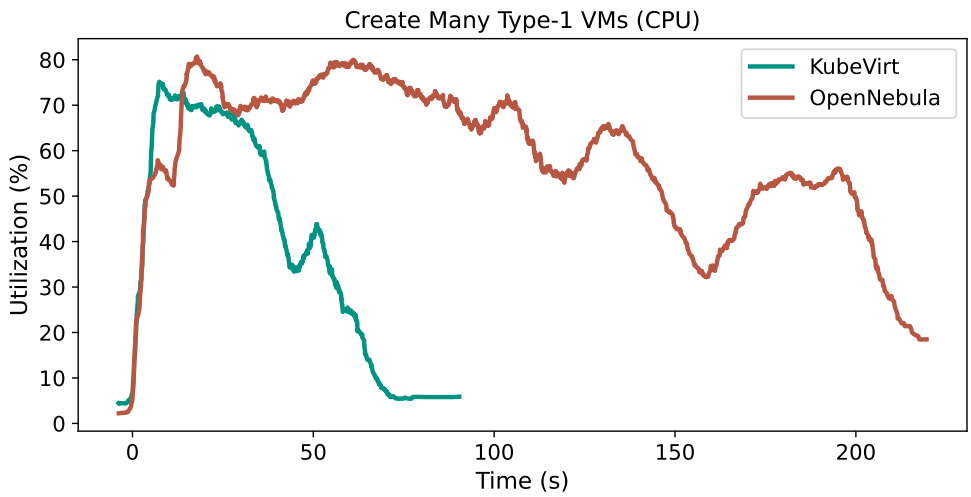

Figure B.11: CPU utilization on the control node when creating 20 type-1 VMs. The CPU utilization peaks quickly for both KubeVirt and OpenNebula due to the burst of requests. KubeVirt reaches a slightly lower peak than OpenNebula and subsides significantly faster. OpenNebula does not manage to return to a utilization level of an idle state within the duration of the test, as evidenced by the utilization not reaching base utilization in the end.

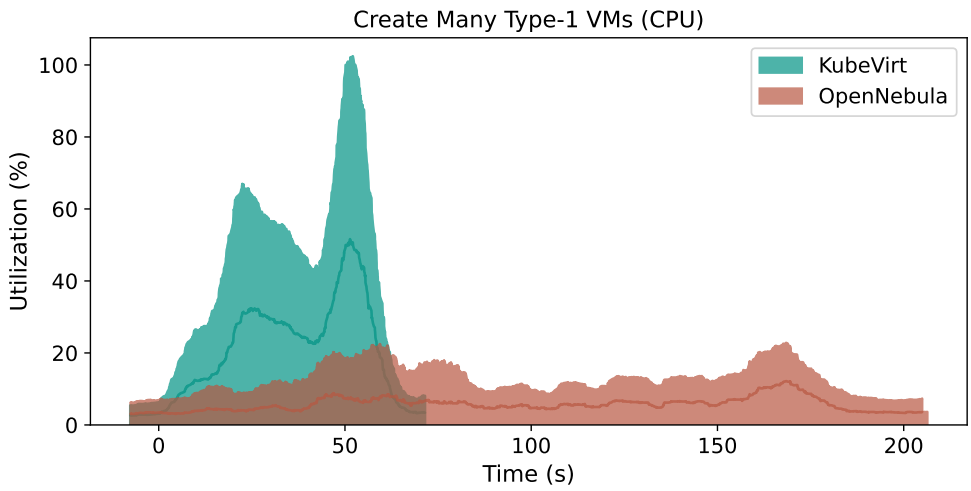

Figure B.12: Cumulative CPU utilization among worker nodes when creating 20 type-1 VMs. The CPU utilization among the worker nodes is vastly different between KubeVirt and OpenNebula, where KubeVirt experiences a short-lived large burst of CPU utilization while OpenNebula barely reaches a cumulative 20% utilization.

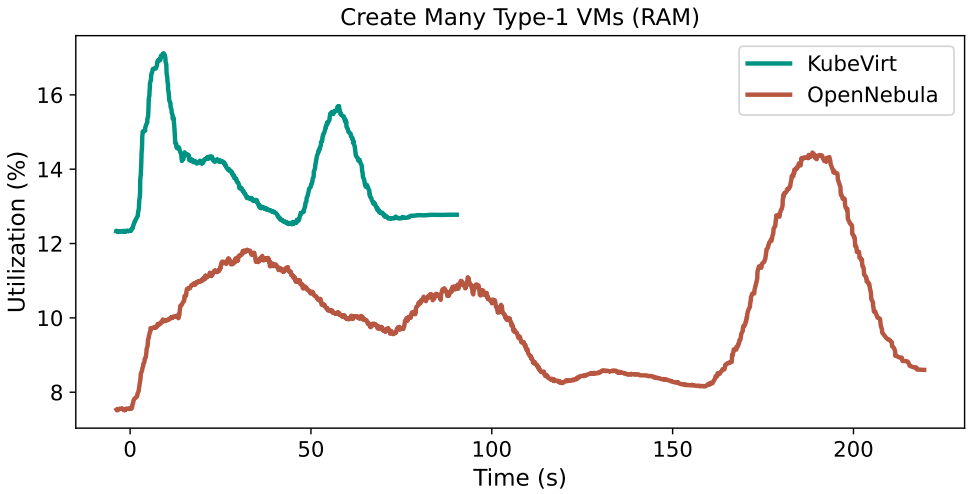

Figure B.13: Memory utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a burst of memory utilization in the beginning, while OpenNeubla has a more gradual increase before spiking quickly toward the end of the test. While KubeVirt has a larger base memory utilization, OpenNebula experiences a larger induced load from the test, as evidenced by its longer slopes.

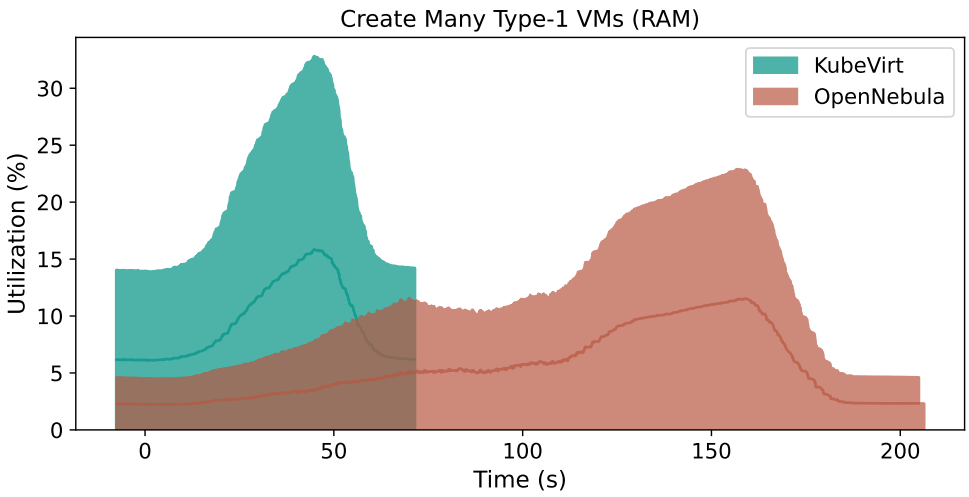

Figure B.14: Cumulative memory utilization among worker nodes when creating 20 type-1 VMs. While KubeVirt experiences a larger memory utilization due to a larger base, both systems have a similar magnitude of the induced load, albeit KubeVirt's load is more short-lived.

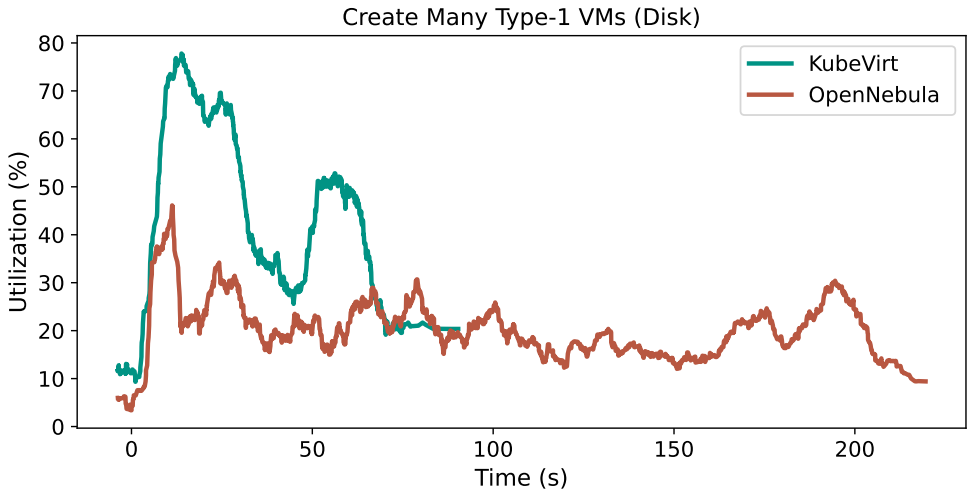

Figure B.15: Disk utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a steady disk utilization throughout the test.

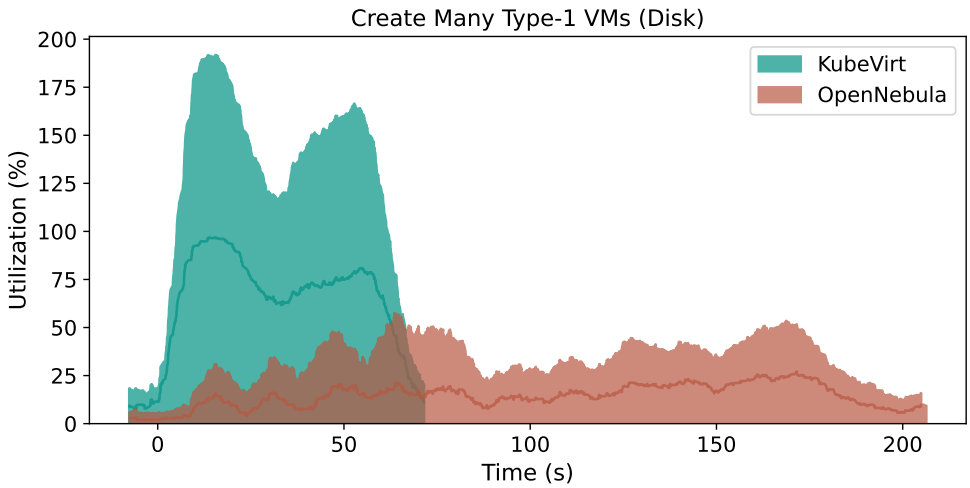

Figure B.16: Cumulative disk utilization among worker nodes when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a low and steady disk utilization throughout the test.

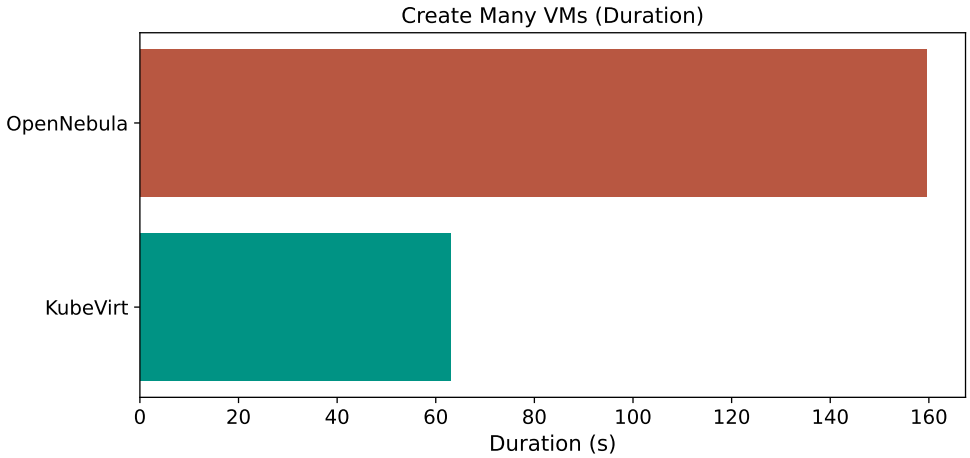

Figure B.17: Duration of creating 20 type-1 VMs. KubeVirt is significantly faster to create 20 type-1 VMs than OpenNebula, taking 67 seconds compared to OpenNebula's 192 seconds.

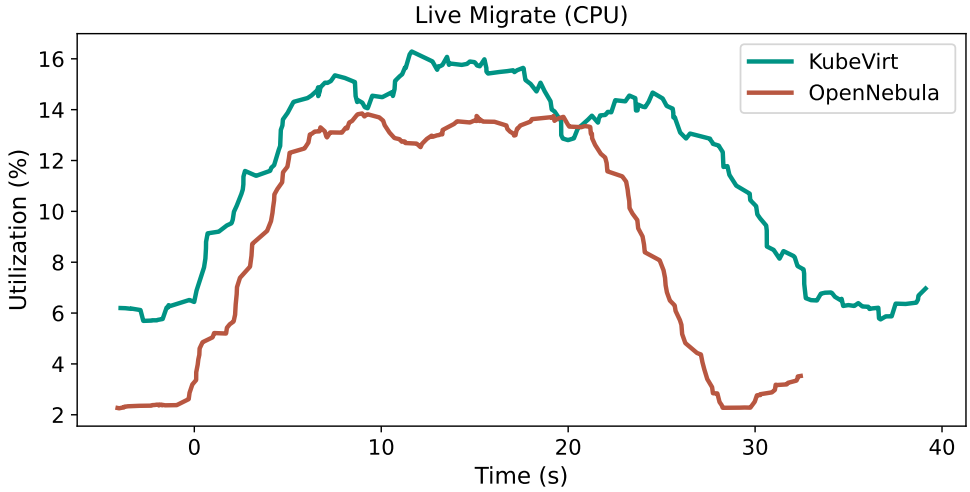

Figure B.18: CPU utilization on the control node when live migrating a type-4 VM. KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is slightly larger for OpenNebula than KubeVirt. They follow a similar pattern peaking at 16% and 14% respectively.

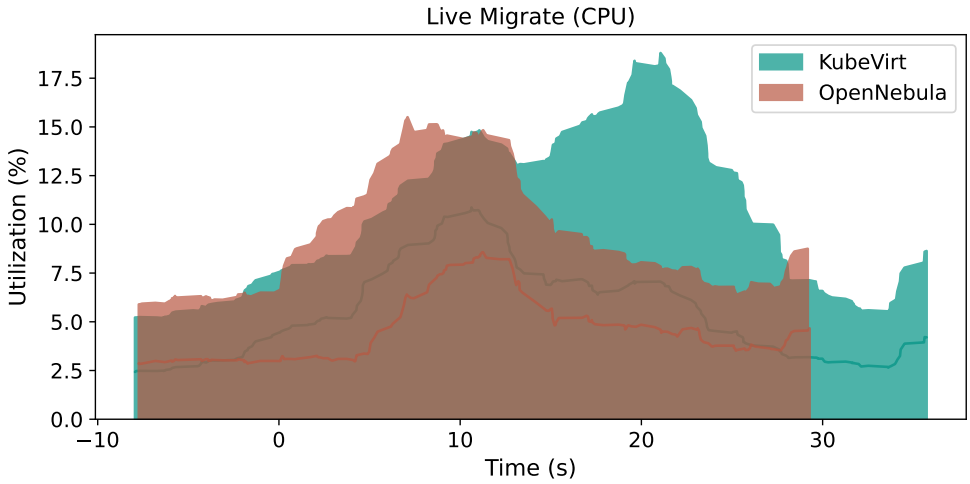

Figure B.19: Cumulative CPU utilization among worker nodes when live migrating a type-4 VM. Both systems follow a similar pattern where OpenNebula experiences a slightly lower utilization than KubeVirt. The induced load is shifted towards the end of the test for KubeVirt since it executes the operation more slowly.

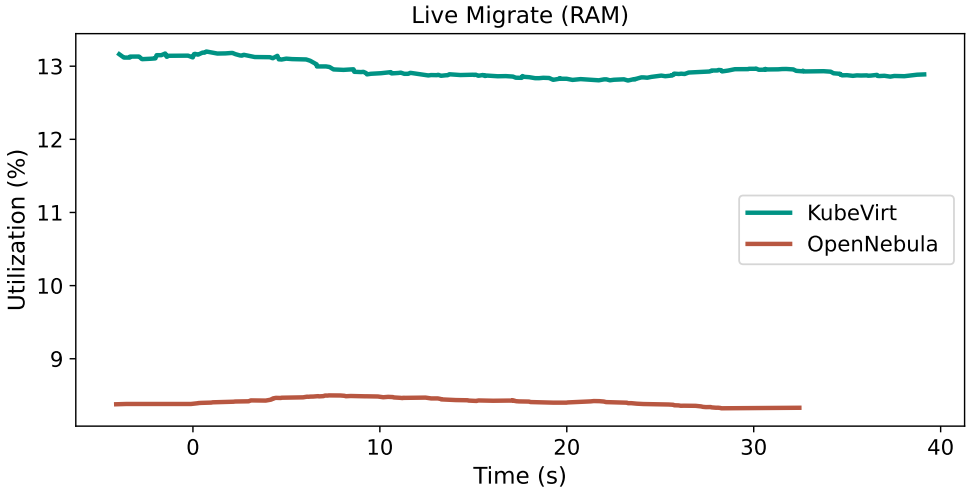

Figure B.20: Cumulative memory utilization on the control node when live migrating a type-4 VM. While KubeVirt has a larger base memory utilization than OpenNebula, the induced load is non-existent for both systems.

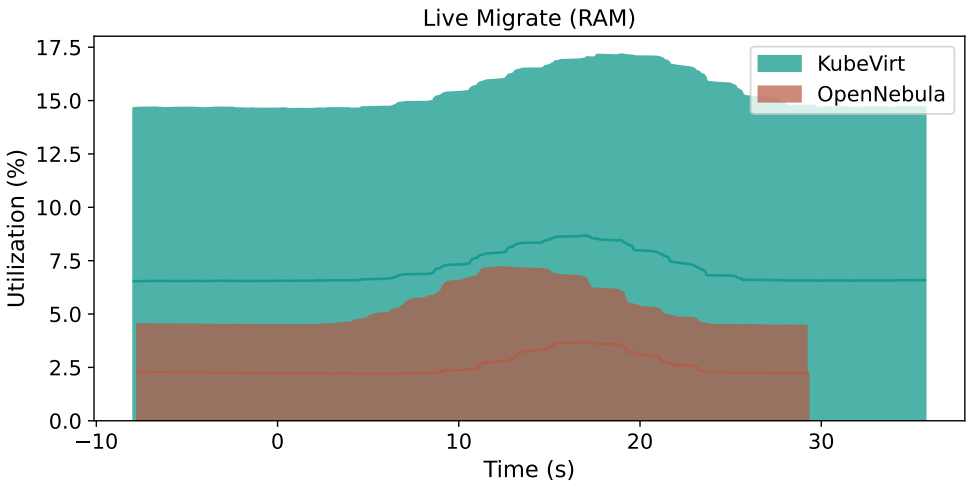

Figure B.21: Cumulative memory utilization among worker nodes when live migrating a type-4 VM. The load is similar for the two systems, where the load is shifted between the two workers over time. The overlap of the peaks between the workers shows that the migration process affects both workers simultaneously during a short period.

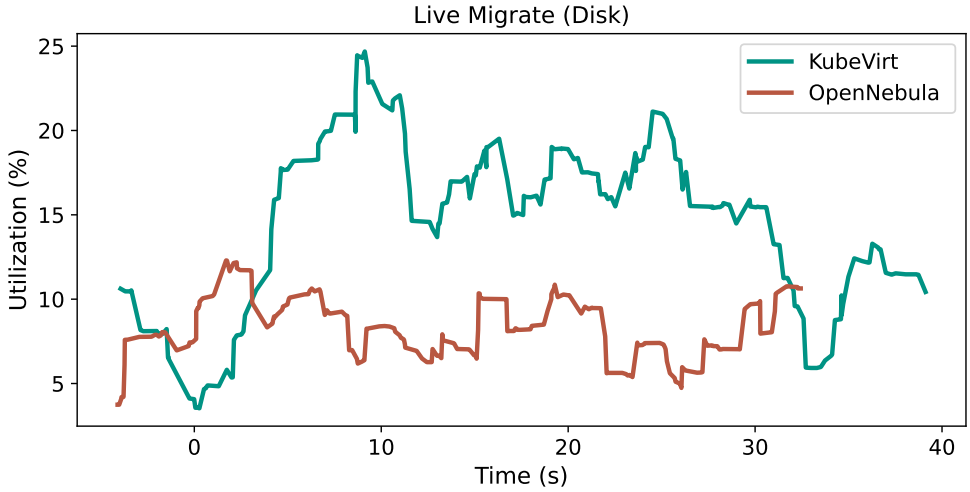

Figure B.22: Disk utilization on the control node when live migrating a type-4 VM. KubeVirt experiences a larger disk utilization than OpenNebula for the entirety of the test. However, the disk usage is sporadic and many peaks are not in line with the operations performed during the test.

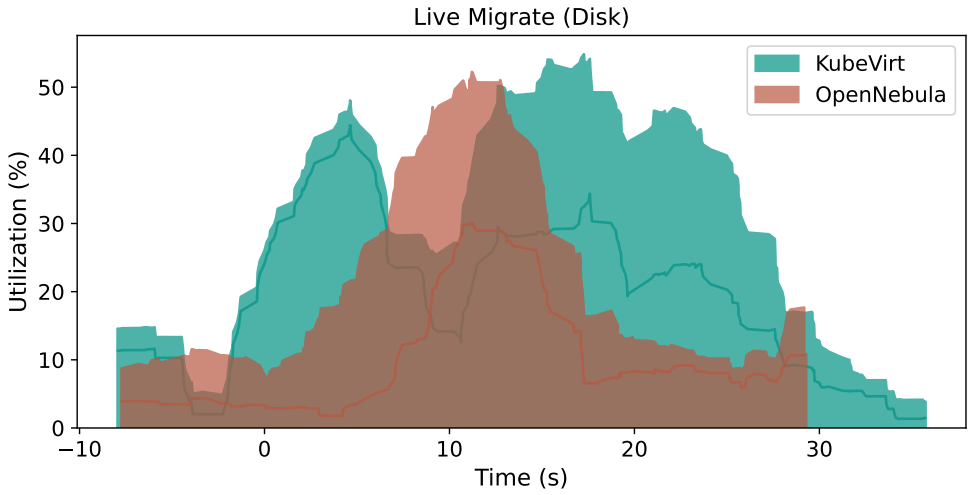

Figure B.23: Cumulative disk utilization among worker nodes when live migrating a type-4 VM. Both KubeVirt and OpenNebula reach a similar peak cumulative disk utilization. However, OpenNebula experiences a shorter plateau of disk utilization than KubeVirt.

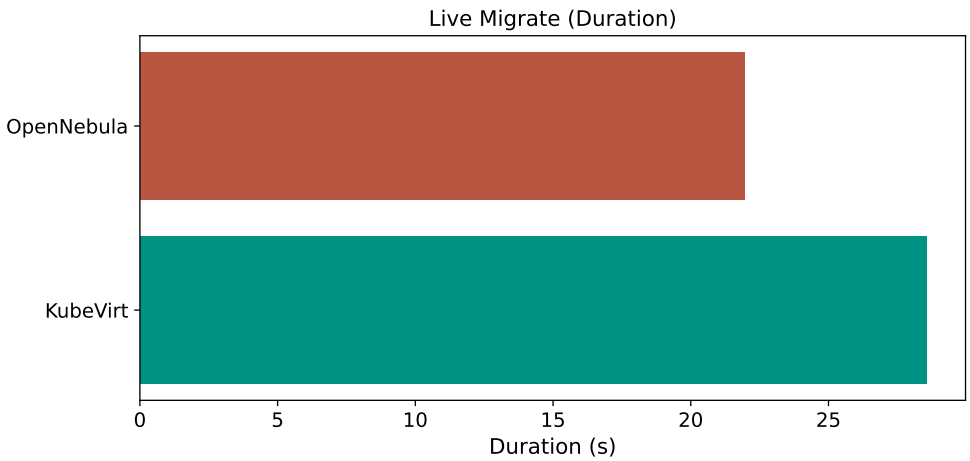

Figure B.24: Duration of live migrating a type-4 VM. KubeVirt is slower than OpenNebula, taking on average 7 seconds longer to complete. The duration is consistent for both systems.

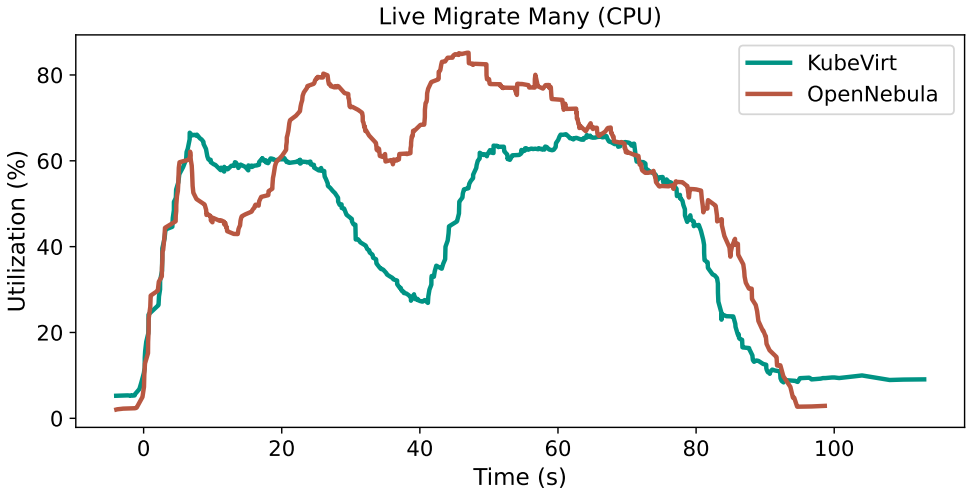

Figure B.25: CPU utilization on the control node when live migrating 10 type-1 VMs. Both systems follow a similar pattern where KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is larger for OpenNebula.

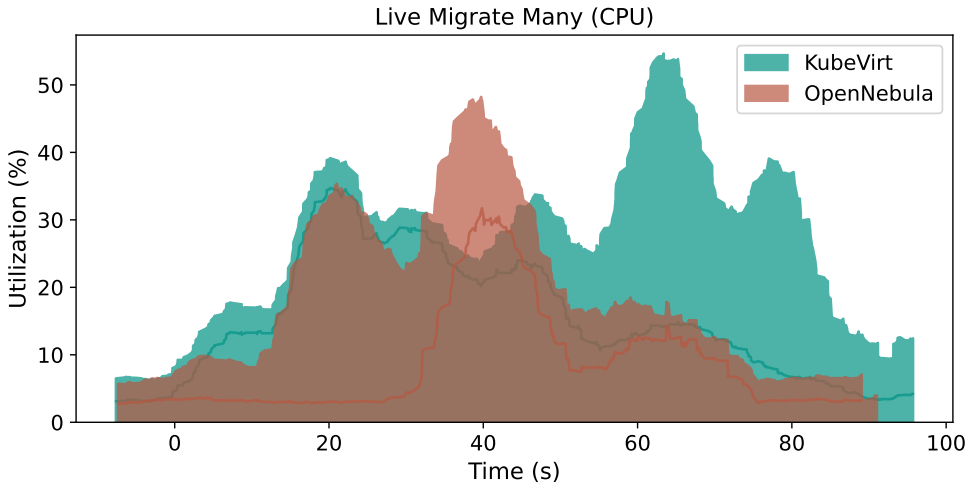

Figure B.26: Cumulative CPU utilization among worker nodes when live migrating 10 type-1 VMs. Both systems experience a similar load for the first half of the test, but KubeVirt experiences a significantly larger load in the second half of the test.

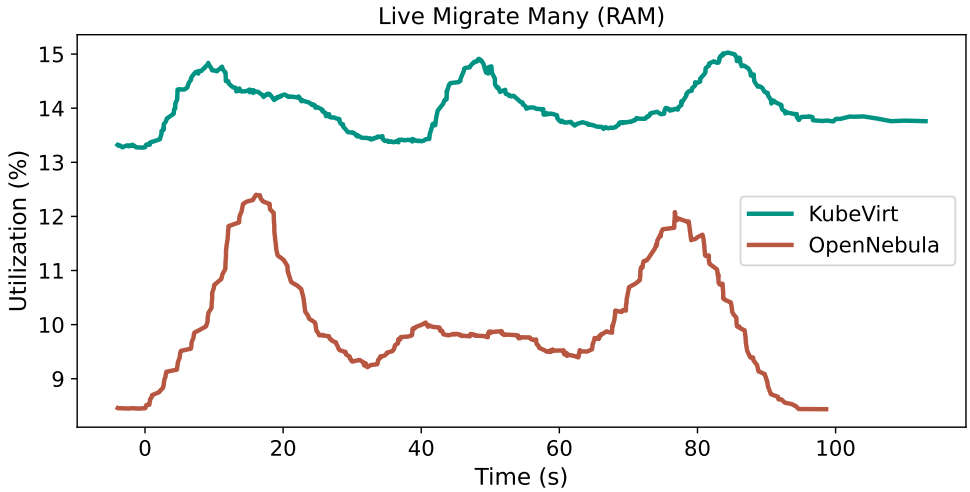

Figure B.27: Memory utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but the induced load is more than double for OpenNebula.

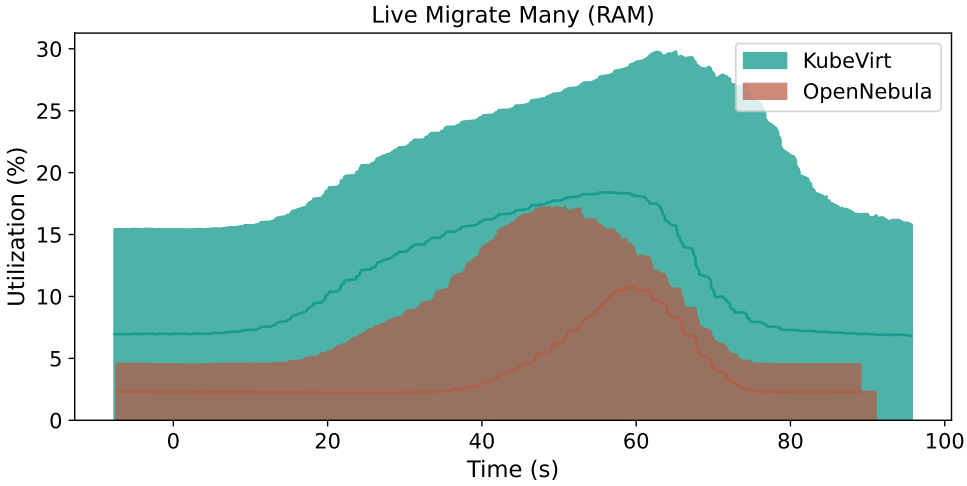

Figure B.28: Cumulative memory utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but both systems experience a similar induced load. OpenNebula returns to an idle state faster than KubeVirt.

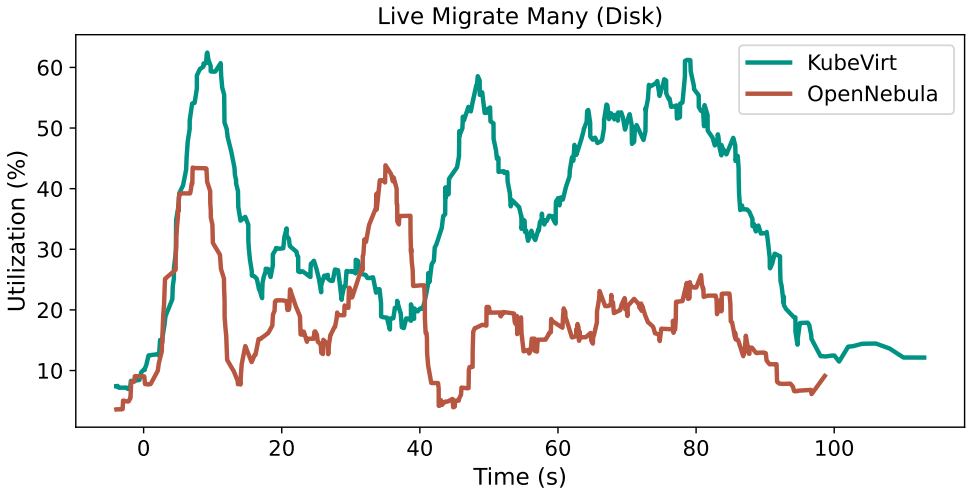

Figure B.29: Disk utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has an overall larger disk utilization than OpenNebula. However, the disk utilization has sporadic peaks that are not in line with the operations performed during the test.

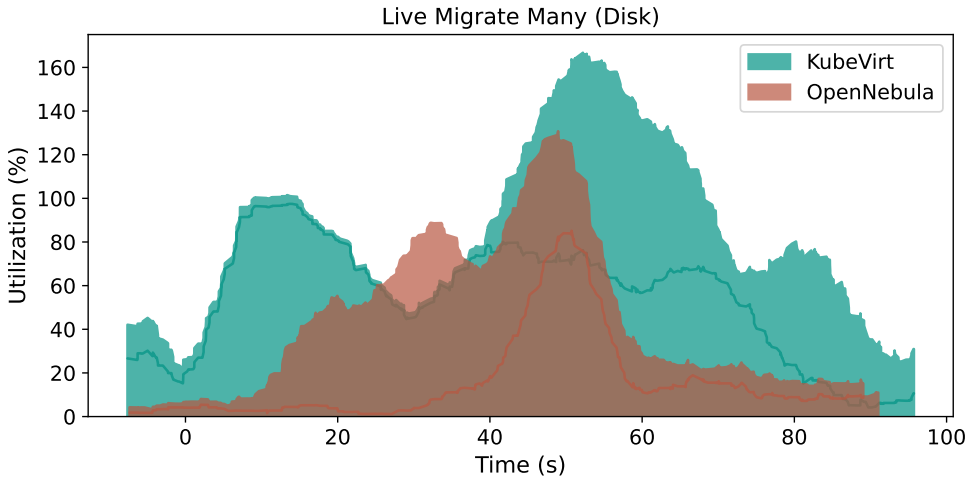

Figure B.30: Cumulative disk utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has both a larger base utilization and cumulative disk utilization than OpenNebula. Both systems show a clear pattern of the disk utilization being shifted to a second worker node after half the duration of the test. However, KubeVirt takes a longer time to return to an idle state of the first worker node than OpenNebula.

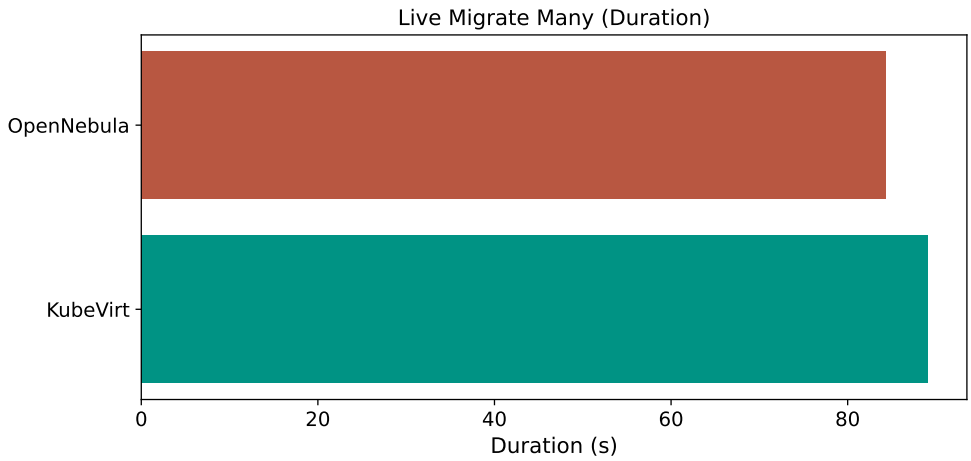

Figure B.31: Duration of live migrating 10 type-1 VMs. KubeVirt is slightly slower than OpenNebula, taking on average 3 seconds longer to complete, 87 seconds compared to OpenNebula's 84 seconds.

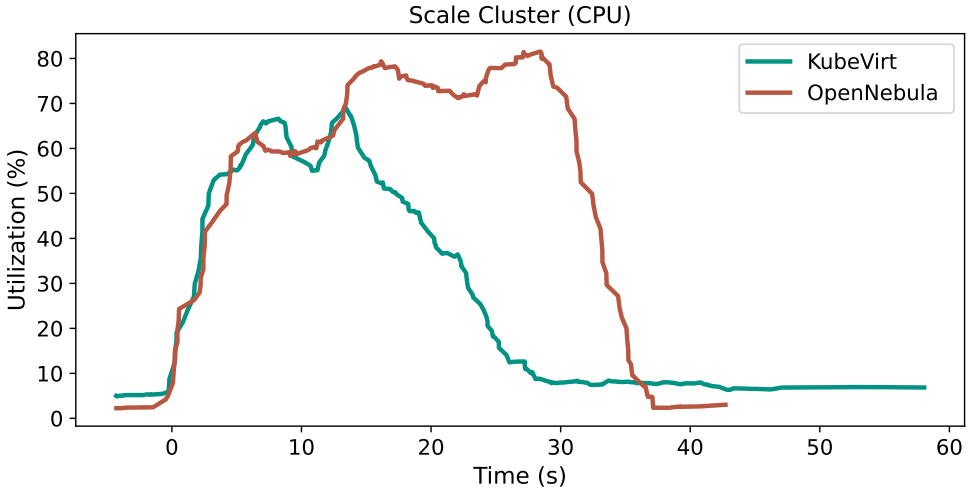

Figure B.32: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. Both systems are initially identical in CPU utilization, but KubeVirt subsides faster than OpenNebula.

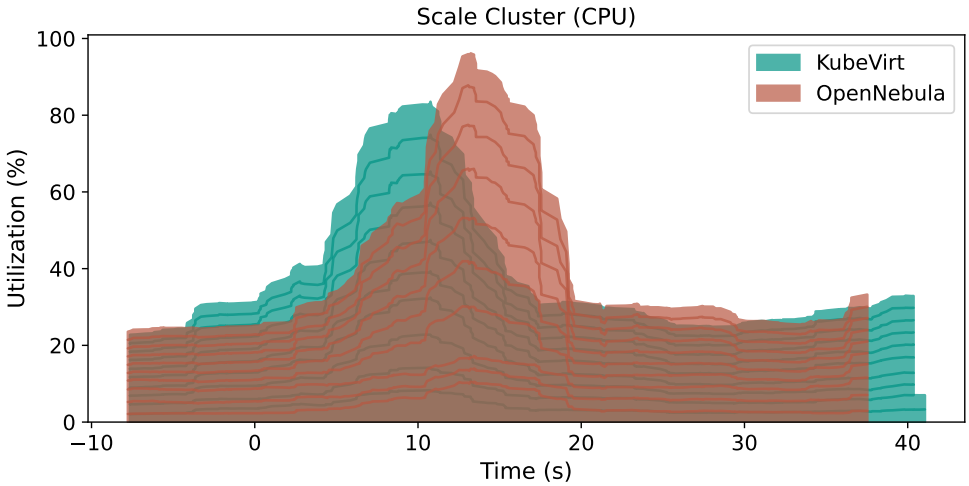

Figure B.33: Cumulative CPU utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. Both the base utilization and induced load are similar between the two systems. However, OpenNebula is slightly faster to complete the scaling operation than KubeVirt.

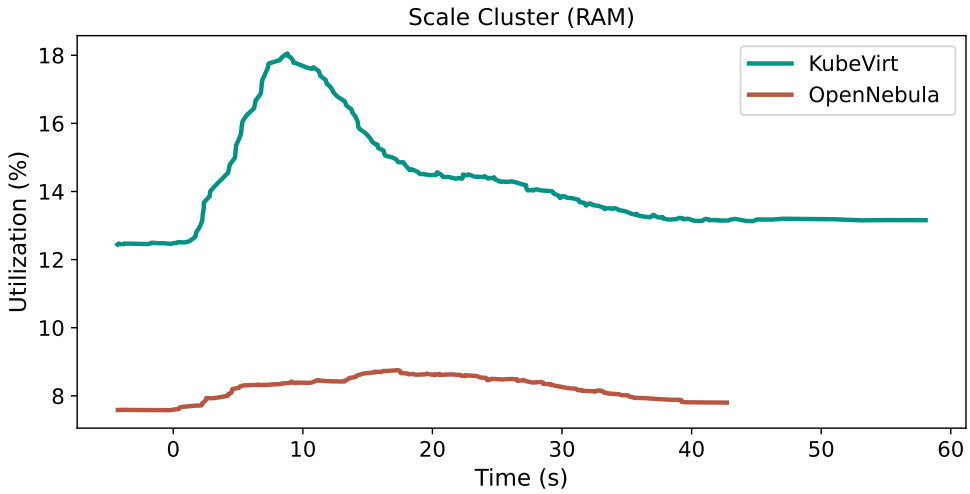

Figure B.34: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger memory utilization than OpenNebula. OpenNebula's memory utilization is barely affected by the scaling operation.

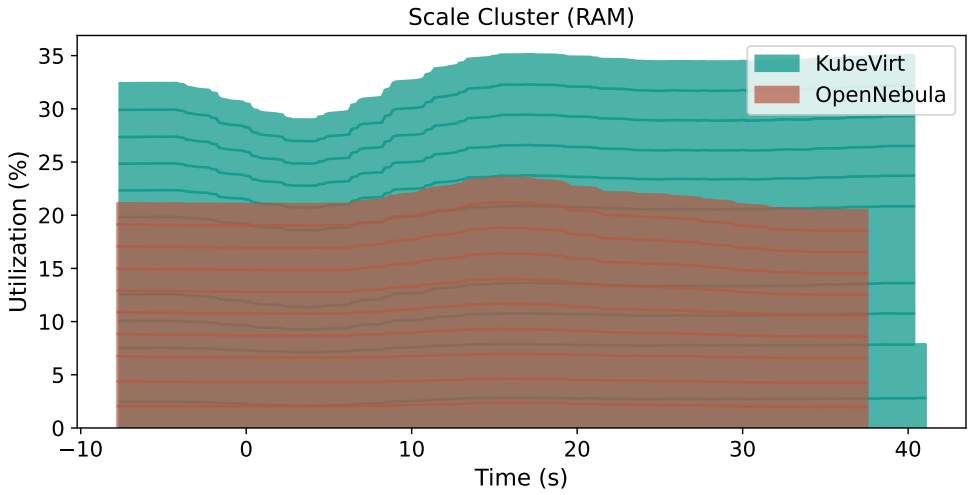

Figure B.35: Cumulative memory utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a slightly larger base memory utilization than OpenNebula, as well as a larger induced load from the scaling operation.

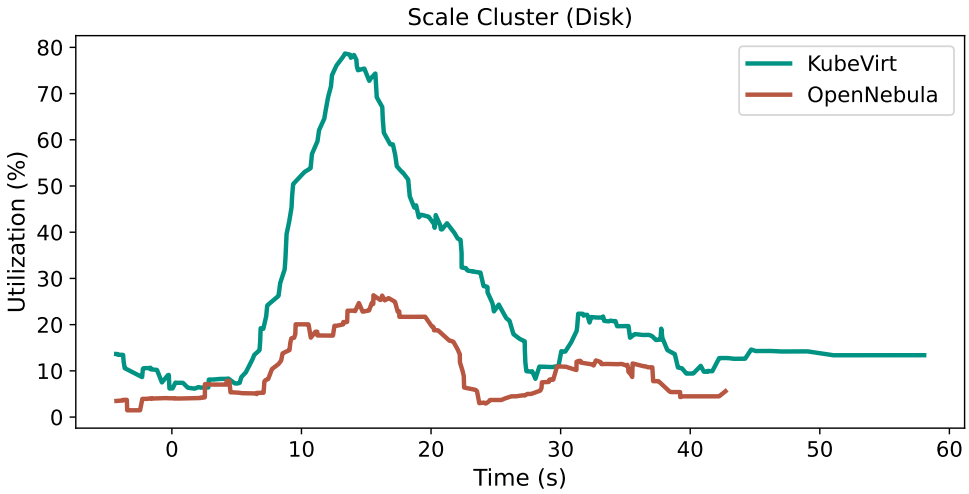

Figure B.36: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a rapid increase in disk utilization, reaching a peak of over 70% utilization. OpenNebula has a less aggressive increase reaching barely 30% utilization.

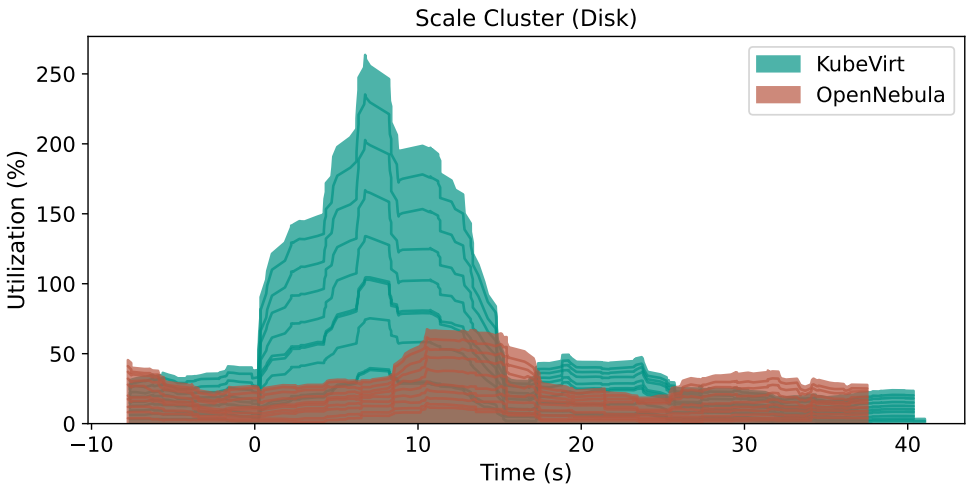

Figure B.37: Cumulative disk utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger disk utilization than OpenNebula with a burst of disk utilization whereas OpenNebula has a more gradual increase.

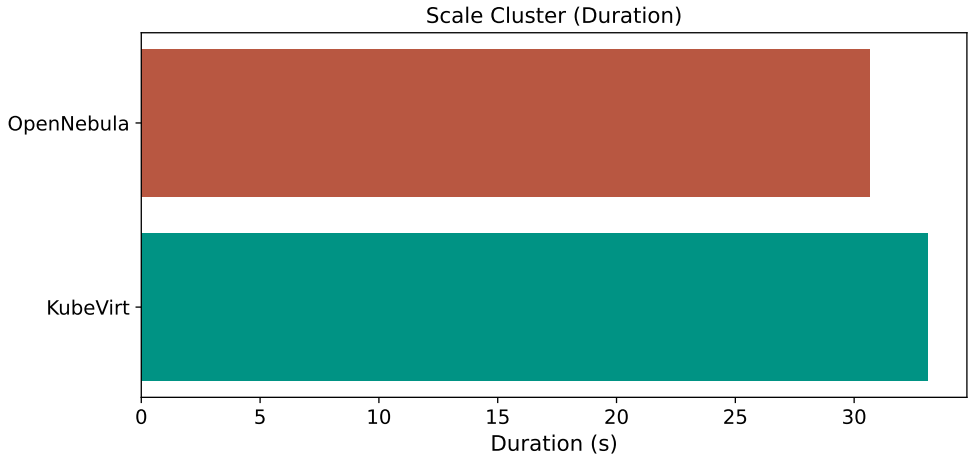

Figure B.38: Duration of scaling the cluster. KubeVirt is slightly slower than OpenNebula, taking on average 3 seconds longer to complete, 33 seconds compared to OpenNebula's 30 seconds.

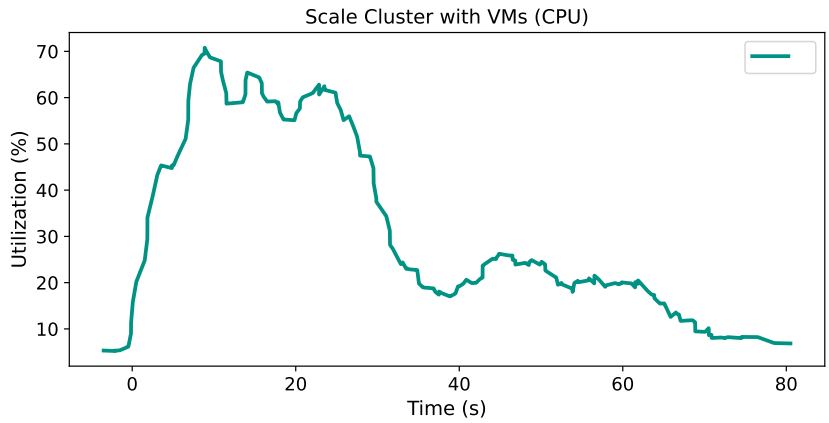

Figure B.39: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences an initial burst of CPU utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

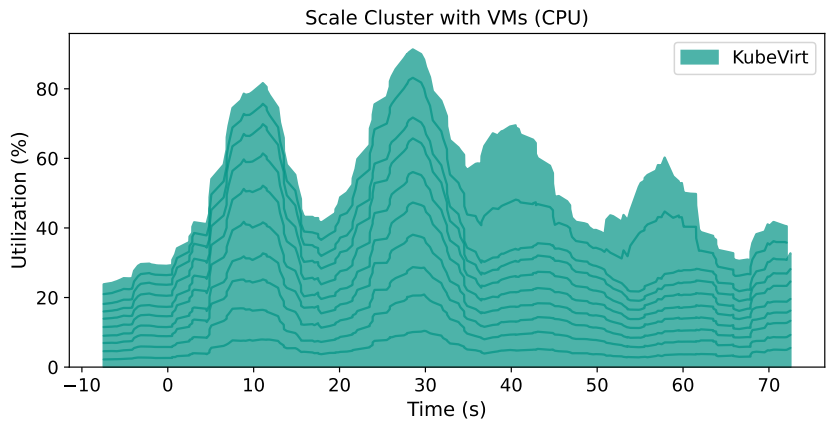

Figure B.40: Cumulative CPU utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt maintains utilization with periodic peaks throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

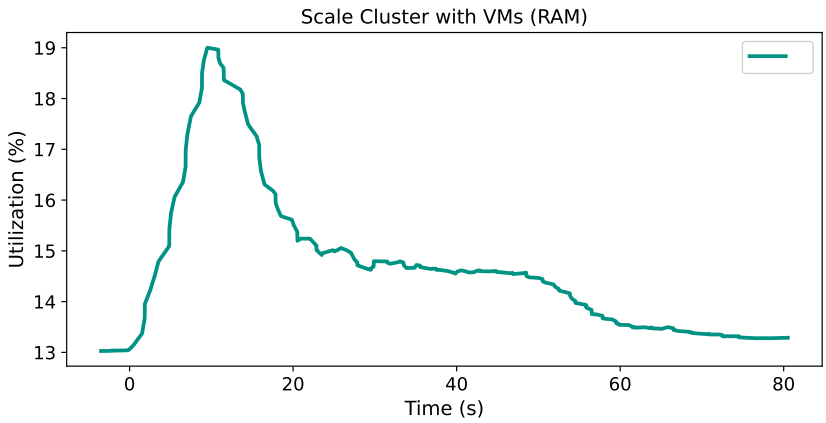

Figure B.41: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large burst of memory utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation that gradually decreases towards the end. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

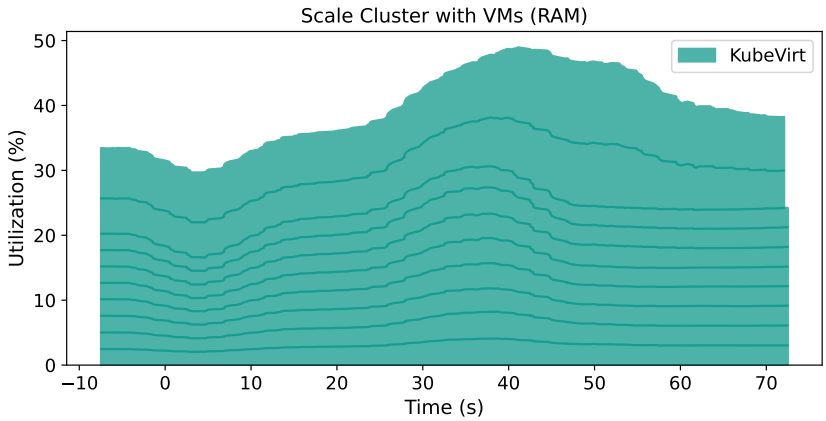

Figure B.42: Cumulative memory utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt maintains a steady memory utilization throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

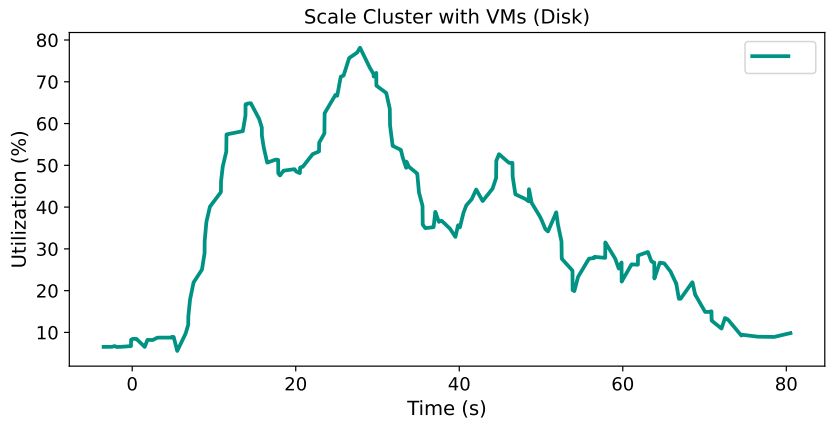

Figure B.43: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large initial burst of disk utilization that gradually subsides toward the end of the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

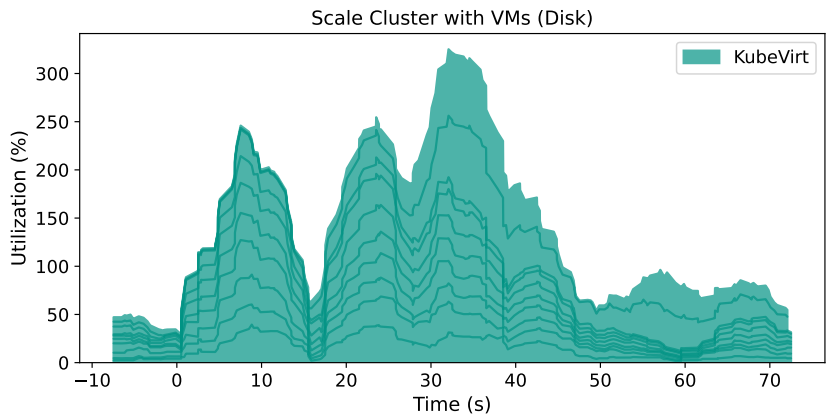

Figure B.44: Cumulative disk utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences large periodic peaks of disk utilization throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

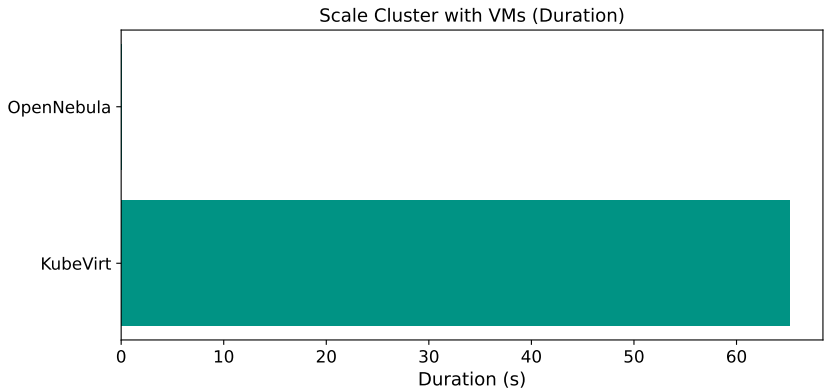

Figure B.45: Duration of scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt completes the test in just below 70 seconds. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

Appendix C

Usability Tasks

The following is a replicated version of what was given to the participants for the usability analysis formatted for the appendix. The original version was provided as a separate document to the participants.

Introduction

This is a usability analysis of KubeVirt that involves performing a set of tasks and then reflecting on the experience, where the goal is to evaluate how well KubeVirt works in practice. By participating in the usability analysis you provide valuable feedback and insight that can be used to improve KubeVirt. Your answers will be included as-is in the appendix of the project report. Upon request, the answers can be anonymized.

The following is a brief introduction to KubeVirt, its most relevant features and how it compares to previous VM management systems. If you are already familiar with KubeVirt and its machinery, you can skip this section.

KubeVirt is an extension to Kubernetes that allows for the creation of virtual machines (VMs) and is designed to provide a similar experience to managing any other resource in Kubernetes, such as Pods or Deployments. This means that VMs are defined in YAML files and integrate with the command line tool *kubectl*. KubeVirt defines its resources as a *Custom Resource Definition* (CRD) in Kubernetes, where the most prominent resources are *VirtualMachine* and *VirtualMachineInstance*. The relationship between them is comparable to that of a Deployment and a Pod, where the Deployment defines a desired state and a Pod represents the running state. See References below for more information.

Certain VM operations are not possible using kubectl, such as accessing the VM using SSH or VNC. For this reason, KubeVirt provides a complementary CLI tool called *virtctl*.

Being built on top of Kubernetes allows the VMs to integrate with other resources, including the Service and PersistentVolume resources. For instance, a VM can be exposed to the network using a LoadBalancer Service or manage storage dynamically using a persistent storage provisioner. However, since a VM requires a disk image (such as a QCOW2 image), it means that a PersistentVolume must point to a folder with a disk image in it. For this reason, manually creating a PersistentVolume for a VM can be tedious. Instead, KubeVirt primarily provides two alternatives for managing disks that are presented below. See the References section for more information.

- **containerDisk**: A disk that is created at every boot. It is not persistent and is similar to a container.
- **DataVolume**: A disk that is persistent and initialized when the VM is created. It uses PersistentVolumeClaim and a PersistentVolume under the hood where it injects a disk image.

Basic usage (Task 1/3)

The first task is about creating a VM using KubeVirt. You will need to create a VM and then access it using SSH or VNC. You can use a containerDisk for simplicity.

Manifests

Below is a minimal manifest for creating the VM using a containerDisk. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
```

```
    cpu:
      cores: 1
    devices:
      disks:
      - name: containerdisk
        disk:
          bus: virtio
      interfaces:
      - name: default
        masquerade: {}
    resources:
      requests:
        memory: 128M
  networks:
  - name: default
    pod: {}
  volumes:
  - name: containerdisk
    containerDisk:
      image: quay.io/kubevirt/cirros-
         container-disk-demo
```

Steps

1. Create a VM

```
kubectl apply -f my-vm.yaml
```

2. Access the VM through SSH or VNC (credentials vary depending on the image since no CloudInit is used)

```
virtctl ssh cirros@my-vm
virtctl vnc my-vm
```

3. Stop, start and restart the VM

```
virtctl stop my-vm
virtctl start my-vm
virtctl restart my-vm
```

4. Delete the VM

```
kubectl delete vm my-vm
```

Questions

After you have finished the tasks, reflect on the experience. You may use the following questions as a guide, but feel free to elaborate with your own thoughts.

- How intuitive was the process of creating and deleting a VM?
- How straightforward was it to access the VM using SSH and VNC?
- Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

Maintenance (Task 2/3)

The second task is related to maintenance, where you will create a snapshot of a VM and then restore it. You will also perform a live migration on a VM to move it from one node to another without shutting it down.

Both snapshots and live migrations are treated as a resource by KubeVirt, meaning that the operation will take place in the background after a VirtualMachineSnapshot resource or a VirtalMachineInstanceMigration resource is created. See the References section for more information.

A live migration resource in KubeVirt does not move resources itself and instead uses the Kubernetes scheduler to decide where to place the VM. This means that a LiveMigration resource only tells the scheduler that the VM can be moved but not where to move it. See the References section for more information. The Kubernetes scheduler is sophisticated and supports a wide range of scheduling strategies, but for this task, we can force a move between two nodes by using the *spec.template.spec.nodeSelector* in the VM and editing the labels of the worker nodes in the cluster. To not interfere with other participants, you should create a unique label for you.

A snapshot in KubeVirt works similarly to a snapshot in other VM management systems, where it captures the state of the VM at a specific point in time. However, the snapshot uses Kubernetes VolumeSnapshot, which is a Kubernetes API for taking snapshots of PersistentVolumes. This means that a VM with a containerDisk cannot be snapshotted. Instead, you should use a VM with a DataVolume. See the References section for more information.

Live Migration Manifests

The following is a minimal manifest for creating a VM that runs on a node with label **name: worker-1**. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      nodeSelector:
        # Edit this to use a label of your
           choice
        my-label: some-value
      domain:
        cpu:
          cores: 1
        devices:
          disks:
          - name: containerdisk
            disk:
              bus: virtio
          interfaces:
          - name: default
            masquerade: {}
        resources:
          requests:
            memory: 128M
      networks:
      - name: default
        pod: {}
      volumes:
      - name: containerdisk
        containerDisk:
          image: quay.io/kubevirt/cirros-
             container-disk-demo
```

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstanceMigration
metadata:
  name: my-vm-migration
spec:
  vmiName: my-vm
```

Live Migration Steps

1. Ensure worker node 1 has your label (Make sure to replace `my-label` and `some-value` with your own values):

   ```
   kubectl label node usability-worker-1 my-label=some
      -value
   ```

2. Create a VM and wait for it to start. The VM should start on **usability-worker-1**:

   ```
   kubectl apply -f my-vm.yml
   ```

3. (Optional) Check if the VM is running on worker node 1:

   ```
   kubectl get vmis
   ```

4. Create a LiveMigration resource or use virtctl:

   ```
   kubectl apply -f my-vm-migration.yml
   ```

   ```
   virtctl migrate my-vm
   ```

5. Add the same label to worker node 2 and remove it from worker node 1 (Make sure to replace `my-label` and `some-value` with your own values):

   ```
   kubectl label node usability-worker-2 my-label=some
      -value
   kubectl label node usability-worker-1 my-label-
   ```

6. (Optional) Check if the VM is now running on worker node 2:

   ```
   kubectl get vmis
   ```

7. Unlabel worker node 2

```
kubectl label node usability-worker-2 my-label-
```

8. Delete the VM

```
kubectl delete vm my-vm
```

Snapshot Manifests

The following is a minimal manifest for creating a VM with a DataVolume. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
        devices:
          rng: {}
          disks:
          - disk:
              bus: virtio
            name: datavolume-disk
          interfaces:
          - name: default
            masquerade: {}
        resources:
          requests:
            memory: 1Gi
      networks:
      - pod: {}
        name: default
      volumes:
      - dataVolume:
          name: datavolume
        name: datavolume-disk
  dataVolumeTemplates:
```

```
  - metadata:
      name: datavolume
    spec:
      pvc:
        storageClassName: azurefile-csi-kubevirt
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 5Gi
      source:
        registry:
          url: docker://quay.io/kubevirt/cirros-
             container-disk-demo
```

The following is a minimal manifest for creating a VirtualMachineSnapshot resource.

```
apiVersion: snapshot.kubevirt.io/v1alpha1
kind: VirtualMachineSnapshot
metadata:
  name: my-vm-snapshot
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: my-vm
```

The following is a minimal manifest for restoring a VM from a snapshot.

```
apiVersion: snapshot.kubevirt.io/v1alpha1
kind: VirtualMachineRestore
metadata:
  name: my-vm-restore
spec:
  target:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: my-vm
  virtualMachineSnapshotName: my-vm-snapshot
```

Snapshot Steps

1. Create a VM

   ```
   kubectl apply -f my-vm.yml
   ```

2. Create a snapshot of the VM

   ```
   kubectl apply -f my-vm-snapshot.yml
   ```

3. Wait for the snapshot to be ready:

   ```
   kubectl get vmsnapshots
   ```

4. Stop the VM

   ```
   virtctl stop my-vm
   ```

5. Restore the VM from the snapshot

   ```
   kubectl apply -f my-vm-restore.yml
   ```

6. Wait for the VM to be restored

   ```
   kubectl get vmrestore
   ```

7. Delete the resources

   ```
   kubectl delete vmsnapshot my-vm-snapshot
   kubectl delete vm my-vm
   ```

Questions

After you have finished the tasks, reflect on the experience. You may use the following questions as a guide, but feel free to elaborate with your own thoughts.

- How intuitive was the process of migrating a VM from one node to another?
- How straightforward was it to create and restore a snapshot of a VM?
- Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

Debugging (Task 3/3)

The third task is about debugging a VM that is not accessible through SSH. You will be presented with a VM manifest which, if deployed, will result in a VM that is running but cannot be accessed through SSH. Since the VM is running, you can access it through VNC using virtctl. You can use a containerDisk since the task is not related to persistent storage.

Manifests

The following is a manifest for creating a Ubuntu VM that is not accessible through SSH. Credentials for the Ubuntu image should be user **cloud** and password **cloud**.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
        devices:
          disks:
          - name: containerdisk
            disk:
              bus: virtio
          - name: cloudinit
            disk:
              bus: virtio
          rng: {}
        resources:
          requests:
            cpu: 2
            memory: 2Gi
      volumes:
      - name: containerdisk
        containerDisk:
```

```
      image: quay.io/containerdisks/ubuntu
        :22.04
  - name: cloudinit
    cloudInitNoCloud:
      userData: |-
        #cloud-config
        users:
        - name: cloud
          # Generated with 'mkpasswd -m sha
            -512 --rounds 4096'
          passwd: $6$rounds=4096$abc
          shell: /bin/bash
          lock-passwd: false
          chpasswd: { expire: False }
          sudo: ALL=(ALL) NOPASSWD:ALL
          ssh_authorized_keys:
          - ssh-ed25519 abc
        ssh_pwauth: True
```

Steps

1. Create a VM

   `kubectl apply -f my-vm.yml`

2. Access the VM through VNC

   `virtctl vnc my-vm`

3. Find out what the issue is.

4. Edit the VM manifest to fix the issue.

5. (Optional) Restart the VM (some changes require a restart)

   `virtctl restart my-vm`

6. Ensure the VM is accessible through SSH. You can use the *–local-ssh* flag if there is a problem with the SSH keys.

   `virtctl ssh cloud@my-vm`

7. Delete the VM

```
kubectl delete vm my-vm
```

Questions

After you have finished the task, reflect on the experience. You may use the following questions as a guide, but feel free to add comparisons with other VM management systems you have used.

- How easy was it to debug and find the issue?
- How easy was it to fix the issue?
- Does the debug process differ from debugging a traditional VM management system?
- Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

Summary

After you have completed the tasks, reflect on the overall experience of using KubeVirt. You may use the following questions as a guide, but feel free to discuss and compare them with other VM management systems you have used.

- How would you compare the experience of using KubeVirt with other VM management systems you have used?
- How intuitive is the workflow of KubeVirt? Can you use it without reading the documentation?
- How would you describe the overall experience of using KubeVirt?

Thank you for participating in the usability analysis of KubeVirt!

References

- **VirtualMachineInstance**: `https://kubevirt.io/user-guide/virtual_machines/virtual_machine_instances/`
- **Disks**: `https://kubevirt.io/user-guide/virtual_machines/disks_and_volumes`

- **Snapshots**: `https://kubevirt.io/user-guide/operations/snapshot_restore_api/`
- **Live Migration**: `https://kubevirt.io/user-guide/operations/live_migration/`

Appendix D

Usability Analysis Answers

D.1 Participants

- Participant 1: Linus
- Participant 2: Osvald
- Participant 3: Karl
- Participant 4: Sai

D.2 Task 1/3

How intuitive was the process of creating and deleting a VM?

- **Linus**

  The act of doing *kubectl apply -f* and *kubectl delete* is quite neat, but the VM definition is not very neat. It would be quite difficult for me to figure out I need to use *bus: virtio* without having someone else tell me I need to. The *virtctl create vm* interface on the other hand seems to be much easier to use. As referenced from the official documentation.

  At this point, I decided I wanted to try to launch a Debian VM.

  Using *virtctl create vm* however, it was not clear which container disks are generally available or how I can create one (*virtctl create vm –help* and the help for *–volume-containerdisk*). What I'd like is an explanation of what that flag means.

What I'm searching for now is how to create a Debian VM, looking at `https://quay.io/organization/kubevirt` where the example provided was I could not find anything called *debian*. Just trying *virtctl create vm | kubectl apply -f -* did not provide a working VM either, so I can't just modify that to work as I want.

The documentation referred to *quay.io/containerdisks*, which did have some more useful disks but not Debian (only Ubuntu, Fedora and Centos).

Thought: *containerDisks are not a good solution for any workload that requires persistent root disks across VM restarts, so a VM with a containerDisks is probably not what I want for my bog-standard VM.*

At this point, I thought I would just try to look at the standard KubeVirt documentation, hoping that they mention how you run a VM that isn't just a demo. What I found were two guides:

1. KubeVirt Lab 1 (`https://kubevirt.io/labs/kubernetes/lab1.html`)
   - Did not mention anything on how to run my VM.
2. KubeVirt Lab2 (`https://kubevirt.io/labs/kubernetes/lab2.html`)
   - This might be what I want but it is not clear what a CDI is nor what I should use it for, even after clicking through to `https://github.com/kubevirt/containerized-data-importer`.
   - The first step of the guide is to install an operator, which I don't think I want to do just to get started.
   - As an aside it is a bit worrying that they have written this:
     * If the importer pod completes in error, you may need to retry it or specify a different URL to the fedora cloud image. To retry, first delete the importer pod and the DataVolume, and then recreate the DataVolume.
     * That means I can't rely on just creating the resource and having it do what I want. I need to monitor it with my retry logic to be sure it works.
   - This guide specifies how I can provide my SSH keys via cloud-init, which is nice.
   - Trying to follow that guide gave me this error when I ran *kubectl describe datavolumes.cdi.kubevirt.io*

    * Normal ErrClaimNotValid 48s datavolume-import-controller PVC spec is missing accessMode and no storageClass to choose profile
    * While this doesn't seem like an unsurmountable problem I decided to stop here.

- **Osvald**

  Very easy! I like that it's as simple as creating a YAML file and then applying it. Much simpler than any other way I've tried for managing VMs.

How straightforward was it to access the VM using SSH and VNC?

- **Linus**

  Quite, I would have loved to easily be able to integrate the SSH proxy command into my SSH config. Also, I find *virtctl console* nicer than VNC.

- **Osvald**

  Very simple! Simply doing *virtctl ssh cirros@my-vm* and then typing in the password and then I was in!

Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

- **Linus** I could not figure out how to create a Debian image. The documentation was very lacking as it seemed geared towards how to operate a KubeVirt cluster, not how to use one.

  From the documentation, it also seems that KubeVirt is plagued by the same issues a lot of the Kubernetes ecosystem is: a lot of strange abstractions that aren't well explained (CDI, DataVolume, containerDisks etc.).

- **Osvald**

  Getting VNC to work on Mac was a bit tricky, but I downloaded Tiger VNC and saved the script from (`https://github.com/kubevirt/kubevirt/issues/812#issuecomment-391832335`) as */usr/local/bin/remote-viewer*.

Summary

- **Karl**

  Everything was smooth sailing (except when I didn't read the instructions properly and missed the provided password). This process for managing VMs is much more direct, transparent and reproducible than using a UI-based tool.

D.3 Task 2/3

How intuitive was the process of migrating a VM from one node to another?

- **Linus**

  Not very, it was unintuitive that the migration succeeded even though nothing was migrated, even though it should have been (I used a *nodeSelector* that didn't exist, so it should not have been running on any node).

  I also found it unintuitive that I needed to create the migration resource after everything was correctly labeled. I would have expected that just updating the *nodeSelector* would move the VM.

- **Osvald**

  Very simple! I once again really like that you just edit YAML files and apply labels to conveniently move VMs between worker nodes. Much simpler than our current way of doing it in OpenNebula.

Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

- **Linus**

  The documentation mentions that I should look at the status of the VMI, which I did not think to do (regarding the intuitive question above).

  Had I read the documentation I would at least have known that the migration decided to do nothing when I used a non-existent *nodeSelector*.

  Once again the documentation deals a lot with how to operate the migration infrastructure and not anything on how one would do a migration operation. Granted it is listed under the *Operations* section of the documentation but I found nothing when searching under the *Virtual Machines* section where I would have expected it to be.

- **Osvald**

  I had to delete the entry from */.ssh/kubevirt_known_hosts* after each time I restarted the VM because *virtctl ssh* complained about a key mismatch.

Summary

- **Linus**

  I found this unintuitive as if it couldn't migrate (nothing matching the *nodeSelector*) I would have expected the migration to fail, not succeed.

  Once again this is also a bit strange, that I have to remove the migration resource and recreate it for something to happen. That means I can't just create a resource and trust KubeVirt will manage it but will need to have logic around the migration.

D.4 Task 3/3

How easy was it to debug and find the issue?

- **Linus**

  Not very, I did not manage to solve all the issues.

  The password issue was easy as not being able to log in was very obvious with *virtctl console*. The SSH issue I solved by accident, the *known-hosts: key mismatch* made me look into the *.ssh/authorized_keys* file and updated that with *virtctl console*. It didn't solve the key mismatch issue but it did allow me to continue with local SSH. I did not manage to solve the key mismatch issue as there is very little in the way of debugging *virtctl ssh*.

- **Osvald**

  Easy! Using *virtctl vnc* I could change the password and then try logging in.

How easy was it to fix the issue?

- **Linus**

  Two of the issues were easy to fix, I appreciated that *virtctl restart* worked as I did not think cloud-init would update the configs.

- **Osvald**

  Very easy, I just generated a new password.

Does the debug process differ from debugging a traditional VM management system?

- **Linus**

  I'm not sure what a traditional VM management system refers to. It is quite similar to the OpenNebula command lines. I appreciated that *virtctl console* was so easy to use.

- **Osvald**

  I've used IPMI which is similar to the VNC session in this case.

Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

- **Linus**

  I could not (easily) find documentation about how *virtctl ssh* worked so I could not figure out one of the issues. There seems to be no documentation for *virtctl*. Is it the same *virtctl* as the one from OpenShift? The fact that *kubectl ssh* broke without even a verbose option is worrying.

- **Osvald**

  The most difficult thing was to get the VNC viewer working on Mac, but after that, it was smooth sailing.

Summary

- **Linus**

  A "key mismatch" error is difficult to debug because there are no good debug options for *virtctl ssh*.

- **Karl**

  Debugging the issue was easy because the manifest uses standard constructs that are easily googleable.

D.5 Summary

How would you compare the experience of using KubeVirt with other VM management systems you have used?

- **Linus**

  I like the *virtctl* tool (when it worked) and like the idea of declaring my VMs. However, the system is very complicated with a lot of parts that it isn't clear how they fit together.

  The documentation is also very lacking. The fact that you manage the system with two different tools, both *virtctl* and *kubectl* is unnecessary and confusing. I find it telling that I didn't even manage to run a Debian VM.

- **Osvald**

  It seems to be much easier to deploy and manage machines! I want to start using this in production for our build pipelines and internal clusters.

How intuitive is the workflow of KubeVirt? Can you use it without reading the documentation?

- **Linus**

  Not at all. I would not have been able to use it without the documentation. I could barely use it with the documentation.

- **Osvald**

  Well, I had the example files you provided which were simple. But the CLI tools also gave good help.

How would you describe the overall experience of using KubeVirt?

- **Linus**

  The idea is good but the implementation is severely lacking. I could see the potential and it seemed very promising but from this experience, KubeVirt did not deliver.

  One could probably build something sensible with it but I feel the barrier of entry is very high.

- **Osvald**

  Great! I want more!

- **Karl**

  For developers with previous Kubernetes experience, it looks to be a very intuitive way to handle virtual machines. In an environment where Kubernetes clusters are already part of the workflow, it is obvious how aspects such as automation and access control can be transferred to apply also to the VM infrastructure.

  Since I was just trying it out, I haven't had any real issues that I need to debug yet.